

Politecnico di Torino

Politecnico di Torino

Master's Degree in Computer Engineering

A.Y. 2025/2026

Graduation Session December 2026

GroundOne: A Specialist Decision Model for Groundedness Verification in Local Retrieval-Augmented Generation

Fine-tuning compact language models as calibrated single-forward-pass verifiers on Apple Silicon

Supervisor: Prof. Paolo Garza

Candidate: Francesco Pio De Dominicis

Summary

Retrieval-augmented generation (RAG) is the standard way to give language models access to private documents, and running the whole pipeline on the user's own device is now practical on consumer hardware, which helps with privacy, offline use and cost. Retrieval does not, however, eliminate hallucination: generated answers still contain sentences that the retrieved context does not support or that contradict it. Detecting them is a prerequisite for trustworthy local assistants, and since the check must run on every answer, it has to fit on the same constrained machine as the generator.

Locality is the backbone of this work, and it has a precise technical consequence. On a single device, reading text is cheap and writing it is expensive: the model reads its input in a single forward pass, that is, one pass through all its layers, limited by compute, but generates its output one token at a time, each token limited by the memory bandwidth needed to read all the weights. On the laptop used here, writing a token costs fourteen to sixteen times more than reading one, and a review of current local hardware, from phones and laptops to dedicated machines such as Apple's Mac Studio and NVIDIA's DGX Spark, shows the same asymmetry everywhere, with new hardware accelerating reading much more than writing. The best open-weight models now trail the closed frontier by only a few months, and those that fit on a consumer graphics card by 6–12 months, so capable local pipelines are realistic; the question is how to verify them at a cost the device can bear.

Verification currently follows two paradigms. **Encoder-based checkers**, such as AlignScore, MiniCheck and HHEM, are small and fast, but return one fixed score per pair of document and sentence. **LLM judges**, by contrast, are flexible but write their verdicts token by token, so their cost grows with what they write; small LLM judges often break the output format, and the confidence they express is rarely usable as a probability. A third family has recently emerged: **decision models**, such as the proprietary Jev and the open SemIf and Rizzo-Flow, answer closed questions in a single forward pass by reading the probability of each answer option from the logits. The evidence for them is mostly reported by their authors, and the open implementations are general-purpose. Whether a compact decision

model specialised in one task can match LLM judges and dedicated encoders, where its advantage comes from, and whether its probabilities are usable, all on consumer hardware, had not been studied in a controlled way.

This thesis designs, trains and evaluates **GroundOne**, a decision model specialised in one decision: for each sentence of a RAG answer, whether the retrieved document supports it, contradicts it or does not mention it. A marker follows every sentence of the answer, and the verdicts for all K sentences are read from the logits of three answer letters in one forward pass, without generating any text. Qwen2.5 models with 0.5B and 3B parameters are fine-tuned with LoRA on the human-labelled answers of RAGTruth, with a class-weighted loss restricted to the three letters; a temperature and a bias fitted on development data calibrate the probabilities, and a threshold chosen on the same data turns them into consistent labels and strict or lenient flags. Everything, from building the dataset to training, evaluation and serving, runs on one Apple M4 laptop with 16 GB of memory, with MLX and no external service, and every baseline is run on the same machine. The evaluation uses three test sets built and frozen before training (the RAGTruth test split, LLM-AggreFact and VitaminC) and compares GroundOne with cues that never read the document, the untrained models, LLM judges of the same size with and without probabilities, two general-purpose decision models and specialised encoders, reporting cluster-bootstrap intervals.

On the RAGTruth test set GroundOne reaches a three-way balanced accuracy of 0.769 with 0.5B parameters and 0.832 with 3B, against 0.37 and 0.44 for the untrained models (for the 3B on a 600-answer subsample); its AUROC (0.905 and 0.940) exceeds that of the encoders (about 0.82), and in three-way balanced accuracy it clearly outperforms both general-purpose decision models: Rizzo-Flow, eight times larger than the 0.5B, and Liquid AI's d1-3B, of the same size as the 3B, which stops at 0.657 and in turn is much stronger on the out-of-domain VitaminC test. On LLM-AggreFact the 3B is on par with MiniCheck-RoBERTa (74.8 against 74.0), above the published zero-shot 7B model and below the closed decision model Jev as measured independently (78.6). When an answer contains fourteen or more sentences ($K \geq 14$), GroundOne is 1.7 to 2 times faster than a terse LLM judge and about six times faster than a verbose LLM judge of the same size and, unlike the LLM judges, always produces a valid output. Net of reading the text, which every approach pays, its decision is 18 to 33 times faster than that of the terse LLM judge, and when the document has already been read, as in the verification of a rewritten answer, reusing the KV cache brings the overall advantage to 3.4 to 3.6 times. It also uses less energy (5.5 joules per answer with the 0.5B, 33 with the 3B), but the saving is smaller than the saving in time, because a long prefill draws more power than decoding. Combining vendor specifications with the measured length of the LLM judges' outputs, a 3B LLM judge must spend at least 1.8 seconds only on writing its verdicts for a long answer on the thesis

laptop, and 0.12 seconds even on the fastest consumer graphics card: a cost that GroundOne does not pay on any hardware. Asked for the same output, LLM judges provide probabilities that barely separate correct from incorrect decisions, while calibration brings GroundOne's expected calibration error on the test set, the average gap between the stated confidence and the observed frequency of correct answers, to 0.019. How the decision is read also matters: with the same trained model, comparing only the three admissible letters is more accurate than letting the model generate its verdict freely, with a gain in balanced accuracy of 0.25 for the 0.5B and 0.07 for the 3B. Some aspects remain to be improved, and they point to natural directions for future work: performance drops on domains far from the training data, such as the fact checking of VitaminC; the subtlest hallucinations, hidden in sentences that otherwise follow the source faithfully, are detected only in part; and training is not yet fully stable across random seeds, which simple changes to the optimiser could mitigate.

Finally, GroundOne Studio, a macOS application, puts the verifier to work in a complete RAG assistant that runs in a single process on the laptop, without network access: retrieval, generation, verification and comparison with a local LLM judge take at most 6.9 GB of memory allocated by MLX. The generator's answer appears first, and the verification and the rewriting of flagged sentences happen on request. In a case study with 100 injected errors, GroundOne flags the error in 97 of the 98 answers in which the sentence receives a verdict, the LLM judge in 84; after two rounds of targeted rewriting the answers that still contain the error fall to 6 %, with numbers and dates mostly restored from the source and invented sentences deleted. The three-way verdict decides the repair: contradicted sentences are corrected, sentences that are not mentioned are often removed.

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Figures x

1 Introduction 1
- 1.1 Motivation . . . 1
  - 1.1.1 Retrieval-augmented generation on the user's device . . . 1
  - 1.1.2 Hallucination and the need for verification . . . 2
  - 1.1.3 The cost of checking an answer on a laptop . . . 2
- 1.2 Decision models . . . 3
- 1.3 Approach and research questions . . . 4
- 1.4 Scope . . . 6
- 1.5 Contributions . . . 7
- 1.6 Main results . . . 8
- 1.7 Thesis structure . . . 9

2 Local Inference of Language Models 10
- 2.1 From hosted to local models . . . 10
- 2.2 Open-weight and small language models . . . 13
  - 2.2.1 Open weights are not open source . . . 13
  - 2.2.2 The gap with closed models is closing . . . 14
  - 2.2.3 Small language models . . . 17
- 2.3 The cost of inference on one device . . . 17
  - 2.3.1 Prefill and decode . . . 18
  - 2.3.2 The roofline model . . . 19
  - 2.3.3 A lower bound on generation . . . 19
- 2.4 Hardware for local inference . . . 20
  - 2.4.1 Classes of hardware . . . 21
  - 2.4.2 The asymmetry holds on every class . . . 25
- 2.5 Runtimes and quantisation . . . 26
  - 2.5.1 Runtimes . . . 26
  - 2.5.2 Specialised inference engines . . . 26
  - 2.5.3 Quantisation . . . 29

2.5.4 Faster generation without fewer tokens . . . 30
2.6 Energy . . . 30
2.7 Consequences for verification on the device . . . 31

3 Background and Related Work 32
3.1 Decision models . . . 32
3.2 Retrieval-augmented generation and hallucination . . . 35
3.3 Groundedness verification . . . 35
3.4 Calibration and selective prediction . . . 36
3.5 Parameter-efficient fine-tuning . . . 37
3.6 Research gap . . . 39

4 GroundOne: Method and Implementation 40
4.1 Task and decision . . . 40
4.2 All verdicts in a single forward pass . . . 41
4.3 Training objective . . . 44
4.4 Calibration . . . 47
4.5 Software . . . 51

5 Data 53
5.1 Task and labels . . . 53
5.2 Source datasets . . . 54
5.3 Construction . . . 56
5.4 The resulting dataset . . . 57
5.5 Reliability of the test sets . . . 59
5.6 Exploratory analysis . . . 61
5.7 Leakage control . . . 64

6 Experimental Setup 65
6.1 Models . . . 65
6.2 Baselines . . . 66
6.3 Metrics . . . 69
6.4 Statistical protocol . . . 70
6.5 Latency, energy and hardware . . . 71
6.6 Licences and terms of use . . . 72

7 Results 75
7.1 Training . . . 75
7.2 Variability across seeds . . . 77
7.3 Accuracy on the RAG test set (T1) . . . 79
7.4 Comparison with the baselines . . . 80
7.5 Transfer to a different domain (T3) . . . 83

7.6 Readout versus generation . . . 84
7.7 Calibration and selective prediction . . . 85
7.8 Probabilities at equal output . . . 87
7.9 Latency as a function of K . . . 88
7.9.1 Reading the text and deciding . . . 89
7.10 Energy per verification . . . 94
7.11 Locality: offline execution, quantisation and co-residence . . . 96
7.12 Error analysis . . . 96

8 GroundOne Studio: a local application and a case study 98
8.1 Goals and requirements . . . 98
8.2 Platform choices . . . 99
8.3 Architecture . . . 100
8.4 Porting GroundOne to Swift . . . 101
8.5 Library and document retrieval . . . 103
8.6 Using the application . . . 103
8.7 Case study: injected errors . . . 110
8.8 An example: a subtle numerical error . . . 114
8.9 Calibrated probabilities and decision threshold . . . 114
8.10 Limitations . . . 115

9 Discussion, Limitations and Threats to Validity 117
9.1 When a specialist decision model pays off . . . 117
9.2 Limitations . . . 121
9.3 Threats to validity . . . 123

10 Conclusions and Future Work 125
10.1 Summary of findings . . . 125
10.2 Future work . . . 126

A Output of every system on one answer 129
A.1 Verdicts sentence by sentence . . . 130
A.2 Raw outputs . . . 131

Bibliography 133

List of Figures

1.1 Two ways of verifying the K sentences of an answer. Parallelograms: inputs and outputs; blue rectangles: components of the model; yellow rectangle: processing outside the model; diamonds: decisions. The decision model reads the document and the answer once and, for every sentence, chooses among three admissible answers, also returning their probability. The generative LLM passes through the model again for every token it writes (red loop), and the verdicts must then be extracted from the text, which may not follow the required format. 5

2.1 Capability of the best closed and the best open-weight model over time on Epoch AI's aggregate capabilities index (dots: individual models; lines: best so far). Plotted from Epoch AI's published data table [34]. 15

2.2 Artificial Analysis Intelligence Index (version 4.3.2, ten evaluations) on 20 October 2026, redrawn from the published chart [35]. Closed models in orange, open-weight models in blue; hatched, the only model that Artificial Analysis classifies as small. In brackets, the reasoning setting at which each model was evaluated. The open-weight status is taken from the data of the same leaderboard. It is a snapshot: scores and ranking change with every new release. 16

2.3 Time per token when the model reads its input (prefill) and when it writes its output (decode), measured on the thesis machine (Apple M4, 16 GB, MLX 8-bit, one request at a time). Prefill: median over contexts of 500, 1,000 and 2,000 tokens. 18

2.4 Roofline model of the thesis machine [43]. Sloped roof: Apple's memory bandwidth (120 GB/s). Flat roof: throughput reached in prefill, a lower bound of the unpublished peak. Intensities computed for one feed-forward layer of Qwen2.5 with 8-bit weights and 16-bit activations; attention is ignored. 20

2.5 Memory needed for the weights of a model as a function of its size and precision (parameters times bits per weight, including half a bit of per-group scales for the quantised formats). Dashed lines: common memory sizes of local hardware. The key-value cache, activations and other running models come on top. 21

2.6 The two architectures of local inference hardware. Blue rectangles: compute units; green rectangles: memory; orange rectangle: the model weights; line width: indicative bandwidth. Capacity and bandwidth values from Table 2.2. In a discrete system a model that does not fit in VRAM must go through the much slower PCIe link; in a unified-memory system the model can occupy almost all of the machine's memory, but every generated token reads it at a lower speed than a high-end graphics card. 22

2.7 Some of the local inference systems compared in this section (images not to scale). Sources: (a), (c) Apple; (b) LoMit, Wikimedia Commons, CC0; (d), (e) NVIDIA. The vendors' images are promotional material, reproduced for the purpose of scientific documentation. . . 23

2.8 Local inference hardware on the two axes that decide what can run: memory capacity (which models fit) and memory bandwidth (how fast tokens are generated). Vendors' specifications: markers at the largest memory configuration, shaded bars over the range of memory sizes offered; the thesis machine is shown at its actual 16 GB. 25

2.9 Lower bounds on the cost of generating text on each class of local hardware, computed as weight bytes divided by the vendor's bandwidth (one request at a time; key-value cache ignored). (a) Time per generated token with the two GroundOne base models in 8 bits; diamonds: decode time measured on the thesis machine. (b) Minimum time that a 3B LLM judge spends only on writing the verdicts for an answer of fourteen or more sentences (median generated tokens of the latency protocol). A decision model generates no token, so both costs are zero for it on every machine. 27

2.10 Streaming experts from the SSD in an engine such as Colibri. Memory holds the part of the model used by every token (attention, router, shared experts) and a cache of recently used experts; every expert is stored on the SSD. For each token the router chooses a few experts: those already cached are used at once, the missing ones are read from disk, at a bandwidth of a few gigabytes per second. For example, GLM-5.2 keeps about 10 GB resident and leaves about 429 GB of weights on disk [36]. The diagram is not to scale. 28

3.1 Three ways of verifying the K sentences of an answer. A specialised encoder evaluates each (document, sentence) pair in a separate forward pass and returns only a probability of support. An LLM judge reads everything, but writes its verdict token by token (red loop), so its cost grows with the text produced and the output has to be interpreted by a parser. A specialist decision model reads document and answer in a single prefill and derives three probabilities from each marker, without generating any token. 37

3.2 Sketch of a risk–coverage curve (illustrative shape, not a result). The decisions are sorted from the most to the least confident; for each share decided (coverage) the error rate (risk) of the accepted decisions is reported. At coverage 1 the risk equals the overall error; giving up the least confident decisions lowers the error. The area under the curve (AURC) summarises the curve in one number: the smaller it is, the better the confidence separates correct decisions from wrong ones. 38

3.3 Low-rank adaptation applied to a linear layer. The weight matrix W stays frozen (8-bit in GroundOne); only the two small matrices A and B are trained, and their product, scaled by α, is added to the output of the layer. With rank r the adapter has $r(d+k)$ parameters instead of the dk of W. 39

4.1 The marker prompt of GroundOne (design b). The sentences of the answer are numbered and every scored sentence is followed by the marker “ `Verdict:`”; a sentence without checkable content (here a heading) remains as context, without a marker. The model processes the whole sequence in a single forward pass, and the output head, restricted to the letters A, B and C, is applied only at the marker positions. Because of causal attention, the verdict on sentence i depends on the document and on sentences 1 to i. 42

4.2 Estimated cost of verifying an answer of $S = 800$ tokens as a function of K, computed with only the constants measured on the M4: the per-token time of prefill and decoding and the mean number of tokens each LLM judge generates per verdict on the latency subset (mean K 8.7). It is an analytical model, not a measurement: the measured latencies are in Section 7.9. For the same text, the cost of the decision model grows by only three prefill tokens per decision, that of the LLM judges by a few decoding tokens per decision. In real answers, however, more sentences also mean a longer text, whose prefill both pay: Section 7.9 separates the two components. 43

4.3 The two solutions for obtaining K verdicts from a single reading. (b) The marker readout, adopted as the default, inserts a marker after every sentence and reads all the verdicts in the same pass. (a) The fallback solution reads the shared prefix once and appends a short question for each sentence, evaluated in a batch; it requires, however, K copies of the KV cache of the prefix, and it is the natural format for an untrained model. 44

4.4 Where GroundOne applies LoRA. Each transformer block of Qwen2.5 contains seven linear projections: four in the attention (W_q, W_k, W_v, W_o) and three in the MLP block (W_{gate}, W_{up}, W_{down}). Their weights remain frozen at 8 bits, and each is paired with a rank-16 adapter (in blue) that corrects its output following the scheme of Figure 3.3. The block is repeated 24 times in the 0.5B and 36 in the 3B; the embedding and the output head have no adapter. 46

4.5 How the AUROC arises (a sketch with synthetic scores, not a result). (a) Distribution of P(supported) for supported and unsupported sentences: the two distributions overlap, and every threshold accepts part of each. (b) Moving the threshold from 1 to 0 yields the ROC curve; the black dot corresponds to the threshold of panel (a). The area under the curve is the AUROC: the more separated the two distributions, the closer the curve gets to the top-left corner and the area to 1. The diagonal corresponds to random scores. 50

4.6 From logits to returned labels. The raw decision, that is, the argmax of the logits, serves the classification metrics of Chapter 7. The calibrated probabilities, obtained with temperature and bias fitted on the dev set, feed instead the single rule of the returned output: supported if P(supported) $\geq \tau$, otherwise the more probable of contradicted and not mentioned. The strict and lenient flags derive from this. 51

4.7 Architecture of the software. At the top, the experimental pipeline, from the public datasets to the frozen tests, run under the memory guard; at the bottom, the verification service, in which the command line and the HTTP server (native and compatible endpoints) share the same Engine, which serialises the requests and runs every MLX call on a single worker thread. The adapter and the calibration file produced by the pipeline are the only artefacts the service reads. . . 52

5.1 Construction of the dataset. RAGTruth answers are segmented, filtered of sentences without checkable content, labelled from the spans and cleaned of exclusions; the official test split becomes T1, while the training sources are divided by source into training set and dev set. LLM-AggreFact and VitaminC provide only test sets (T2 and T3). An automatic leakage check accompanies every build, and the test sets are frozen with a hash before any training. 56

5.2 Rule by which the span-level annotations of RAGTruth become sentence labels. Sentences without checkable content remain in the prompt as context but are not decisions; among the others, conflict prevails over baseless information. 58

5.3 Class distribution among the labelled sentences of each split. In the RAGTruth splits (train, dev and T1) about 88–91 % of the sentences are supported. T2 (LLM-AggreFact) has binary labels, so its contradicted and not-mentioned claims form a single unsupported class (hatched). T3 is sampled with 1,000 pairs per class. 59

5.4 Distribution of K, the number of scored sentences per answer, in the training set and in T1. Dashed lines delimit the bands of K used to report the results. 60

5.5 Share of contradicted or not-mentioned sentences by the model that wrote the answer, in the training set and in T1. 62

5.6 Word overlap between a sentence and its document, that is, the share of the sentence's content words present in the document, by class, in the training set, in T1 and in T3. The mean is given in brackets. 63

5.7 Share of contradicted or not-mentioned sentences by the position of the sentence in the answer. 63

6.1 Map of the baselines. Each one differs from GroundOne in one specific respect and therefore answers a different question: how much training matters (B1), how much the restricted readout matters compared with the generated token (B2), how much the way the verdict is produced matters (LLM judges B0a, B0b, B0d), how much specialisation matters compared with a general-purpose decision model (B3, B3b), how much the architecture matters (encoders B4) and how far one gets without reading the document (elementary cues). 67

6.2 Paired cluster bootstrap. The sentences of the test set are grouped by leakage group (source, document or page); at each repetition the groups are drawn with replacement, the metric of the two systems is computed on the same drawn sources and their difference is recorded. After $B = 2{,}000$ repetitions, the 2.5 and 97.5 percentiles of the differences give the 95 % interval, and the share of differences with the opposite sign to the observed one gives the p-value. 71

7.1 All paired differences between systems on the frozen test sets, with the 95 % cluster-bootstrap interval. In blue the differences in favour of the first system (A), in orange those in favour of the second (B), in grey the non-significant ones, whose interval contains zero. 76

7.2 Three-way balanced accuracy on the dev set during training of the 0.5B model: overall value (left) and value per task (right). The dotted line marks chance level. The 3B shows the same trend on its own subset of 200 answers of the dev set. 77

7.3 Three-way balanced accuracy on the dev set during training of the 0.5B model, with the same procedure and three different seeds. Two runs converge to the same level; the third collapsed to a constant predictor and was stopped. 78

7.4 AUROC of P(supported) on T1 by task (question answering, summarisation, data-to-text) and by system. GroundOne's advantage is concentrated in data-to-text; in summarisation MiniCheck-RoBERTa outperforms the 0.5B and is on par with the 3B. 81

7.5 T1: AUROC of P(supported) for every system (left) and three-way balanced accuracy for the systems with a three-way output (right). Rizzo-Flow is evaluated on 300 answers and the untrained 3B on 600; all other systems, including d1-3B, are evaluated on the whole test set. 81

7.6 Comparison of the systems on T3 (VitaminC): on the left the AUROC of P(supported), on the right the three-way balanced accuracy for the systems with a three-way output. Rizzo-Flow is evaluated on a subsample of 600 pairs; the dashed line marks chance level. For MiniCheck-RoBERTa, VitaminC is part of the lineage of its training data. 84

7.7 Control B2 on the whole of T1. With the same weights and the same forward pass, the verdict is read from the three answer letters (GroundOne) or taken as the token that greedy generation would emit; format failures are counted as errors. 85

7.8 Reliability diagram of P(supported) of the 0.5B, before calibration (raw) and after the calibration fitted on the dev set, for the dev set, T1 and T3. Each point represents a probability bin containing at least 20 sentences. 85
7.9 Reliability diagram of P(supported) of the 3B, before and after the calibration fitted on the dev set, for the dev set, T1 and T3; each point is a probability bin with at least 20 sentences. As for the 0.5B, the calibration transfers to T1 but not to T3. 86
7.10 Risk–coverage curves of the binary decision of the 0.5B with calibrated probabilities: error rate as the number of least confident sentences that are deferred grows. 87
7.11 Equal-output comparison on the 200 answers of the latency subset. LLM judges of the same size are asked for GroundOne's output, with the probabilities read from the logits of the generated letter or written in JSON; all systems are calibrated on the dev set with the same map. Only the sentences with a valid verdict are considered. . 88
7.12 Median time per answer as a function of K on the same 200 answers of T1 (in-process MLX, one answer at a time, M4 with 16 GB). GroundOne, the decision model with one question per sentence and the terse and verbose LLM judges are compared. Logarithmic scale. 89
7.13 Median time per answer split into reading the text (prefill, in grey, shared by every approach) and deciding (in colour), per K band, on the same 200 T1 answers. The labels give, for $K \geq 14$, how many times longer the LLM judge takes to decide than GroundOne. . . . 91
7.14 Composition of the prefill on the 200 T1 answers used for latency. (a) Tokens of GroundOne's prompt by part, mean per K band; the labels give the share of the document and that of answer and markers. (b) Time per part with the 3B, derived from the runs with nothing, the instructions only, and the instructions and the document in the cache as differences of the band medians, so that each column adds up to the median time without cache. In blue, the parts that stay the same across verifications of the same sources, the only ones worth caching. 92
7.15 LLM judges on the latency subset: three-way balanced accuracy, with format failures counted as errors, and share of sentences whose verdict could not be parsed. 95
7.16 Energy per verified answer on 100 answers of the latency subset, measured with `powermetrics` on the CPU, GPU and Neural Engine net of the idle power, together with the mean net power during the run. 95

8.1 Architecture of GroundOne Studio. The core knows only the ports; the engines, all in the same process, are the retriever, the generator, GroundOne (verifier and, passage by passage, attributor) and the LLM judge, which uses the same weights as GroundOne with the adapter removed. The components in which GroundOne is involved are shown in blue. 100
8.2 The usage flow. The answer appears as the generator wrote it; verification, with GroundOne or with GroundOne and the LLM judge, and rewriting happen only on request, and every rewriting produces a new version that is verified again in the same mode. . . 104
8.3 The native answer of the generator to the question "How does E-ZPass charge tolls?", before any verification. The two verification modes are on the right. 105
8.4 The same answer after verification with GroundOne and the LLM judge. The sentences are coloured according to the GroundOne verdict; the panel reports counts, times and disagreements with the LLM judge. 106
8.5 The panel of a selected sentence: calibrated probabilities of GroundOne, verdict of the LLM judge, times and available actions. 107
8.6 The evidence for a sentence: the passage that, according to GroundOne, most clearly contradicts the sentence, with the closest span highlighted. 108
8.7 The statistics section: key indicators, verification time as a function of the number of sentences for GroundOne and for the LLM judge, and cost of the same verification with a cloud LLM judge. 109
8.8 Outcome of each injected error after the loop, by error type, with GroundOne 3B: value restored from the source, sentence rewritten, sentence deleted, error still present and flagged, error accepted by GroundOne. 112

9.1 Where the accuracy of GroundOne on T1 comes from: the same base model untrained (readout, design a), trained but with the decision taken from the token it would generate (control B2, format failures counted as errors) and trained with the restricted readout. The gain over the previous step is shown in brackets. The untrained 3B is evaluated on 600 answers; the dashed line marks the chance level. . 119
9.2 When a specialist decision model pays off, according to the results of this thesis. The summary is qualitative: the times refer to the M4 laptop and to answers with 14 or more sentences, and the out-of-domain transfer is the one observed on LLM-AggreFact and VitaminC. 121

Chapter 1

Introduction

1.1 Motivation

1.1.1 Retrieval-augmented generation on the user's device

Retrieval-augmented generation (RAG) has become the standard way to let a language model answer questions about documents it was never trained on. A retriever selects the passages that are relevant to the question, and the model writes its answer with those passages in its context [1]. The same recipe serves a company searching its internal reports, a lawyer reading a contract and a student questioning their own notes, and in each case the value of the answer depends on its staying faithful to the documents.

Until recently such a pipeline required a hosted model, reached over the network. This is no longer the case. Compact open-weight models and efficient runtimes now run on consumer hardware [2, 3], and platform vendors ship models of a few billion parameters directly on their devices [4]. The gap with the best closed models is small and shrinking: models that fit on a single consumer graphics card reach the performance of the frontier of the time within six to twelve months [5], and the lead of the best closed model over the best open-weight one on a popular leaderboard fell from 8 % to under 2 % in a single year [6]. The whole pipeline, from retrieval to answer, can therefore stay on the user's own machine. This matters for four reasons. When the documents are private or regulated, they never leave the device; privacy becomes a technical property of the system rather than a promise of a provider, which the user cannot verify. A recent dispute over credit between OpenAI and two mathematicians, who had written unpublished drafts inside one of the company's products, showed how real the question is: the provider itself stated that it could not rule out that data derived from that usage had helped improve its models [7]. When there is no network, the assistant keeps working. When the behaviour of the system must stay stable, the weights change only if the user changes them,

instead of following a provider's updates. Finally, the cost becomes that of the hardware and its energy, instead of a price per token. Local models also expose their logits, which hosted interfaces often hide or limit, and this property turns out to be essential for the approach studied here.

Locality is a constraint as well as an advantage. A laptop has limited memory, limited bandwidth and a battery, and every component of the assistant must fit within them at the same time. This thesis treats locality as a requirement rather than as an afterthought: every step of the work, from building the dataset to training, evaluation and serving, runs on one consumer laptop with 16 GB of memory, and no step depends on an external service. Because locality is the backbone of the work, Chapter 2 is devoted entirely to it: why it matters, which models and hardware make it possible, and what it costs.

1.1.2 Hallucination and the need for verification

Retrieval does not remove hallucination. Even with the right passages in context, generated answers still contain sentences that the passages do not support, or that contradict them [8, 9]. An example from the data used in this thesis shows how subtle the problem can be. Asked to describe a restaurant from a structured record, a model writes six fluent sentences. Four of them are correct. One states that the restaurant is open seven days a week, whereas the record shows that it is closed on Mondays: the sentence contradicts the source. Another states that the restaurant has a rating of 4.5 stars on Google, whereas the record gives the rating without naming any source: the sentence adds a detail that the source does not mention (Appendix A).

Two observations follow from this example. First, the error sits in a single sentence of an otherwise correct answer, so a verdict on the whole answer is of little help: the user needs to know which sentence to distrust. Second, the two erroneous sentences are wrong in different ways. A contradiction is a factual error with respect to the source, whereas an unsupported addition may be harmless background or an invention, and an application may want to treat the two differently. A useful verifier therefore decides, for every sentence, whether the document supports it, contradicts it or does not mention it. Since no user can be expected to check every sentence against its source, the verifier must run on every answer, automatically, and on the same machine that generated it.

1.1.3 The cost of checking an answer on a laptop

A verifier that runs next to the generator competes with it for the same memory, time and battery. The most common solution, asking a language model to judge the answer, is expensive precisely where a laptop is weakest. An LLM judge, that

is, a large language model asked to act as a judge, writes its verdicts token by token, and on local hardware each generated token costs far more than each token it reads. Reading the prompt is limited by computation, which a laptop GPU handles well, whereas writing the output is limited by memory bandwidth, because every new token must read all the weights [10]. On the laptop used in this thesis, reading one token takes about 0.37 milliseconds for a model of 0.5B parameters, while writing one takes about 5.9, sixteen times more, and the asymmetry holds on every class of local hardware, from phones to dedicated AI machines (Chapter 2). An LLM judge that explains its reasoning before each verdict multiplies this cost by the length of its explanation, and the cost grows again with the number of sentences in the answer. Small LLM judges also break the required output format, and the confidence they express is rarely usable as a probability.

Encoder-based verifiers[1] avoid generation altogether. Models such as AlignScore, MiniCheck and HHEM read a pair of document and sentence and return a score of support [11, 12, 13]. They are small and fast, but they give one fixed binary score per pair: they do not distinguish a contradiction from an omission, they read each sentence without the rest of the answer, and adapting them to a new decision requires a new model.

1.2 Decision models

A different way to use a language model for decisions has recently attracted attention: *decision models*. The category was introduced under the name of *System One models* [14], which evokes the fast, intuitive mode of thinking as opposed to deliberate reasoning; this thesis adopts the term decision model, later taken up by the ecosystem. Instead of generating an answer, such a model reads the question and the context once and returns a probability for each admissible answer, for example by reading the logits of the option letters at the end of the prompt (e.g., A: 90 %, B: 5 %, C: 5 %) [15, 16]. Three properties follow directly. No text is generated, so the decision costs only the reading of the input. The output is always one of the admissible answers, so it is always valid. Many questions about the same context can share a single forward pass[2] over it, so each additional question costs little.

[1] An *encoder* is a transformer in which every token attends to all the others, both those that precede it and those that follow it. It produces a representation of the text instead of generating text, like BERT and the models derived from it.

[2] A *forward pass* is one complete pass of the input through all the layers of the network: for a language model, the computation that turns the input tokens into the logits of the next token. A generative model runs one for every token it writes; a decision model runs a single one to read the whole input.

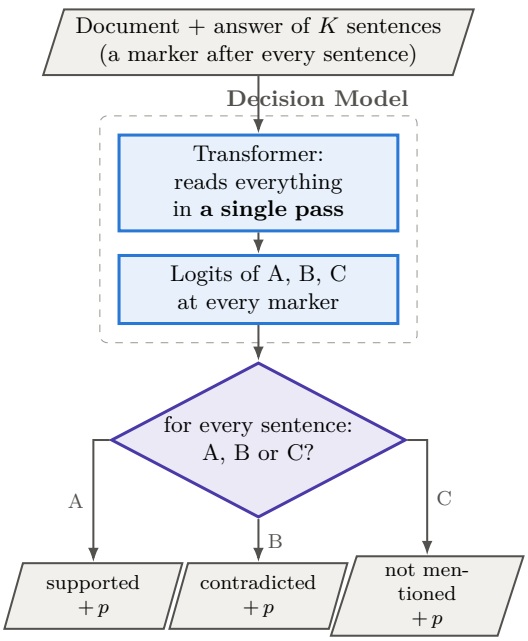
The idea fits verification well. An answer of K sentences poses K closed questions about the same document, and each of them has three admissible answers. A decision model can answer all of them while reading the document only once, and the probability it returns for each sentence can be calibrated and compared with a threshold chosen by the application. A traditional LLM, instead, produces its answer autoregressively: after reading the document it must generate the verdicts one token at a time, with a new pass through the model for every token written, and only at the end can the text it produced be interpreted as a decision (Figure 1.1).

The field is young and moves quickly. After the first hosted model, open implementations followed [15, 16], and on 9 October 2026 the Hugging Face hub officially introduced a tag for decision models, which the following day already grouped about 1,500 models, more than half of them published in the previous weeks [17]. Established organisations released their own, such as Liquid AI's d1-3B and Cloudflare's Clef [18, 19]. The evidence, however, lags behind the releases. Most of it comes from the authors of each system, independent evaluations are recent and often less favourable [20, 21], and a common leaderboard finds that general-purpose decision models with fewer than one billion parameters barely exceed the trivial baseline on hallucination detection in retrieval-augmented answers [22]. All the open implementations, moreover, are general-purpose systems that answer any typed question.

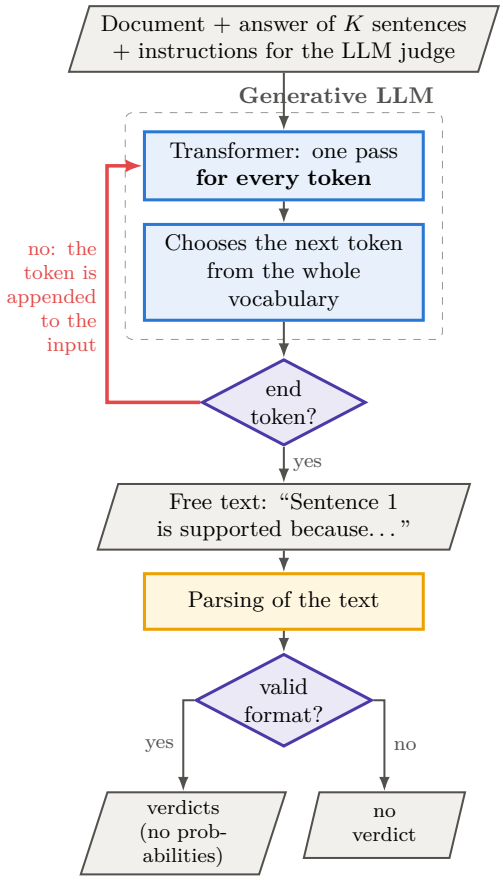
Three problems therefore remain open. The first is whether, for a single task, a small specialised decision model is worth using. The second is how such a model compares with the alternatives: a general-purpose decision model, an LLM judge and a dedicated verifier. The third is where any advantage comes from: from the specialisation, from the way the decision is read or, more simply, from the size of the model.

1.3 Approach and research questions

This thesis designs, trains and evaluates GroundOne, a compact decision model specialised in one decision: for every sentence of a RAG answer, whether the retrieved document supports it, contradicts it or does not mention it. The answer is written with a short marker after each sentence, and the verdicts of all sentences are read in one forward pass, from the logits of three answer letters at the markers, without generating any text. Qwen2.5 models with 0.5B and 3B parameters are fine-tuned with LoRA on human-labelled answers from RAGTruth [23, 24, 8], and their probabilities are calibrated on development data. Three test sets were built and frozen before any model was trained, so that no design choice could be tuned on them. The three open problems become five research questions: the first two restate them in a testable form, and the other three add what matters for real use



(a) Decision model: one reading and, for every sentence, a choice among three admissible answers.



(b) Traditional LLM: one pass through the model for every token, until the text is complete.

Figure 1.1: Two ways of verifying the K sentences of an answer. Parallelograms: inputs and outputs; blue rectangles: components of the model; yellow rectangle: processing outside the model; diamonds: decisions. The decision model reads the document and the answer once and, for every sentence, chooses among three admissible answers, also returning their probability. The generative LLM passes through the model again for every token it writes (red loop), and the verdicts must then be extracted from the text, which may not follow the required format.

on the device, namely the reliability of the probabilities, the cost and the value of a three-way decision.

RQ1, specialisation. Can a decision model of 0.5B or 3B parameters, fine-tuned on the task, decide whether each sentence is faithful to the document? And

does it do so as well as larger models used zero-shot[3] and as the encoders specialised in this kind of verification?

RQ2, readout versus generation. How much does the decision-model mechanism, that is, the readout[4] of the decision from the logits, contribute at the same model size? The comparison is twofold: against LLM judges that write their verdicts and against the same model left free to generate its answer. And how does the difference change as the number of sentences to verify grows?

RQ3, calibration. Are the probabilities returned by the model reliable as a measure of confidence? In particular, can they be used to set a decision threshold and to defer uncertain cases to the user? And are they better than those that can be obtained from an LLM judge?

RQ4, cost on the device. Does the verifier fit within the memory, time and energy limits of a 16 GB laptop? And how much does it cost compared with an LLM judge running on the same machine?

RQ5, beyond a binary score. What does a user of the verifier gain from a three-way decision for every sentence, compared with the plain score of a binary verifier?

1.4 Scope

The thesis studies the verification step in isolation. The retrieved document is given, so the quality of retrieval is not evaluated, and an answer is judged only against the document it was generated from, not against world knowledge. GroundOne itself flags sentences but does not correct them, and it returns verdicts and probabilities rather than written explanations; the application of Chapter 8 adds retrieval and rewriting, which are not evaluated as an experimental contribution. The data are in English. Documents longer than the context used in training are split into parts, as described in Chapter 6. Finally, the experiments target one class of hardware, a consumer laptop with unified memory, and every system, including every baseline, runs on the same machine.

[3] A model is used *zero-shot* when it tackles a task without any task-specific training and without examples in the prompt: it receives only an instruction that describes the task.

[4] The *readout* is the reading of the decision directly from the logits of the model: instead of letting it write an answer, the scores it assigns to the admissible answers alone (here the letters A, B and C) are compared and the most probable one is chosen.

1.5 Contributions

The thesis makes five contributions.

1. **GroundOne**, an open specialist decision model for sentence-level groundedness verification that runs entirely on a laptop. Thanks to a marker placed after every sentence, it reads all the verdicts of an answer in one forward pass. It is released as an installable package with a command-line interface and a local HTTP API, which also includes an interface compatible with the System One interface of Jev, already used by other clients.

2. **A sentence-level dataset derived from public annotations.** No label was annotated anew: the human annotations of RAGTruth, which mark spans of text, were converted into a three-way label for every sentence, with rules fixed in advance. The three test sets come from public benchmarks, were checked automatically, for instance on sentence segmentation, and were frozen before any model was trained, with leakage controlled at the level of documents[5].

3. **A controlled decomposition of the gains.** The comparison isolates the contribution of specialisation, of the restricted readout and of model size. GroundOne is compared with cues that never verify a sentence, untrained models, LLM judges with and without probabilities, two general-purpose decision models (Rizzo-Flow, with 4B parameters, and Liquid AI's d1-3B, of the same size as the 3B) and specialised encoders, all on the same hardware and with intervals obtained by cluster bootstrap.

4. **A study of calibration, time and energy**: how the probabilities can be calibrated and turned into consistent labels, how much they help in deferring uncertain decisions, and how the time and energy of a verification grow with the length of the answer. The study is set in an analysis of local inference that combines vendor specifications with own measurements to bound, for each class of consumer hardware, what a verifier that generates text must pay and a decision model does not.

5. **GroundOne Studio**, a macOS application in which retrieval, generation and verification take place in the same process, on the same laptop and without external services. The generator's answer appears first; the user can then verify

[5] *Leakage* occurs when test-set data end up, directly or indirectly, among the data used to train or configure the model: for example, the same document, or answers to the same document, present in both sets. The model can then recognise examples it has already seen instead of generalising, and the test metrics become optimistic.

it with GroundOne, alone or together with an LLM judge running locally, see the evidence supporting each sentence and have the flagged sentences rewritten. A case study with injected errors measures the loop of verification and rewriting.

1.6 Main results

The results show that specialisation works in the domain it was aimed at. On the RAGTruth test set GroundOne reaches a three-way balanced accuracy[6] of 0.769 with 0.5B parameters and 0.832 with 3B, against 0.37 and 0.44 for the same models before training (for the 3B on a 600-answer subsample). It ranks supported sentences better than the specialised encoders and, in three-way balanced accuracy, it clearly outperforms both general-purpose decision models: Rizzo-Flow, eight times larger than the 0.5B, and d1-3B, of the same size as the 3B. On LLM-AggreFact, a benchmark drawn from different sources, the 3B is on par with the best small encoder.

The decision-model mechanism pays off in speed and in accuracy. When an answer contains fourteen or more sentences, GroundOne verifies it 1.7 to 2 times faster than a terse LLM judge and about six times faster than a verbose LLM judge of the same size, with an output that is always valid and with less energy. The overall ratio is bounded by reading the text, which every approach pays: net of it, GroundOne decides 18 to 33 times faster than the terse LLM judge. When the document has already been read, as in the verification of a rewritten answer, reusing the KV cache raises the advantage on the same long answers from 1.7–2 to 3.4–3.6 times. How the decision is read also matters: with the same trained model, choosing the most probable of the three admissible letters is more accurate than letting the model generate its verdict freely, with a gain in balanced accuracy of 0.25 for the 0.5B and 0.07 for the 3B. After calibration, the probabilities of GroundOne are close to the observed frequencies, whereas the probabilities of LLM judges asked for the same output barely separate correct from incorrect decisions.

Some aspects remain to be improved, and they point to natural directions for future work. Performance drops on domains far from the training data, where the general-purpose decision model d1-3B is stronger; the subtlest hallucinations, hidden in sentences that otherwise follow the source faithfully, are detected only in

[6] *Balanced accuracy* is the mean over the classes of the recall, that is, of the fraction of examples of each class that are classified correctly. Unlike plain accuracy, it does not reward a model that always predicts the most frequent class: a constant predictor scores 0.5 in the binary case (50 when expressed as a percentage) and about 0.33 with three classes.

part; and training is not yet fully stable across random seeds[7].

Finally, the GroundOne Studio application shows that a complete RAG assistant can be built around the verifier, in a single process and without network access, with at most 6.9 GB of memory allocated by MLX. In a case study with injected errors, GroundOne flags the error in 97 of the 98 answers in which the sentence receives a verdict, against 84 for the LLM judge, and targeted rewriting reduces the answers that still contain it from 100 % to 6 %.

1.7 Thesis structure

Chapter 2 is devoted to local inference: the reasons for keeping models on the device, open-weight and small models, the cost model of inference on one device, the hardware landscape, runtimes and quantisation, energy, and the consequence for verification. Chapter 3 reviews decision models and their ecosystem, hallucination in retrieval-augmented generation, verification methods and calibration. Chapter 4 describes GroundOne: the decision, the single-pass readout, the training objective, the calibration and the served output. Chapter 5 defines the labels and describes how the training and test sets were built, checked and frozen. Chapter 6 presents the trained models, the baselines, the metrics and the measurement protocol. Chapter 7 reports the results, Chapter 8 presents the GroundOne Studio application and the case study, Chapter 9 discusses what the results mean and where they stop, and Chapter 10 concludes. Appendix A shows the output of every system on the example answer described in Section 1.1.2.

[7]The *seed* is the number that initialises the random number generator used during training, for instance to initialise the added weights and to shuffle the order of the data. Repeating the training with different seeds produces slightly different models: if the results change a lot from one seed to another, the procedure is not robust.

Chapter 2

Local Inference of Language Models

This thesis builds a verifier that runs entirely on the user's own machine, and this choice shapes every other one: the size of the models, the way decisions are read from them, the measurements and the comparisons. This chapter explains why local inference matters, what makes it possible today and what it costs. It starts from the reasons for keeping a model on the device and from the open-weight models that make this possible. It then derives a simple cost model of inference on one device, which shows why reading a token costs far less than writing one, and surveys the hardware on which local models run, from the laptop used in this work to dedicated desktop machines. It continues with the software and the quantisation that bring models within reach of that hardware, and with energy. It closes by drawing the consequence for verification, which motivates the design of GroundOne in the following chapters.

2.1 From hosted to local models

Most people use language models through hosted services. The user's input travels over the network to a provider, a model in a datacenter processes it, and the answer travels back. The model, the hardware and the policies that govern the data all belong to the provider. Running the model on the user's own device changes all of this at once. Five properties follow, and each of them matters for a retrieval-augmented assistant that works on private documents.

Privacy as a technical guarantee. With a hosted model, every document and every question is sent to a third party, and what happens to them afterwards, how long they are kept, who can access them and whether they are used to improve

future models, is governed by contract and policy. The user can read the policy but cannot verify that it is respected. With a local model the documents never leave the machine, so the question does not arise: privacy becomes a property of the system rather than a promise. Surveys of edge intelligence, that is, of running models on peripheral devices close to the user, state the point directly: on-device inference eliminates privacy leakage and the need for an Internet connection [25].

European law makes the difference concrete. Under the General Data Protection Regulation (GDPR), the processing of personal data explicitly includes its "disclosure by transmission" [26, Art. 4(2)]. Sending a document that contains personal data to a hosted model therefore makes the provider a processor[1], bound by a contract with the specific guarantees of Article 28. If, in addition, the provider operates outside the European Union, the transfer of the data must have a legal basis: an adequacy decision[2], appropriate safeguards or one of the derogations provided by Articles 44–49 [26]. A verifier that runs on the user's device involves no processor and no transfer. It does not remove every obligation, since the organisation that uses it must still respect the principles of Article 5 and secure the device, but it removes the obligations that depend on trusting someone else. Many confidential documents, such as unpublished research, contracts or source code, contain no personal data at all; for them the regulation does not apply, and the argument becomes one of confidentiality.

A recent episode illustrates what is at stake. In September 2026 OpenAI announced that an internal multi-agent system had produced a proof of blow-up (the formation of a singularity in finite time) for the forced Navier–Stokes equations, one of the Millennium Prize problems, formalised in the Lean proof assistant [7]. Two mathematicians, Tristan Buckmaster and Levent Alpöge, had released related results for the Euler equations at about the same time, and disputed the credit. Buckmaster had written his drafts in OpenAI's own coding assistant, and asked the company "whether the model had been trained on, or had access to, our sessions in Codex, into which we had been putting all our drafts" [27]. OpenAI denied that its researchers or agents had seen the work before its release, but added: "While unlikely, we cannot rule out that de-identified data derived from their usage of our products helped improve our models" [7, 28]. Nothing in the public record shows that the drafts were misused, and the dispute is mainly about credit. What the episode does show is that, once unpublished work is inside a hosted model, neither the user nor, by its own statement, the provider can exclude that it contributed to the provider's models. With a local model this question cannot be asked.

[1] In the GDPR, the *controller* decides why and how personal data are processed; the *processor* processes them on behalf of the controller, for instance by providing a computing service.

[2] A decision by which the European Commission recognises that a third country ensures a level of data protection essentially equivalent to that of the European Union.

Providers are aware of the problem. Apple describes on-device processing as the cornerstone of its approach to privacy, and for requests that exceed the device it built Private Cloud Compute, a dedicated cloud with custom servers, no privileged access for staff, no persistent storage of requests and a public log of the software images that run on it [29]. The design is a serious attempt to make hosted inference verifiable, and its very complexity is an argument for locality. Privacy in the cloud requires custom silicon, a hardened operating system and cryptographic attestation, that is, a signed proof of which software is running on the servers. Even so, the system rests on trust in the hardware, in the supply chain and in the attestation itself, which only security researchers, not users, can inspect [29].

Availability. A local model works without a network and without depending on the availability of a provider. For an assistant used on a train, in a hospital ward with restricted connectivity or in an air-gapped environment, that is, one physically isolated from any network, this is a requirement rather than a convenience.

Stability and reproducibility. The weights of a local model change only when the user changes them. A hosted model can be updated, re-tuned or withdrawn at any time, often silently, so that the same request produces a different answer from one month to the next. For a verifier whose thresholds are calibrated on development data (Chapter 4), a silent change of the underlying model would invalidate the calibration without warning.

Cost. A hosted model is paid per token, and a verifier that checks every answer multiplies the number of tokens processed. A local model is paid once, with the hardware, and then costs only its energy.

Access to the model's internals. A local model exposes its logits, the scores from which every probability is computed. Hosted interfaces often hide them or return them only for a few tokens. The approach studied in this thesis reads the decision directly from the logits of a few answer letters and calibrates them, so it requires this access; it is, in this sense, possible only with a local model.

The price of locality. These advantages have a cost. A laptop holds only small models, and it must hold them next to the operating system, the user's applications and every other component of the assistant. It has less memory bandwidth and less computation than a datacenter accelerator, and it runs on a battery. Part of the literature concludes that pure on-device deployment is too limited and proposes to split the work with an edge server, that is, a server close to the user, or with the cloud [25, 30]: a small local model handles easy requests and forwards hard ones,

or drafts an answer that a larger remote model checks. Apple's combination of an on-device model with Private Cloud Compute is the most visible example [4, 29]. These hybrid designs reduce the cost of locality but sacrifice part of its benefits, since whatever is forwarded leaves the device. This thesis takes the strict position: the whole verification stays on the device, and the rest of the chapter examines what this requires.

Table 2.1 summarises the comparison.

Table 2.1: Hosted and local inference compared on the properties that matter for a verifier of private documents.

Property	Hosted model	Local model
Privacy	Contractual: documents are sent to a processor; the user cannot verify how they are used.	Technical: documents never leave the device.
Regulation (GDPR)	Processor contract (Art. 28); third-country transfers (Arts. 44–49).	No processor and no transfer; Art. 5 principles and device security still apply.
Availability	Requires network and provider uptime.	Works offline.
Stability	The provider may update or withdraw the model.	Weights change only when the user changes them.
Cost	Price per token.	Hardware once, then energy.
Logits	Often hidden or limited.	Fully accessible: readout and calibration possible.
Capability	Frontier models.	Small models, limited by memory, bandwidth and battery.

2.2 Open-weight and small language models

Local inference requires a model whose weights can be downloaded. This section describes what such models are, how far they trail the best closed models, and which of them are small enough for a personal device.

2.2.1 Open weights are not open source

A model is called *open-weight* when its trained parameters are published and can be run and modified by anyone. This is not the same as open source. The Open Source Initiative defines an open-source AI system through the four freedoms of the

free-software tradition, to use, study, modify and share the system, and requires that the form preferred for modifying it be available: the parameters, the complete code used to train and run it, and enough information about the training data for a skilled person to build a substantially equivalent system [31]. The data themselves need not be released, but weights alone do not qualify. Most models that are commonly called open publish their weights and an inference recipe, but neither the training code nor a detailed description of the data. Being open is therefore a matter of degree rather than a binary property. A recent index scores models on five aspects: the availability of the weights, the disclosure of the pre-training and of the post-training data (the two phases of training, first on generic text and then on instructions and preferences), the methodology and the licence [32]. No model reaches the maximum score; fully documented research models such as OLMo score high, whereas some popular open-weight models publish little beyond the weights.

Licences add a further distinction. The two models fine-tuned in this thesis come from the same family but carry different licences: Qwen2.5-0.5B is released under the permissive Apache 2.0 licence, whereas Qwen2.5-3B is released under a research licence that restricts commercial use [23]. In the terminology above both are open-weight models, and the thesis uses this term for them. The distinction also matters for regulation. The European AI Act relaxes some transparency obligations for general-purpose models released under free and open-source licences, but not for models that pose systemic risk. A model is presumed to pose systemic risk when its training required more than 10^{25} floating-point operations (FLOP[3]) [33], far more than those used for the models of this thesis.

2.2.2 The gap with closed models is closing

Local inference would be of limited interest if open-weight models were far behind closed ones. They are not. On the Chatbot Arena leaderboard, where users compare the answers of two anonymous models, the lead of the best closed model over the best open-weight model fell from 8.04 % in January 2024 to 1.70 % in February 2025 [6]. On Epoch AI's capabilities index, which aggregates many benchmarks, the best open-weight models trailed the closed frontier by about four months on average between January and May 2026 (Figure 2.1) [34].

A more recent snapshot confirms the picture and also shows its limit. On the Artificial Analysis Intelligence Index, which aggregates ten evaluations, on 20 October 2026 the best closed model, Claude Opus 5.5, scores 58 points, while the best open-weight model, MiMo-V2.6-Pro, scores 46 (Figure 2.2) [35]. Several

[3]A FLOP is a single arithmetic operation on floating-point numbers, such as an addition or a multiplication. Computing speed is measured in FLOP per second: one TFLOP/s corresponds to 10^{12} operations per second.

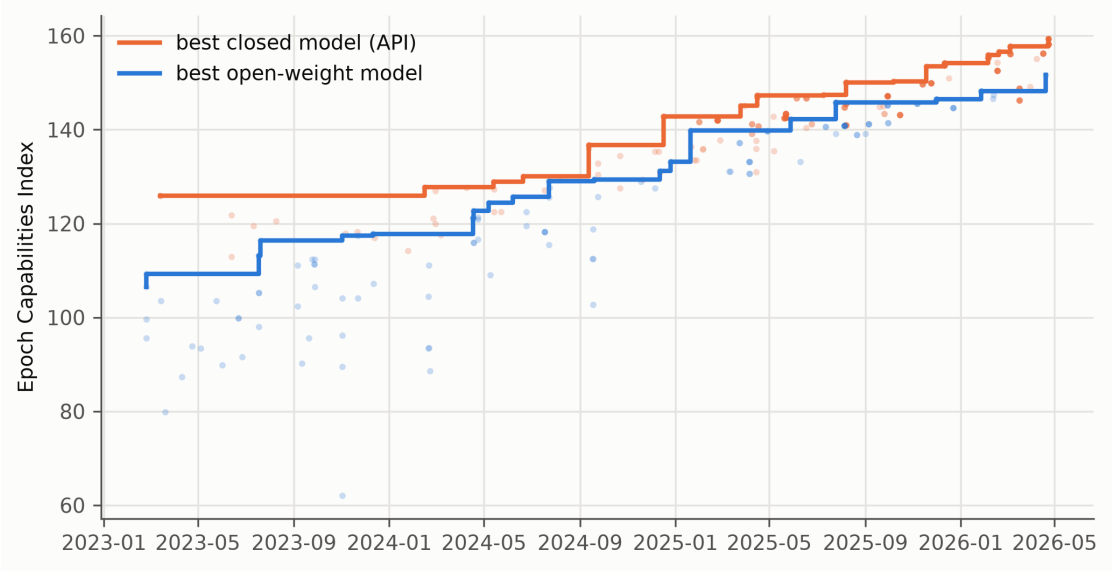


Figure 2.1: Capability of the best closed and the best open-weight model over time on Epoch AI's aggregate capabilities index (dots: individual models; lines: best so far). Plotted from Epoch AI's published data table [34].

open-weight models, such as DeepSeek V4.1 Flash, GLM-5.3 and Kimi K3, match or exceed mid-range closed models such as Claude Haiku 5.5 and Gemini 3.8 Flash. All these open-weight models, however, belong to the class of large models, whereas the only model in the chart that Artificial Analysis classifies as small, Qwen3.8 27B, stops at 34 points.

This does not mean that large models are excluded from local hardware. The specialised inference engines described in Section 2.5.2, such as ds4 and Colibri, run large open-weight models, for example DeepSeek V4 Flash and GLM-5.2, even when their weights do not fit in memory: they keep the always-active part resident and read the rest from the SSD. The limit then shifts from capacity to speed. According to the authors of Colibri, GLM-5.2 generates 1.83 tokens per second on a Strix Halo machine with 128 GB and less than one token per second on a laptop [36]. For a personal device, therefore, the question is not only which model can be run, but how fast it can generate text; and generation is precisely the phase that a decision model avoids.

For interactive use, what matters most is the best model that fits entirely in the memory of consumer hardware, and can therefore generate without depending on the bandwidth of the SSD. Epoch AI tracks this quantity directly. Considering models small enough to run on a single consumer graphics card, at most about 28 billion parameters for a card with 24 GB and about 40 billion for one with 32 GB, it finds that across four benchmarks they reach the performance of the

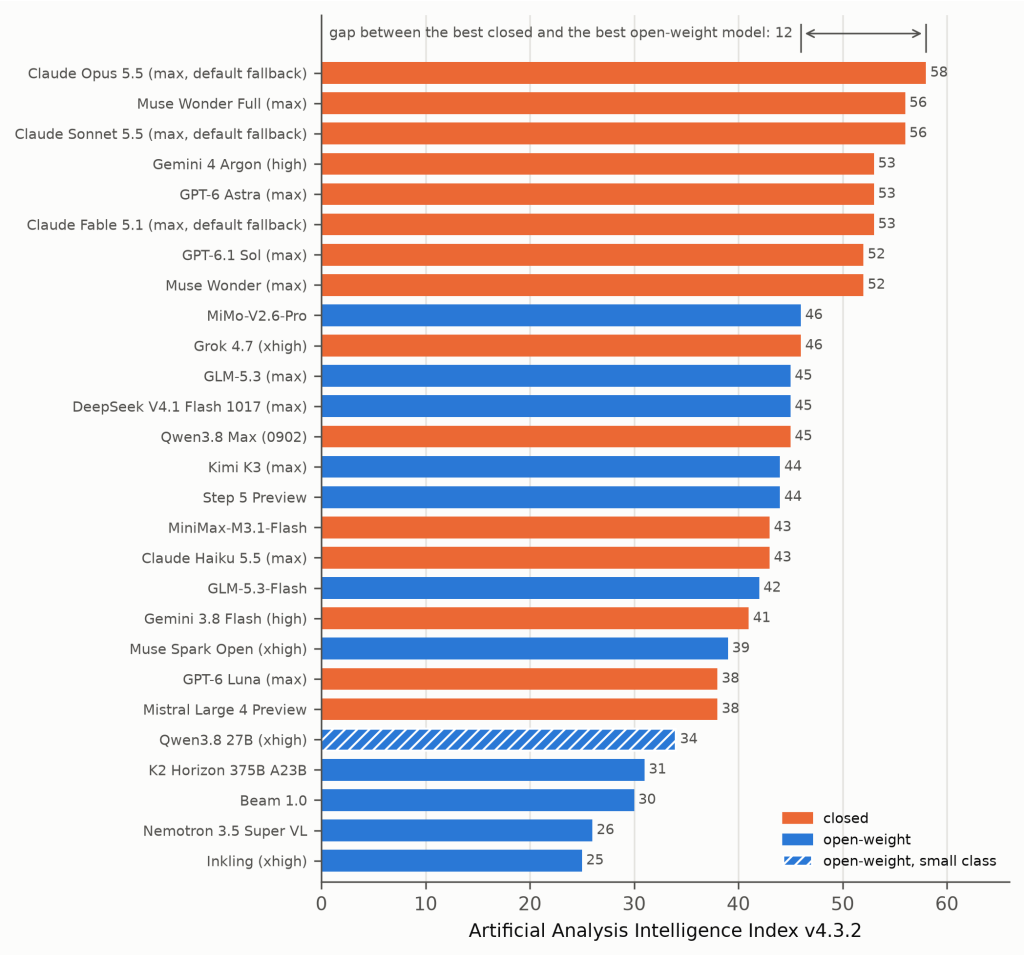


Figure 2.2: Artificial Analysis Intelligence Index (version 4.3.2, ten evaluations) on 20 October 2026, redrawn from the published chart [35]. Closed models in orange, open-weight models in blue; hatched, the only model that Artificial Analysis classifies as small. In brackets, the reasoning setting at which each model was evaluated. The open-weight status is taken from the data of the same leaderboard. It is a snapshot: scores and ranking change with every new release.

frontier model of the time after six to twelve months [5]. Three forces drive the catch-up: open-weight developers scale their training compute about as fast as closed ones, distillation, that is, training a small model to imitate the answers of a large one, transfers much of its capability, and each new generation of graphics cards raises the size of the models that fit [37]. The same trend appears from the other side: the smallest model scoring above 60 % on the MMLU benchmark, a set of multiple-choice questions on 57 subjects, had 540 billion parameters in 2022 and 3.8 billion in 2024, a reduction of 142 times [6].

2.2.3 Small language models

At the low end of the scale are *small language models*. Definitions vary; a survey that measures several dozen of them on devices places the range at 100 million to 5 billion parameters, with a decoder-only architecture, that is, the architecture of common LLMs that generate text from left to right [38]. Small models are trained on far more data than would be worthwhile if the only aim were to minimise the cost of training. The reason is that, for them, the cost that matters is inference, repeated millions of times after training: it is therefore worth spending more once to obtain a small but capable model. A model of half a billion parameters can thus see trillions of tokens [38]. Measured on edge devices with 4-bit weights, models between 0.1 and 3 billion parameters need between 0.3 and 2.5 GB of memory, and at long context most of it is taken by the key-value cache, which stores the context already read (Section 2.3.1), and by the computation buffers [38]. Research on models below one billion parameters argues that, on phones, an application should not use more than about a tenth of the device's memory, and shows that a carefully designed 350-million-parameter model matches a general 7-billion-parameter model on a narrow task such as calling an application interface [39].

Platform vendors have adopted the same idea. Apple's foundation model for its devices has about three billion parameters, is quantised to under four bits per weight and is specialised for each feature with small low-rank adapters, additional modules with few parameters trained on the single task (Section 3.5) [4]; Google's Gemma 3n family is designed to run on phones and laptops with a reduced memory footprint, and is marketed explicitly for privacy and offline use [40]. GroundOne follows the same pattern: a small open-weight model, quantised, specialised by an adapter for one task.

2.3 The cost of inference on one device

The previous sections explain why a model should run locally and which models can. This section explains how much it costs to run one, and why the cost depends so strongly on what the model is asked to do.

2.3.1 Prefill and decode

A decoder-only language model processes a request in two phases. In the *prefill* phase it reads the whole prompt at once. To do so it uses the attention mechanism, through which each token gathers information from the tokens that precede it: every layer computes, for each input token, two vectors (a *key* and a *value*) that later tokens will consult, and stores them in the *key-value cache*. In the *decode* phase it produces the output one token at a time; each new token consults the cache through attention and extends it by one position [10]. Prefill processes many tokens in parallel; decoding processes one. Serving systems exploit the difference in several ways, for instance by managing the cache in pages to fit more concurrent requests [41] or by running the two phases on different machines suited to each [42].

The consequence is visible on the laptop used in this thesis (Figure 2.3). With the 0.5B model, reading one token of the prompt takes less than 0.4 milliseconds, while writing one token of output takes 5.9 milliseconds, sixteen times more. With the 3B model the figures are 2.2 and 31.8 milliseconds, fourteen times more. A model that writes a short paragraph thus spends more time on it than on reading a document of several thousand tokens.

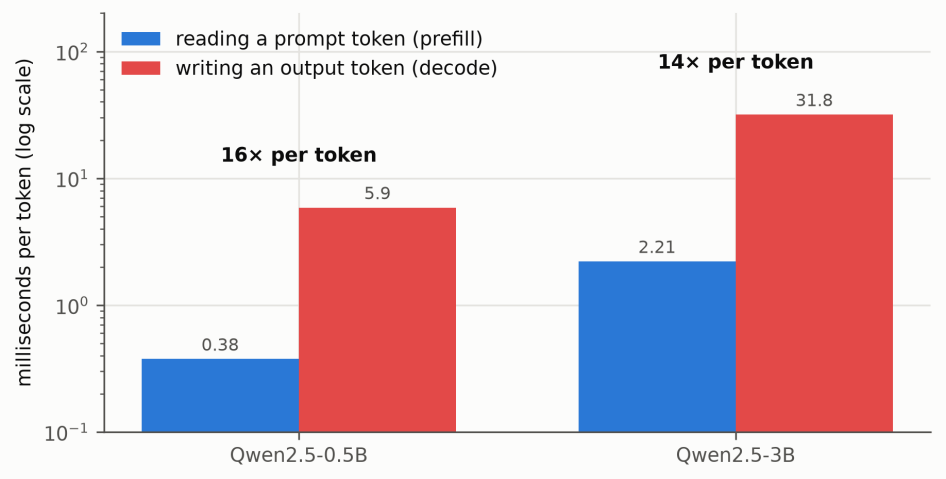


Figure 2.3: Time per token when the model reads its input (prefill) and when it writes its output (decode), measured on the thesis machine (Apple M4, 16 GB, MLX 8-bit, one request at a time). Prefill: median over contexts of 500, 1,000 and 2,000 tokens.

2.3.2 The roofline model

The reason for this asymmetry is captured by the roofline model, a simple model of performance introduced for multicore processors [43]. A computation is characterised by its *arithmetic intensity*, the number of floating-point operations it performs for each byte it reads from memory. A processor has two limits: a peak rate of computation and a peak memory bandwidth. The attainable rate is the minimum of the peak computation and the product of bandwidth and intensity. A computation with low intensity is limited by the bandwidth, and one with high intensity by the computation; the intensity at which the two limits meet is called the ridge point [43].

Applied to a transformer, the model separates the two phases cleanly [44]. Each weight matrix of a linear layer must be read from memory once per forward pass. During prefill the same matrix multiplies the hidden states of hundreds or thousands of tokens, so each byte of weights is used for hundreds of operations: the intensity is high and the computation is the limit. During decoding, the matrix multiplies a single vector, so each weight is used for about two operations: the intensity is close to one and the bandwidth is the limit. For a 7-billion-parameter model in 16-bit precision, the linear layers reach intensities above one thousand operations per byte in prefill and about one in decode [44]. In datacenters this imbalance is softened: the decoding of many concurrent requests is grouped into batches, so that each weight read from memory serves to generate several tokens and the intensity rises [10]. A personal device serves a single user, one request at a time, and cannot rely on this remedy: this is where the asymmetry between the two phases weighs most.

Figure 2.4 draws the roofline of the thesis machine. Apple publishes its memory bandwidth, 120 GB/s, but no peak rate of computation [45]; the flat roof in the figure is therefore the rate actually reached during prefill, about 2.8 TFLOP/s, which is a lower bound of the peak. The ridge point lies at about 23 operations per byte. Generating a token sits at about two operations per byte, an order of magnitude below the ridge, whereas reading a document of hundreds of tokens sits an order of magnitude above it. The two phases of the same model, on the same machine, live in different regimes.

2.3.3 A lower bound on generation

Because decoding reads every weight once per generated token, its time per token cannot be lower than the size of the weights divided by the memory bandwidth, whatever the hardware, unless techniques such as speculative decoding (Section 2.5.4) or a more aggressive quantisation are used. For the 3B model in 8-bit precision, whose weights occupy about 3.3 GB, this floor is 27 milliseconds per token on a machine with 120 GB/s; the measured 31.8 milliseconds are only about one sixth

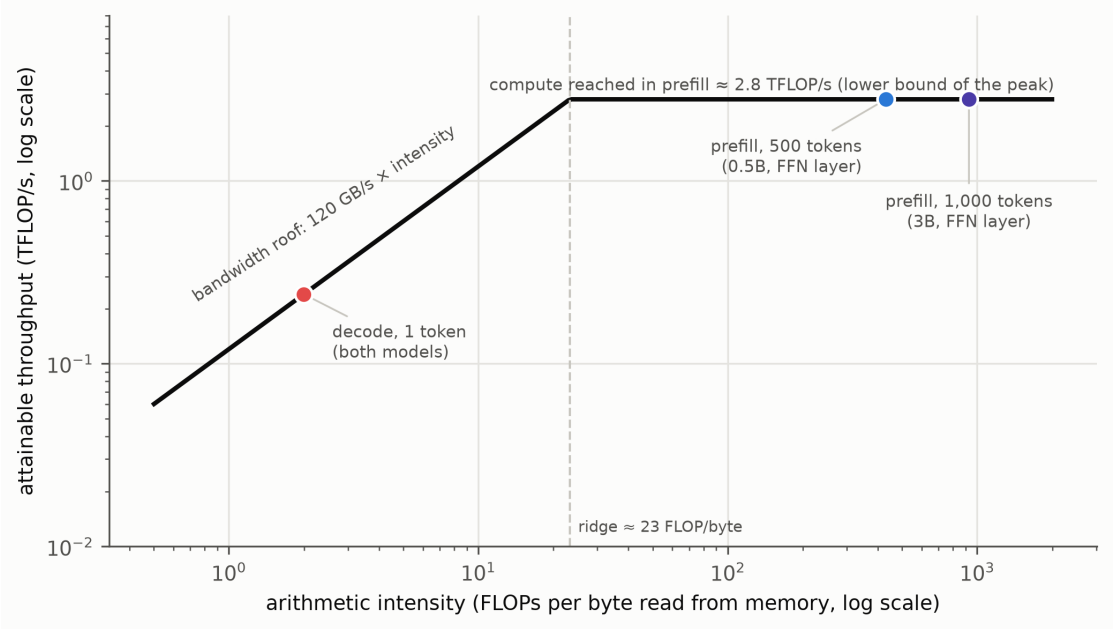


Figure 2.4: Roofline model of the thesis machine [43]. Sloped roof: Apple's memory bandwidth (120 GB/s). Flat roof: throughput reached in prefill, a lower bound of the unpublished peak. Intensities computed for one feed-forward layer of Qwen2.5 with 8-bit weights and 16-bit activations; attention is ignored.

above it. The bound depends only on two numbers, the size of the model and the bandwidth of the machine, and it applies to every model that generates text. It is the key to comparing hardware in Section 2.4 and to the consequence drawn in Section 2.7.

Memory capacity sets a second, harder limit: the weights must fit. A model needs about two bytes per parameter in the 16-bit format used for training, one in 8 bits and half in 4 bits, plus a few small scale factors for each group of weights, needed to reconstruct their values (Figure 2.5). A 7-billion-parameter model in 16 bits needs 14 GB for its weights alone, beyond what most personal devices can spare [46]; in 4 bits it needs about 4 GB. The key-value cache, the activations, the operating system and every other model that runs at the same time come on top. A laptop with 16 GB can therefore hold a generator of 7–8 billion parameters in 4 bits next to a small verifier, but not much more.

2.4 Hardware for local inference

The cost model reduces the choice of hardware for local inference to three quantities. *Memory capacity* decides which models fit. *Memory bandwidth* decides how fast

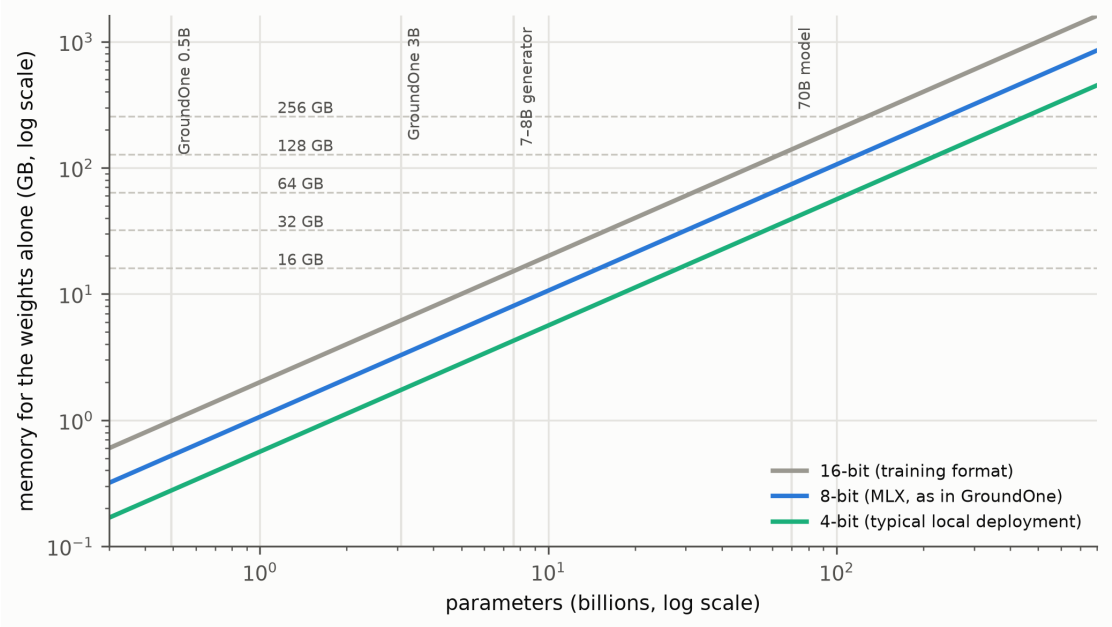


Figure 2.5: Memory needed for the weights of a model as a function of its size and precision (parameters times bits per weight, including half a bit of per-group scales for the quantised formats). Dashed lines: common memory sizes of local hardware. The key-value cache, activations and other running models come on top.

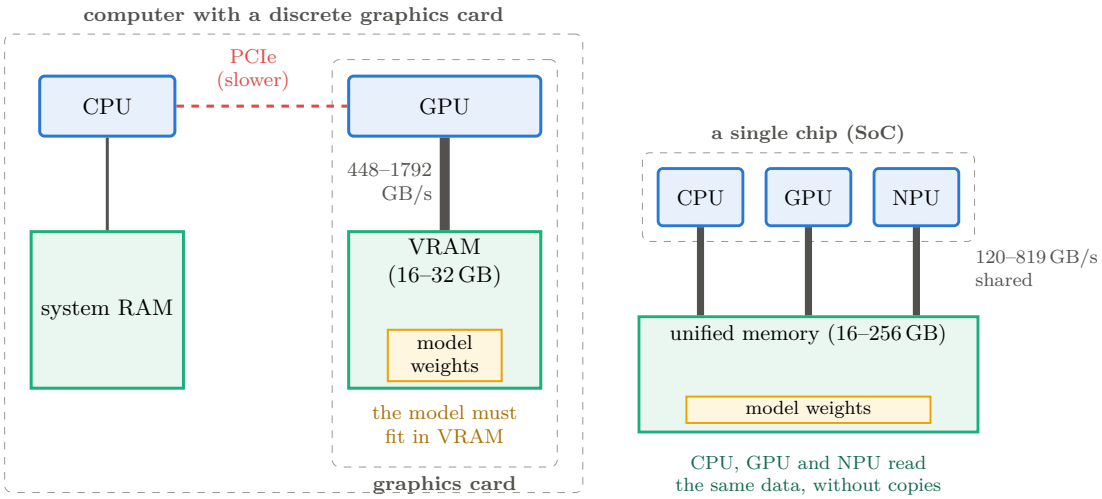
they can generate. *Computation* decides how fast they can read. Power and price complete the picture. This section surveys the hardware available in October 2026 along these axes, using the vendors' published specifications.

2.4.1 Classes of hardware

Two architectures dominate. In a *discrete* system, the graphics card has its own memory, fast but small, and the model must fit in it. In a *unified-memory* system, the processor and the graphics unit share one large pool of memory, so a model can use most of the machine's memory, at a lower bandwidth than a high-end graphics card. Apple's processors, NVIDIA's DGX Spark and AMD's Ryzen AI Max (Strix Halo) all follow the second design. Figure 2.6 compares the two architectures.

Table 2.2 and Figure 2.8 place the main options on the two memory axes. Figure 2.7 shows some of the systems discussed, to give a concrete idea of what they look like.

Laptops. Laptops with 16 to 32 GB and 120 to 150 GB/s, such as the MacBook Pro used in this thesis, are the most common personal computers on which local



(a) Discrete system: the GPU reads the weights from its own VRAM, very fast but limited to 16–32 GB; the rest of the system is reachable only through PCIe.

(b) Unified memory: CPU, GPU and NPU on the same chip share one pool of memory, larger but with lower bandwidth.

Figure 2.6: The two architectures of local inference hardware. Blue rectangles: compute units; green rectangles: memory; orange rectangle: the model weights; line width: indicative bandwidth. Capacity and bandwidth values from Table 2.2. In a discrete system a model that does not fit in VRAM must go through the much slower PCIe link; in a unified-memory system the model can occupy almost all of the machine's memory, but every generated token reads it at a lower speed than a high-end graphics card.

models run. They hold small models comfortably and models of 7–8 billion parameters in 4 bits. The thesis machine sits at the bottom left of Figure 2.8: if a verifier runs well there, it runs on everything above it.

Desktop machines with unified memory. A second class, within or close to a budget of about three thousand dollars, offers much more memory at moderate bandwidth. AMD's Strix Halo processor, sold for instance in the Framework Desktop, lets the graphics unit use up to 96 GB of its 128 GB of memory; the 128 GB configuration was priced at 1,999 dollars at launch in August 2025, with a processor power of 55 W [51]. Apple's Mac mini with the M5 Pro reaches 64 GB at 307 GB/s [48]. Such machines can hold models of tens of billions of parameters or mixture-of-experts models, which are composed of many sub-networks (the experts) of which each token activates only a part, so that decoding reads only the weights of the active experts, at the price of a slower generation than on a graphics card.



(a) 14-inch MacBook Pro with M4, the thesis machine.



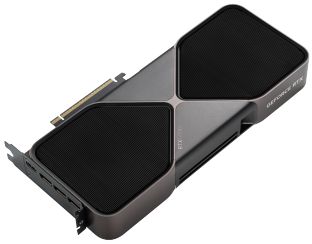
(b) Mac mini (2024, M4).



(c) Mac Studio.



(d) NVIDIA DGX Spark.



(e) GeForce RTX 5090 Founders Edition graphics card.

Figure 2.7: Some of the local inference systems compared in this section (images not to scale). Sources: (a), (c) Apple; (b) LoMit, Wikimedia Commons, CC0; (d), (e) NVIDIA. The vendors' images are promotional material, reproduced for the purpose of scientific documentation.

Dedicated AI machines. Above this budget, the Mac Studio with the M3 Ultra combines up to 256 GB with 819 GB/s [49], whereas NVIDIA's DGX Spark is a small desktop box, with a 240 W power supply, that contains 128 GB of unified memory with 273 GB/s of bandwidth and a Blackwell graphics unit rated at one petaflop (10^{15} operations per second) in 4-bit arithmetic, a figure that exploits sparsity, that is, the skipping of zero weights [50]. The two machines make opposite choices, and a direct comparison shows the cost model at work: on an 8-billion-parameter model with an 8,000-token prompt, the DGX Spark completes the prefill 3.8 times faster than the Mac Studio, thanks to its higher computation, while the Mac Studio generates 3.4 times faster, thanks to its three times higher bandwidth [53]. The same study combines the two machines, running the prefill on the Spark and the decoding on the Mac, which is the disaggregation of the two phases [42] brought to a desk.

Table 2.2: Hardware for local inference: vendors' published specifications (accessed 2026-10-11). Memory: range of the configurations offered; bandwidth: for those configurations; N/s: not stated by the source.

System	Memory (GB)	Bandwidth (GB/s)	Memory type	Source
MacBook Pro, M4 (thesis machine: 16 GB)	16–32	120	Unified	[45]
MacBook Pro / Mac mini, M5	16–32	153	Unified	[47, 48]
Mac mini, M5 Pro	24–64	307	Unified	[48]
MacBook Pro, M5 Max	36–128	460–614	Unified	[47]
Mac Studio, M4 Max	36–128	410–546	Unified	[49]
Mac Studio, M3 Ultra	96–256	819	Unified	[49]
NVIDIA DGX Spark (GB10)	128	273	Unified	[50]
Framework Desktop, AMD Strix Halo	32–128 (96 to GPU)	N/s	Unified	[51]
NVIDIA RTX 5090	32	1,792	Discrete	[52]
NVIDIA RTX 5080	16	960	Discrete	[52]
NVIDIA RTX 5060 Ti	16	448	Discrete	[52]

Graphics cards. Discrete cards offer by far the highest bandwidth, up to 1,792 GB/s for the RTX 5090, but at most 32 GB of memory, at a power of several hundred watts [52]. They are the fastest option for any model that fits, and unusable for any model that does not.

Phones and mobile processors. At the other end, phones have 6 to 16 GB of memory and a bandwidth of a few tens of gigabytes per second shared by all processors [30, 54]. Their neural processing units (NPUs) are fast at 8-bit matrix multiplication and therefore at prefill: on a phone, a 7-billion-parameter model in 4 bits reads its prompt at about 770 tokens per second on the NPU, against 37 on the graphics unit and 8 on the processor cores [54]. Research systems exploit this by running the prefill on the NPU and the decoding elsewhere [55].

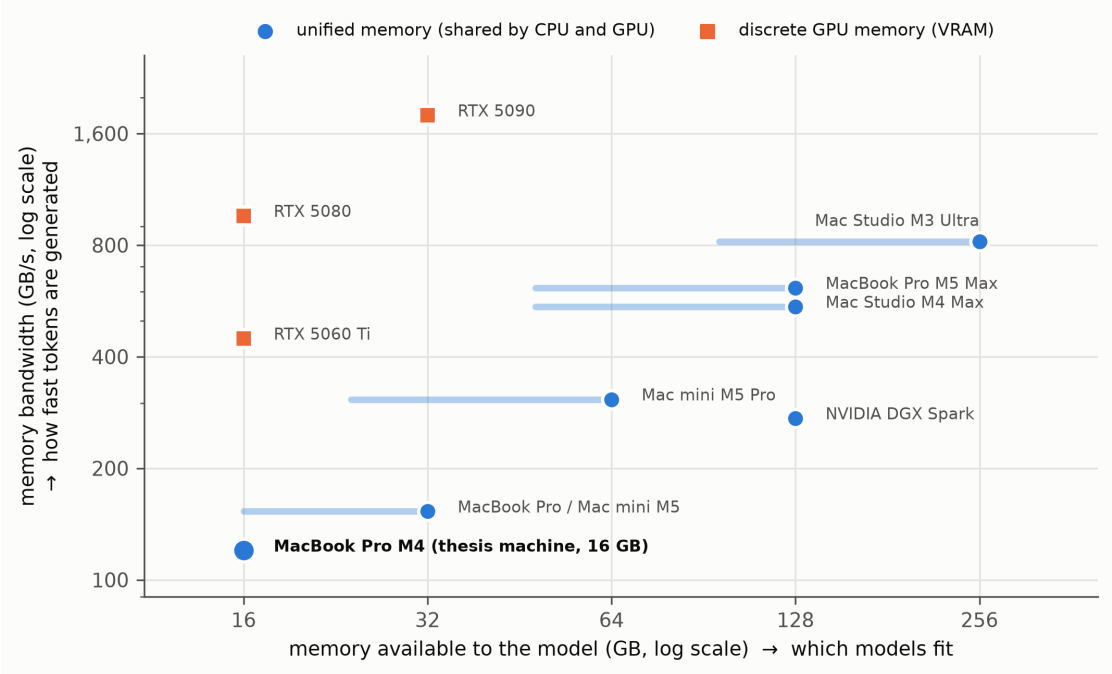


Figure 2.8: Local inference hardware on the two axes that decide what can run: memory capacity (which models fit) and memory bandwidth (how fast tokens are generated). Vendors' specifications: markers at the largest memory configuration, shaded bars over the range of memory sizes offered; the thesis machine is shown at its actual 16 GB.

2.4.2 The asymmetry holds on every class

The two phases respond to different properties of the hardware, and recent hardware generations make the point explicit. Apple reports that its M5 processor, which adds dedicated matrix units to each graphics core, reduces the time to first token, that is, the wait before the model starts answering, which is dominated by prefill, by 3.3 to 4.1 times with respect to the M4, while generation becomes only 1.19 to 1.27 times faster, in line with a memory bandwidth that grew from 120 to 153 GB/s [56]. Apple itself attributes the first figure to computation and the second to bandwidth. On edge devices, a study that compares 18 pairs of configurations finds that slowdowns accumulate in decoding, while the time to evaluate the prompt changes by at most 16 % [57]. On phones, for tasks whose output is short, prefill takes most of the latency [55]. Hardware is thus evolving to read faster, much more than to write faster.

Figure 2.9a applies the lower bound of Section 2.3.3 to the hardware of Table 2.2. On the fastest card, the 3B model could in principle generate a token in 1.8

milliseconds; on the thesis machine the floor is 27 milliseconds, and the measured time is close to it. Better hardware lowers the floor, but no hardware removes it.

2.5 Runtimes and quantisation

2.5.1 Runtimes

The software for local inference has matured quickly. llama.cpp provides inference in C and C++ without external dependencies, with quantisation and support for most consumer accelerators [2]. It also defined the GGUF format, which gathers in a single file the quantised model and everything needed to use it, such as the tokenizer and the conversation format. Tools such as Ollama and LM Studio build on it and reduce downloading and starting a model to a single command. MLX is a computing framework designed by Apple for the unified memory of its processors, with an interface close to NumPy [3]. Its language-model package supports quantisation and LoRA fine-tuning directly on the device [58], and it is the runtime used throughout this thesis.

The runtime matters as much as the chip. A comparison of five runtimes on the same Apple machine reports decoding rates that differ by more than an order of magnitude, with MLX and MLC-LLM at the top and PyTorch's generic Metal back end at the bottom [59]; the study's numbers are not fully consistent with each other and should be read as indicative, but the spread is large enough to survive the uncertainty. On phones, an evaluation of several runtimes and devices finds large differences in speed, energy and thermal behaviour between runtimes and between devices [60].

2.5.2 Specialised inference engines

Next to the general runtimes, a second family of software appeared in 2026: inference engines written for a few models, which give up generality to push further the memory limits described in Section 2.3. DwarfStar (ds4), developed by Salvatore Sanfilippo, is a native engine optimised for a few large open-weight models, starting from DeepSeek V4 Flash, and aimed at Macs with at least 96 GB of memory, the DGX Spark and Strix Halo machines [61]. Two of its choices concern this thesis directly. The first is a key-value cache saved on disk: a conversation that has already been read can be resumed without repeating the prefill, which for models of this size is the most expensive part of a long request. The second is reading the weights directly from the SSD, which makes it possible to run models larger than the available memory, at the price of speed.

Colibri takes this second idea to the extreme [36]. It exploits the fact that a mixture-of-experts model uses only a small part of its weights for each token: it

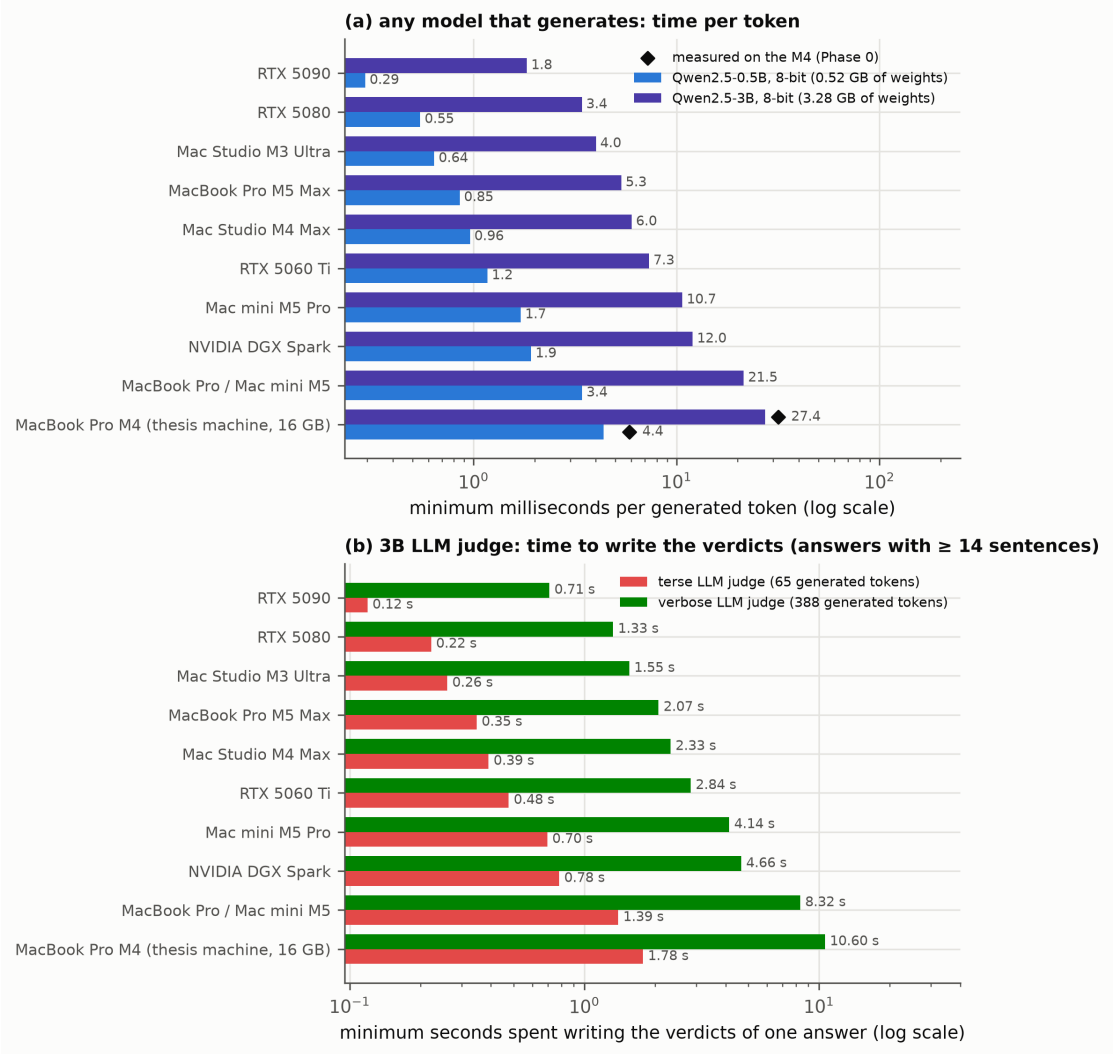


Figure 2.9: Lower bounds on the cost of generating text on each class of local hardware, computed as weight bytes divided by the vendor's bandwidth (one request at a time; key-value cache ignored). (a) Time per generated token with the two GroundOne base models in 8 bits; diamonds: decode time measured on the thesis machine. (b) Minimum time that a 3B LLM judge spends only on writing the verdicts for an answer of fourteen or more sentences (median generated tokens of the latency protocol). A decision model generates no token, so both costs are zero for it on every machine.

keeps the always-active part in memory and reads the experts from disk when the router requests them. In this way GLM-5.2, a model of 744 billion parameters, runs on a laptop with 25 GB of memory. The price is the one predicted by the cost model: the bandwidth of an SSD is a few gigabytes per second, far below that of memory, and according to the authors generation drops to 0.05–0.1 tokens per second when cold on that laptop and to 1.83 tokens per second on a 128 GB Strix Halo machine [36]. It is the idea of LLM in a flash [46] applied to expert models: memory capacity stops being a hard limit and turns into a bandwidth limit, an even stricter one. Figure 2.10 illustrates the mechanism.

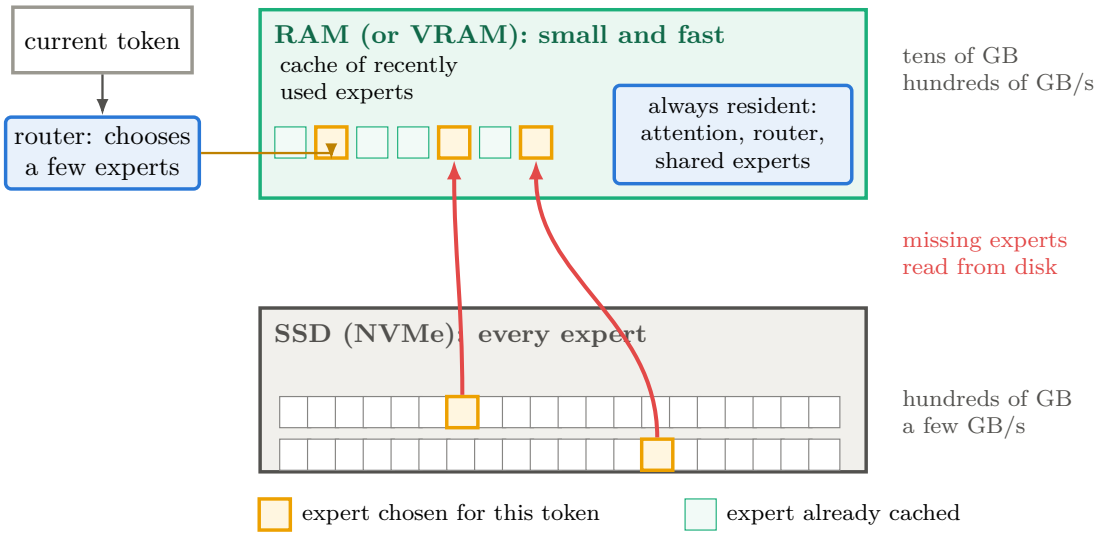


Figure 2.10: Streaming experts from the SSD in an engine such as Colibri. Memory holds the part of the model used by every token (attention, router, shared experts) and a cache of recently used experts; every expert is stored on the SSD. For each token the router chooses a few experts: those already cached are used at once, the missing ones are read from disk, at a bandwidth of a few gigabytes per second. For example, GLM-5.2 keeps about 10 GB resident and leaves about 429 GB of weights on disk [36]. The diagram is not to scale.

Colibri matters to this thesis for a second reason. It exposes the same System One interface as Jev, through which any model it runs answers closed questions by reading the probability of each option instead of generating the answer, and it includes three decision models natively: Laya, GLiNER2.5-Decide and Clef [36]. The authors report that, with Qwen3.6, four questions about the same document are answered 5.7 times faster than by generating the same answers on the same CPU. This is a figure reported by the project and not verified in this thesis, obtained with a different model and hardware, but it points in the same direction as the

results of Chapter 7. That an engine designed to run huge models on modest hardware adopts the reading of decisions as a first-class mode confirms that the decision-model idea is spreading precisely where generation costs the most.

2.5.3 Quantisation

Quantisation stores the weights with fewer bits than the 16 used in training. On local hardware it serves two purposes at once: the model fits in less memory, and decoding becomes faster, because decoding time is dominated by reading the weights. Table 2.3 summarises the methods that shaped the field.

Table 2.3: Weight quantisation methods for language models.

Method	Idea and main result	Source
LLM.int8()	8-bit weights and activations; the few dimensions with very large anomalous values (outliers) are kept in 16 bits. No loss up to 175B parameters; saves memory but does not speed up small models.	[62]
GPTQ	3–4-bit weights, quantised layer by layer by estimating how much each weight affects the error. 4-bit OPT-175B loses 0.03 perplexity[a]; small models lose more; 3.2–4.5× faster decoding from bandwidth.	[63]
AWQ	4-bit weights; the 1 % of channels that matter most, found from activation magnitudes, are protected by scaling. Robust to the calibration data; designed for on-device use.	[64]
GGUF k-quants	2–8-bit block formats of llama.cpp, the de facto standard for consumer distribution.	[2]
MLX affine	4- or 8-bit weights with a scale and a bias per group of 64; supports LoRA training on the quantised model (used in this thesis).	[58]
QLoRA	4-bit normal-float base model with 16-bit adapters, for fine-tuning on limited memory.	[65]

[a] *Perplexity* measures how well a model predicts a text: the lower it is, the less the model is "surprised" by the words it encounters.

Two results from this literature matter for this thesis. The first is that quantisation helps the bandwidth-bound phase. GPTQ attributes its speed-ups entirely to the matrix-vector products of decoding and explicitly obtains no reduction in computation [63]; AWQ illustrates the same point with a roofline: on a desktop graphics card, a 200-token prefill takes 10 milliseconds while generating 20 tokens takes 310 [64]. Reducing the bits of the weights therefore accelerates a model that generates far more than one that only reads. The second is that small models are more fragile. GPTQ loses much more accuracy at 4 bits on a 1.3-billion-parameter

model than on a 175-billion one, and models of three billion parameters or fewer become unusable at 3 bits on phones [63, 60]. LLM.int8() shows that a handful of outlier features control a large share of the attention probability mass [62]. For a model whose output is a probability that must be calibrated, rather than a text that must merely be fluent, these observations argue for a conservative precision: GroundOne uses 8-bit weights. A measurement made in this thesis confirms that, at this precision, quantisation costs no accuracy: applying the same adapter of the 0.5B model to the original 16-bit weights leaves the three-way balanced accuracy on T1 practically unchanged (0.768 against 0.769), the probabilities change by 0.011 on average and the predicted class agrees on 98 % of the sentences (Section 7.11).

2.5.4 Faster generation without fewer tokens

A different family of methods accelerates generation without changing the model. In speculative decoding, a small draft model proposes several tokens, and the large model checks all of them in a single forward pass, accepting the longest prefix that it would have produced itself; the output is identical to that of the large model, and the speed-up reaches two to three times [66]. The method is useful for any system that must generate, but it does not change the nature of the cost: every block of proposed tokens still requires a forward pass of the large model, and the total amount of computation grows [66]. It makes writing cheaper; it does not make it free.

2.6 Energy

On a battery-powered device, energy is a property of the method, not only of the hardware. Measurements on datacenter hardware show that the energy of a request grows with the number of generated tokens and depends strongly on model size and task [67, 68]. On phones, a 3-billion-parameter model in 4 bits spends between 0.58 and 0.76 joules per generated token, with sustained power around 14 W and surface temperatures close to 48 °C [60]; a rule of thumb from work on small models puts the cost at about 0.1 joules per generated token for each billion parameters [39]. Moving the prefill to a phone's NPU reduces its energy by 1.9 to 4.3 times with respect to the graphics unit [55].

The distinction between the two phases also applies to power. Prefill keeps the arithmetic units busy and draws more power; decoding waits for memory and draws less, but for much longer [42]. A method that saves time therefore saves energy too, but by a smaller factor, and Chapter 7 measures exactly this effect: GroundOne spends on average 5.5 joules per answer with the 0.5B model and 33 with the 3B, less than any generative LLM judge of the same size, but with an advantage smaller than in time (Section 7.10).

2.7 Consequences for verification on the device

The chapter can now be summarised in one observation: on a local device, which serves one request at a time, reading a token costs far less than writing one, fourteen to sixteen times less on the thesis machine (Sections 2.3.1–2.3.3), and recent hardware generations accelerate reading much more than writing (Section 2.4.2). This has a direct consequence for a verifier that runs next to the generator.

An LLM judge verifies an answer by reading the document and the answer and then writing its verdicts. It pays a prefill, like any method that reads the document, and then one decoding step for every token it writes. In the latency protocol of this thesis (Section 7.9), a terse 3B LLM judge writes a median of 65 tokens for an answer of fourteen or more sentences, and a verbose one, which explains each verdict, writes 388. The lower bound of Section 2.3.3 turns these counts into a minimum time that no implementation of standard decoding with 8-bit weights can beat (Figure 2.9b): on the thesis machine, at least 1.8 seconds for the terse LLM judge and 10.6 for the verbose one, on top of the prefill; even on the fastest graphics card, 0.12 and 0.71 seconds.

A decision model removes this term altogether. It reads the document and the answer once and takes every decision from the logits computed during that single reading, so its cost is the prefill alone: the compute-bound phase, the one that local hardware performs well and that new hardware accelerates most. Its advantage over an LLM judge is therefore not a property of one machine but of the cost structure of local inference, and it is largest exactly where inference is local. Locality, which at first sight is only a constraint on the size of the model, becomes the reason why the design matters.

Three requirements for the verifier follow from this chapter, and they guide the method of Chapter 4. It must generate no tokens, so that its cost is a single prefill. It must use a model small enough to share a 16 GB machine with a generator and a retriever, quantised conservatively because its output is a probability. And it must read its decisions from logits, which only a local model exposes, so that they can be calibrated.

Chapter 3

Background and Related Work

This chapter reviews the work on which GroundOne builds and the work against which it is compared. The cost of inference on a local device, from which the design follows, was already treated in Chapter 2. The chapter starts from decision models, that is, from the idea of reading the decisions of a language model instead of having it generate them, and from their rapidly growing ecosystem. It then turns to the problem of hallucination in retrieval-augmented generation (RAG) and to the methods that verify answers against their sources. It closes with calibration, with parameter-efficient fine-tuning and with the gap that this thesis sets out to fill.

3.1 Decision models

The term *System One model* was introduced by TypeSafe with Jev, a model offered as a hosted service. Jev answers closed questions about a given state, that is, about the context supplied in the request, and returns typed outputs, such as a boolean, a choice among options or a score, without generating text [14]. The name evokes the fast, intuitive thinking described by Kahneman, as opposed to deliberate reasoning. The vendor reports latencies ranging from tens to hundreds of milliseconds and large savings with respect to LLM judges. These figures, however, remain vendor claims, and the architecture is not disclosed.

The open implementations that followed differ in how they produce the probabilities, and three families can be identified. The most common one presents the question as a multiple choice and keeps the head of the language model, that is, the final layer that assigns a score to every token of the vocabulary: the decision is read from the logits of the option letters, at the position where the model would write

the answer. SemIf applies this readout to a 4B-parameter model kept frozen, that is, without modifying its weights, and reports that 21 criteria about one document are answered in about one second; the best generative baseline able to produce valid JSON, by contrast, takes more than five seconds [15]. Rizzo-Flow, unlike SemIf, fine-tunes a 4B model with LoRA on typed decisions and exposes the same interface as Jev. Its authors report that fine-tuning improves both its accuracy and its calibration [16]. It is one of the two general-purpose decision models used as baselines in this thesis.

The second family encodes the state and each candidate answer separately, and compares their embeddings, that is, the numerical vectors that represent each text. Contrastive Language Models, which belong to this family, are very fast when the candidates are numerous and reusable. In the comparison reported by the authors themselves, however, they agree with Jev's decisions much less than a model in which state and candidate interact [69]. The third family uses small encoders[1] with a dedicated decision head. Laya, for example, is built on a ModernBERT encoder of about 400M parameters and assigns a score to each option at the corresponding mask token. The questions of a request are evaluated in a single call, grouped into a batch, but each of them is encoded together with its own copy of the context [70, 71].

The evidence supporting these systems is uneven. Most of it comes from the authors themselves, often on a single machine and on a single benchmark. Independent evaluations are recent and less favourable to small open models. An evaluation of Jev on 37 datasets finds well calibrated the probabilities that the model assigns to choices among options. Binary probabilities, instead, rank examples well but are poorly positioned with respect to a fixed threshold of 0.5. The same study reports for Jev a mean balanced accuracy of 78.6, expressed as a percentage, on LLM-AggreFact [20]. An independent reproduction of Laya observes a calibration different from the declared one and a strong dependence on a calibration temperature estimated on the target data [21]. A benchmark built on contrastive pairs, that is, on nearly identical decisions that differ in a single decisive detail, places small open models far below Jev [72]. Finally, it has been shown that, when the label of an option and its definition conflict, typed decision models follow the label rather than the definition [71].

The ecosystem, meanwhile, is growing rapidly. On 9 October 2026 the Hugging Face hub officially introduced a tag dedicated to these *decision models* [17]. The following day the tag already grouped about 1,500 models, published by about 600 authors and of very uneven quality. This thesis uses the term *decision model*,

[1]The *mask token* is a special token that, during pre-training, takes the place of the words the model has to reconstruct.

adopted by the ecosystem; *System One* remains the name with which TypeSafe introduced the category, and in what follows it appears only as the name of the Jev-compatible interface.

A few weeks after the release of Jev, established organisations also published their own open decision models. Cloudflare's Clef adds a dedicated decision head to a decoder and answers all the questions with a single forward pass over the state: this is the architecture that had only been hypothesised for Jev [19]. Liquid AI's d1-3B reads the decision from a restricted set of answer letters, with a 3B model whose state is shared by all the questions [18]. Perplexity, finally, fine-tunes a 27B model with a head on the letters [73]. A community leaderboard, the Decision Index, evaluates these models on a common battery of public benchmarks, including RAGTruth evaluated at the level of the whole answer [22]. The results draw a sharp line. General-purpose decision models with fewer than one billion parameters barely beat the trivial baseline on RAGTruth, and only models with tens of billions of parameters reliably detect hallucinated answers.

The studies on Jev add three observations. Its probabilities are calibrated when aggregated over many tasks, but not on a single task. Thresholds and temperatures have to be estimated on the target workload [74, 75]. Finally, choosing the threshold through a lower confidence bound makes it safer to put into production an LLM judge that returns only decisions [75]. Runtimes have adapted as well: llama.cpp now natively supports the System One interface [2]. None of these models, however, is specialised in groundedness, that is, in the adherence of the answer to the document. Nor does any of them report results of three-class, sentence-level verification on the benchmarks used in this thesis.

The letter readout has also been studied outside the decision-model literature, in multiple-choice question answering. Here two ways of obtaining the answer of an instruction-tuned model can be compared: reading which letter has the highest probability as the first token, or letting the model write out the answer in full. The two answers often do not coincide [76]. The readout, moreover, favours particular option identifiers, such as certain letters, and this bias can be estimated and removed [77]. Both results concern models used zero-shot. They suggest that a letter readout, if it is to serve as a decision, needs to be trained, calibrated or both. Moreover, no open decision model has been specialised in groundedness verification and then compared, under controlled conditions and on the same hardware, with LLM judges, with specialised verifiers and with general-purpose decision models, one of them of the same size.

3.2 Retrieval-augmented generation and hallucination

Retrieval-augmented generation (RAG) gives a language model access to documents on which it was not trained: a retriever selects the passages relevant to the question, and the model writes the answer with these passages in its context [1]. It is the standard approach for building assistants over private documents, and it can run entirely on a local machine. It does not, however, eliminate hallucinations. Surveys of the literature distinguish two types of error. Errors of *factuality* are measured against world knowledge, errors of *faithfulness* against the input or the context given to the model [9]. A RAG answer, in particular, can be unfaithful to the very passages it received, by contradicting them or by adding information that they do not contain.

RAGTruth documents how frequent this is. The dataset collects about 18,000 answers, written by six language models on three RAG-style tasks, and annotates in them, with character-level precision, the spans of text that conflict with the context or are not supported by it [8]. RAGTruth's distinction between conflicting content and baseless content corresponds to the classes *contradicted* and *not mentioned* adopted in this thesis.

3.3 Groundedness verification

Verifying an answer against its source is a problem closely related to natural language inference (NLI). In NLI the task is to establish whether a premise implies a hypothesis (entailment), contradicts it or is neutral with respect to it. Large NLI datasets provide training data: MultiNLI [78] and ANLI [79], the latter built adversarially, that is, by collecting examples written on purpose to fool the models of the time. VitaminC [80] adds contrastive pairs, in which a minimal revision of a Wikipedia sentence flips the verdict: the model is thus forced to look at the precise fact rather than at word overlap.

Applying these models to long generated texts requires decomposing them. SummaC splits both the document and the summary into sentences and aggregates the inference scores computed sentence by sentence [81]. FActScore, instead, decomposes long generations into atomic facts and verifies them one by one [82]. Sentence-level verification, adopted by GroundOne, lies between a single score for the whole answer and the decomposition into atomic facts. Sentences are the natural units of an answer, the user can act on a flagged sentence, and the number of decisions grows with the length of the answer.

In practice two families of verifiers prevail. The first is that of specialised encoders, small and fast. AlignScore trains a single alignment function on a

broad mixture of tasks [11]. MiniCheck trains small verifiers on synthetic data, built so as to require reasoning over several sentences, and matches much larger models on grounding benchmarks [12]. HHEM is a hallucination evaluation model of the same kind, released as an open model [13]. These models evaluate each document–sentence pair independently and return a single probability of support.

The second family uses a language model as a judge, the LLM judge, following the LLM-as-a-judge scheme[2] [83]. Evaluation frameworks for RAG such as RAGAS [84] and ARES [85] rely on LLM judges of this kind. Lynx fine-tunes an LLM judge dedicated to hallucination detection [86], while Self-RAG trains the generator itself to emit critique tokens, that is, special tokens with which the model evaluates the retrieved passages and its own output [87]. LLM judges are flexible, but their cost grows with the amount of text they write. Small models used as LLM judges, moreover, often violate the required output format. Guaranteeing an answer that the parser can interpret then requires grammar-constrained decoding, which at each step admits only the tokens compatible with the required format [88]. The reference benchmark for both families is LLM-AggreFact, which gathers eleven manually annotated grounding datasets and maps them to a common binary protocol [12, 89]. Figure 3.1 compares the two families with the specialist decision model studied in this thesis.

3.4 Calibration and selective prediction

A probability is calibrated when, among the predictions made with a given confidence, the fraction of correct ones matches that confidence: in a calibrated model, for example, about 80 % of the predictions made with confidence 0.8 turn out to be correct. Modern neural networks are often poorly calibrated, and a simple and effective correction is temperature scaling: the logits are divided by a single constant, the temperature[3], estimated on held-out data [90]. The usual measure of calibration is the Expected Calibration Error (ECE), which compares confidence and frequency of correct predictions within confidence bins [91] (Section 4.4). For language models queried through a prompt, a per-class bias may also be needed, to correct a systematic preference for some answers [92].

Language models know, within limits, what they know: on closed questions, the probabilities of their tokens are often informative about the correctness of the

[2]In the *LLM-as-a-judge* scheme a language model receives, through a prompt, the text to be evaluated and the evaluation criteria, and writes its own judgement, sometimes preceded by a rationale.

[3]With a temperature $T > 1$ the probabilities become less extreme, with $T < 1$ more extreme. Since all the logits are divided by the same constant, the most probable class does not change.

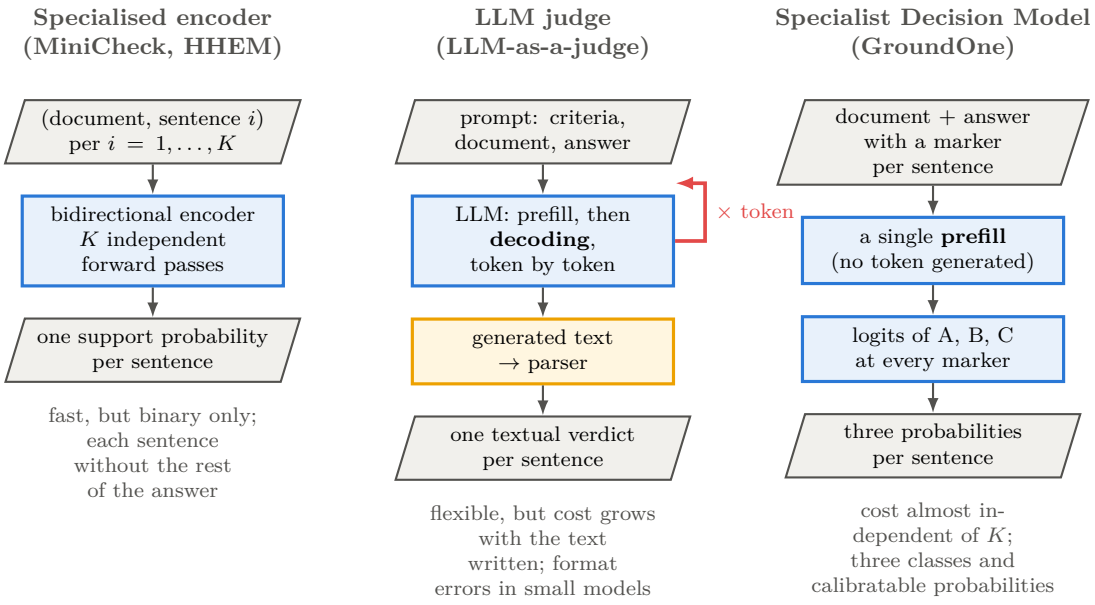


Figure 3.1: Three ways of verifying the K sentences of an answer. A specialised encoder evaluates each (document, sentence) pair in a separate forward pass and returns only a probability of support. An LLM judge reads everything, but writes its verdict token by token (red loop), so its cost grows with the text produced and the output has to be interpreted by a parser. A specialist decision model reads document and answer in a single prefill and derives three probabilities from each marker, without generating any token.

answer [93]. When the probabilities are not accessible, the model can be asked to express its confidence in words. In large models fine-tuned with human feedback, this verbalised confidence can be better calibrated than probabilities estimated by sampling [94]. Systematic evaluations, however, find it coarse and overconfident, with values concentrated on a few round numbers [95].

A calibrated probability becomes useful through selective prediction: the system acts on the decisions it is confident about and defers the others. The risk–coverage curve describes this trade-off. For each share of accepted decisions (the coverage) it reports the error rate on them (the risk), and thus shows how the error rate falls as more decisions are deferred [96]. Figure 3.2 illustrates its typical shape.

3.5 Parameter-efficient fine-tuning

Specialising a pre-trained model does not require updating all of its weights. Parameter-efficient fine-tuning (PEFT) methods train only a small share of the

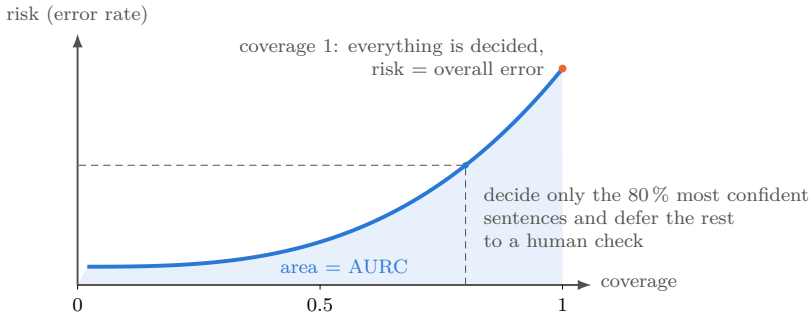


Figure 3.2: Sketch of a risk–coverage curve (illustrative shape, not a result). The decisions are sorted from the most to the least confident; for each share decided (coverage) the error rate (risk) of the accepted decisions is reported. At coverage 1 the risk equals the overall error; giving up the least confident decisions lowers the error. The area under the curve (AURC) summarises the curve in one number: the smaller it is, the better the confidence separates correct decisions from wrong ones.

parameters and leave the rest of the model unchanged. Low-rank adaptation (LoRA) freezes the model and, for some weight matrices, trains only a low-rank[4] update. At the end of training, the update can be merged into the weights or kept as a small separate adapter [24]. Applying LoRA to all linear layers matters more than using a high rank, and the technique combines with quantised base weights, so that even large models fit in little memory [65]. The Qwen2.5 family offers open models starting from 0.5B parameters with a shared tokenizer, and therefore makes it possible to compare different sizes with the same recipe [23]. Figure 3.3 sketches a LoRA adapter applied to a linear layer.

Fine-tuning on small datasets, however, is notoriously unstable. Results vary with the random seed, which determines both the initialisation of the new parameters and the order of the training data [97]. Some training runs fail altogether, and this behaviour is linked to optimisation details such as the absence of bias correction in Adam[5] [98]. Gradient clipping, which bounds the norm of the gradient,

[4]The *rank* of a matrix is the maximum number of its linearly independent rows (or columns). LoRA constrains the correction learned for a $d \times k$ weight matrix to have a small rank r. A matrix of this kind can be written as the product of two narrow matrices, one $d \times r$ and one $r \times k$, so it suffices to store and train these two: $r(d + k)$ parameters instead of the dk of the full matrix. In GroundOne 3B, for example, a 2048×2048 matrix has about 4.2 million weights, while its rank-16 correction requires about 65 thousand, 1.6 %.

[5]Adam is an optimiser that updates each weight using two moving averages, one of the gradient and one of its square, called moment estimates. Since these averages start from zero, in the first steps they underestimate the true values; *bias correction* rescales them to compensate. AdamW

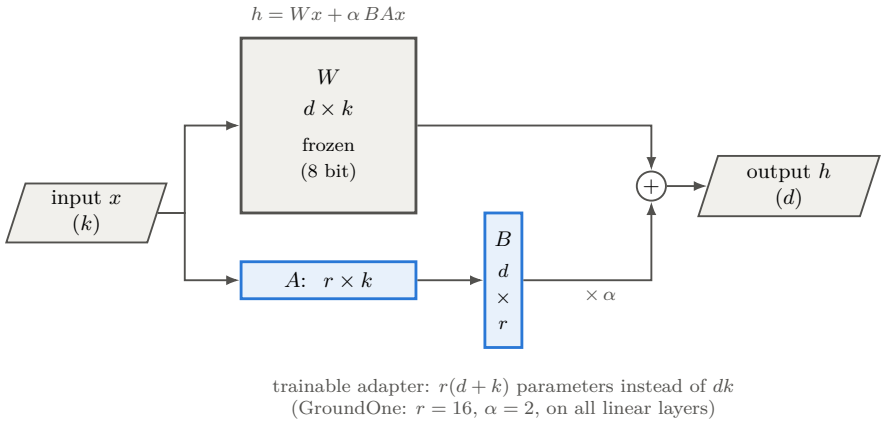


Figure 3.3: Low-rank adaptation applied to a linear layer. The weight matrix W stays frozen (8-bit in GroundOne); only the two small matrices A and B are trained, and their product, scaled by α, is added to the output of the layer. With rank r the adapter has $r(d+k)$ parameters instead of the dk of W.

is finally the standard defence against occasional exploding updates [99].

3.6 Research gap

The components needed for a local, fast and calibrated verifier already exist, but separately: compact open models and runtimes that run them on a laptop, the decision-model idea of reading decisions instead of generating them, specialised verifiers and post-hoc calibration. What is missing is an independent, controlled study that brings them together on a single task. In particular, no work has measured whether a compact decision model, specialised in groundedness verification, can match LLM judges and dedicated encoders on this task. Nor is it known where its advantage would come from: from specialisation, from the readout or from model size. Finally, it remains to be established whether its probabilities are good enough to set thresholds and defer decisions, and how its time and energy compare with those of a LLM judge on the same consumer hardware. The rest of the thesis addresses these questions.

is the variant of Adam that applies weight decay separately from the gradient.

Chapter 4

GroundOne: Method and Implementation

GroundOne is a compact language model used as a decision model. It reads the document and the answer once and, without generating text, returns a verdict for each scored sentence of the answer, deriving it from the logits of a few answer letters. The chapter first describes the decision the model makes and the prompt that gathers all the decisions about one answer into a single forward pass. It then presents the training objective, the calibration of the output probabilities and the rule that turns the probabilities into the returned labels, and it closes with the software that implements all of this.

4.1 Task and decision

For every sentence of an answer, GroundOne chooses among three classes, defined in Section 5.1: with respect to the document, the sentence is *supported*, *contradicted* or *not mentioned*. The *document* is the whole context that the generator received. It may comprise several retrieved passages, possibly from different sources, concatenated in the prompt: this is the case of question answering in RAGTruth (Section 5.2). A sentence is therefore supported when at least one passage backs it, including by combining information scattered across different passages.

Each class corresponds to a letter (A, B and C), and the model returns a probability for each of them. This is the only question the model receives: everything else is derived from these three probabilities. The binary groundedness score is the probability that the sentence is supported. The strict policy flags every unsupported sentence, while the lenient policy flags only contradicted sentences. A single forward pass is therefore enough for all these uses.

Asking the model a separate question for each policy would add nothing: once

the class of a sentence is known, how each policy LLM judges it is already known too. Separate questions would only multiply the number of decisions and, in training, would risk teaching the model mutually inconsistent answers about the same sentence.

4.2 All verdicts in a single forward pass

The prompt contains a short instruction, the document and the answer. The sentences of the answer are numbered, and every sentence associated with a decision is followed by a marker, that is, a short fixed string that indicates where the verdict is read: the string " `Verdict:`". Sentences without checkable content (Section 5.3) remain in the text as context, but without a marker. The model processes the whole sequence once. At the last token of each marker the logits of the letters A, B and C are read and normalised with a softmax[1] restricted to these three letters. In a decoder each position sees only the preceding ones, so the verdict on sentence i depends on the document and on sentences $1, \ldots, i$. This is exactly what verification needs, because references, such as pronouns, point backwards. What follows the last marker affects no verdict, so the sequence ends there. Figure 4.1 shows the structure of the prompt and the points where the readout takes place.

Two details make the readout exact and efficient. The first concerns exactness: the prompt is built by tokenising its parts separately and then concatenating them, so that the position of every marker is known exactly and stays identical in training and at inference. The second concerns efficiency: the output projection onto the vocabulary is applied only at the marker positions. The transformer body processes the whole sequence, but the large vocabulary matrix is multiplied only by the few hidden states that are actually read, that is, by the vectors the model computes at the marker positions.

The marker readout also determines the cost of a verification. Let S denote the total length of document and answer, K the number of decisions and m the length of a marker in tokens. The decision model requires a single prefill of $S + Km$ tokens. An LLM judge, in contrast, must also produce its verdicts during decoding. If each verdict requires n tokens and each decoded token takes a time t_{dec}, the cost of the LLM judge is roughly $\mathrm{prefill}(S) + K\,n\,t_{\mathrm{dec}}$. A marker is only three tokens long, whereas on the reference hardware decoding is much slower than prefill for the same number of tokens; the gap between the two costs therefore widens as

[1] The *softmax* turns a vector of scores into a probability distribution: each score z_i becomes $e^{z_i} / \sum_j e^{z_j}$. Restricting it to three letters means computing it only over their three logits, ignoring the rest of the vocabulary.

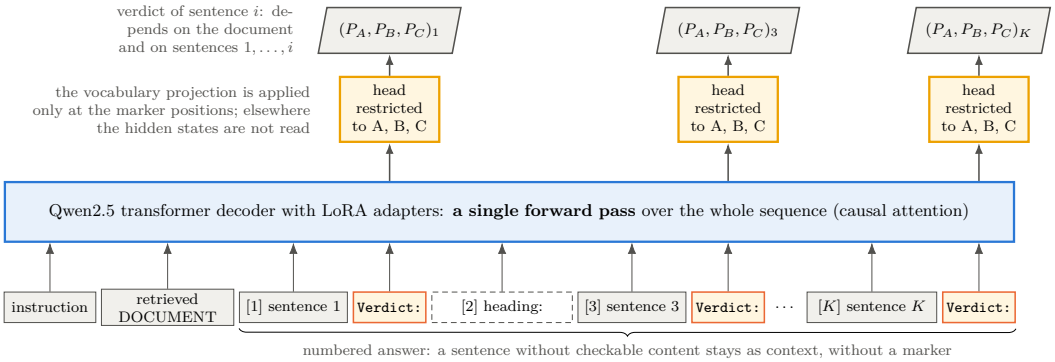


Figure 4.1: The marker prompt of GroundOne (design b). The sentences of the answer are numbered and every scored sentence is followed by the marker " `Verdict:`"; a sentence without checkable content (here a heading) remains as context, without a marker. The model processes the whole sequence in a single forward pass, and the output head, restricted to the letters A, B and C, is applied only at the marker positions. Because of causal attention, the verdict on sentence i depends on the document and on sentences 1 to i.

K grows. On the M4 used in this thesis, with the 0.5B model prefill costs about 0.37 ms per token and decoding 5.9 ms per generated token; with the 3B the two values are 2.2 and 31.8 ms. One decoded token thus costs as much as fourteen to sixteen tokens in prefill. The two phases also stress the hardware differently. Prefill processes many tokens together and is limited by compute, whereas decoding produces one token at a time and is limited by the bandwidth needed to read the weights [10, 42]. The difference resurfaces in the energy measurements of Chapter 7. Figure 4.2 translates the cost model into seconds per answer, using only these measured constants.

There is also a second solution, which reads the shared prefix once and then appends a short question for each sentence, evaluating the questions in a batch. Unlike the marker solution, which needs only a single forward pass, this one requires a KV cache replicated K times. It is kept as a fallback and is the natural format for an untrained model, which understands an explicit question but has no reason to interpret a marker as a request for a verdict. For this reason it is the format used by the untrained baseline (Section 6.2). Figure 4.3 compares the two solutions.

The order of the prompt has a second consequence for the cost. Instructions, document and answer appear in this order, and attention is causal: the keys and values computed for a prefix depend only on that prefix. The KV cache of the instructions, which are the same for every request, and that of the document can therefore be kept and reused exactly by any later answer on the same document, which then costs only the reading of the answer and its markers. This is the case

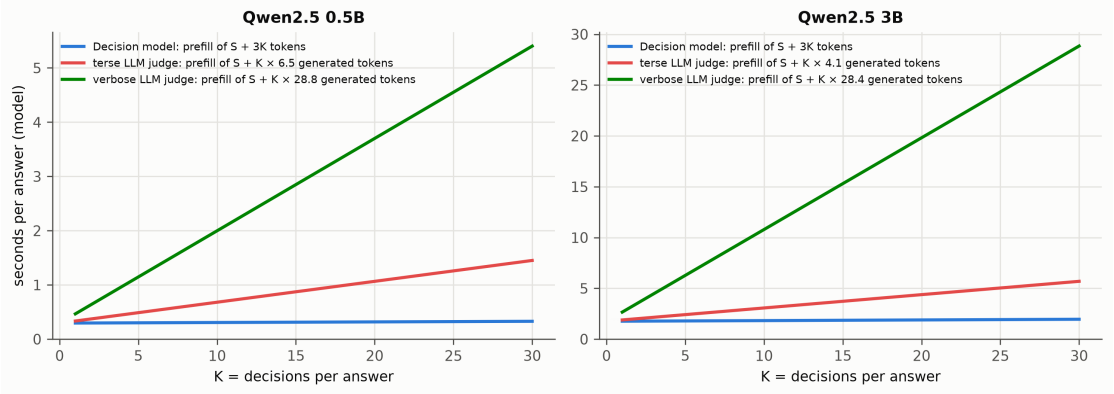


Figure 4.2: Estimated cost of verifying an answer of $S = 800$ tokens as a function of K, computed with only the constants measured on the M4: the per-token time of prefill and decoding and the mean number of tokens each LLM judge generates per verdict on the latency subset (mean K 8.7). It is an analytical model, not a measurement: the measured latencies are in Section 7.9. For the same text, the cost of the decision model grows by only three prefill tokens per decision, that of the LLM judges by a few decoding tokens per decision. In real answers, however, more sentences also mean a longer text, whose prefill both pay: Section 7.9 separates the two components.

of a rewritten answer verified again, of several alternative answers generated from the same sources, or of several questions on the same library. Prefix reuse is not an exclusive advantage of the decision model: the prompt of an LLM judge also starts with instructions and document, and can reuse them in the same way. The marker format, however, makes it particularly effective, because after the prefix only the answer remains to be read, and every additional sentence costs the three tokens of its marker. The same argument holds inside the answer: when a rewrite changes a sentence, the part of the answer before it is unchanged, and so are the verdicts at its markers, which depend only on what precedes them; only what follows the first change has to be read again. An LLM judge cannot exploit this in the same way, because its verdicts follow the whole answer and must all be written again. GroundOne keeps the instructions, the last documents read and the last answer verified, both in the service and in the application of Chapter 8; Section 7.9.1 analyses the composition of the prefill and measures the effect of caching for GroundOne and for the LLM judges.

A classifier, not a faster generator. The readout restricts the decision to three letters, and the restriction is not a detail. After the marker, the fine-tuned model would not write the letter that is read. Its most probable next token is in fact

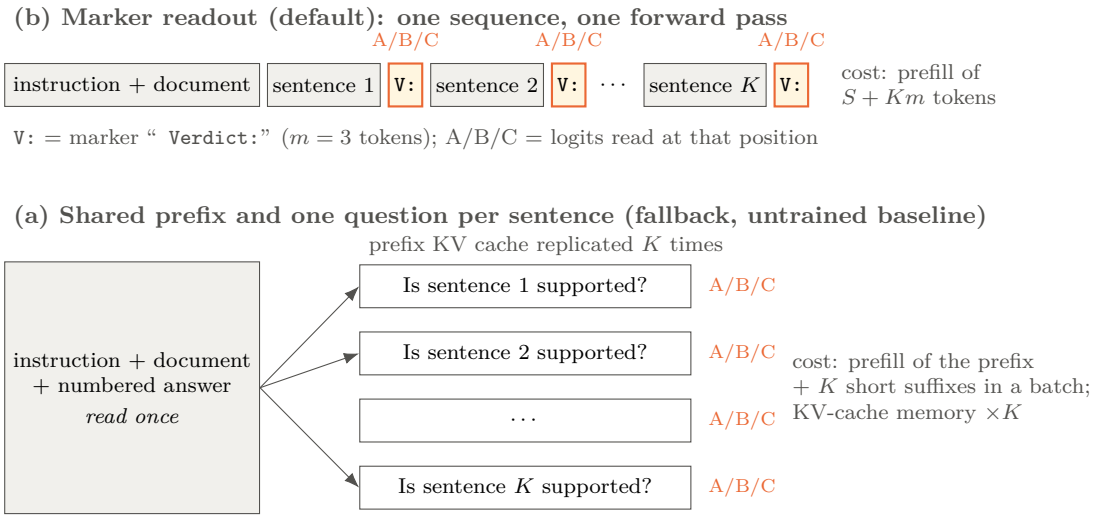


Figure 4.3: The two solutions for obtaining K verdicts from a single reading. (b) The marker readout, adopted as the default, inserts a marker after every sentence and reads all the verdicts in the same pass. (a) The fallback solution reads the shared prefix once and appends a short question for each sentence, evaluated in a batch; it requires, however, K copies of the KV cache of the prefix, and it is the natural format for an untrained model.

the same letter preceded by a space, which for the tokenizer is a different token. At the markers, the three readout letters together collect only 4–5 % of the probability mass. A loss restricted to these letters trains their relative logits, not their absolute probability, and the readout therefore behaves as a three-class classifier grafted onto the language model. This is what defines a decision model: the answer never goes through decoding, but is chosen among the admissible options. The phenomenon also recalls two results from the literature: the probability of the first answer token and the answer the model actually writes may not coincide [76], and the choice among letters is subject to selection bias [77]. A specialised, calibrated readout has to absorb both effects. A control experiment quantifies its difference from generation: it keeps the same weights and the same forward pass, but instead of the restricted readout it takes the token that generation would emit (Section 7.6).

4.3 Training objective

GroundOne is obtained from Qwen2.5-0.5B-Instruct and Qwen2.5-3B-Instruct [23] through low-rank adaptation (LoRA) [24], described in Section 3.5 (Figure 3.3).

The adapters have rank 16, scaling factor 2 and dropout 0.05[2]. They are applied to all the linear layers of all the transformer blocks, that is, to the seven projections of each block (Figure 4.4), because covering every layer matters more than using a high rank [65]. In total, 8.8 million parameters are trained in the 0.5B and 29.9 million in the 3B, about 1.8 % and 1.0 % of the respective models. The weights of the base model remain frozen at 8-bit precision. Each training example is an answer with all its scored sentences, and the loss is computed at each of its markers.

The loss is a cross-entropy[3] computed over the three letters only. The prompt is never supervised, so the training signal comes entirely from the verdicts. About 88 % of the sentences in the training set are supported, so the classes are weighted so that each contributes equally to the loss. With the distribution of the training set this gives weights of about 0.38 for supported sentences, 7.5 for contradicted ones and 4.4 for not-mentioned ones. Without these weights, a model can reach a low loss by answering "supported" almost always. A useful consequence follows: a model that had learnt only the class frequencies would remain at a loss of $\ln 3 \approx 1.10$, so a loss below this value indicates that the model actually exploits the text.

The optimiser is AdamW, used without weight decay, that is, without the penalty that shrinks the magnitude of the weights at every step. As in the default configuration of MLX, the moment estimates receive no bias correction, and no gradient clipping is applied. Training lasts one epoch, that is, a single pass over the answers of the training set. The learning rate rises to 10^{-4} with a linear warm-up over the first 3 % of the steps and then decreases to zero with a cosine schedule[4]. Answers of similar length are grouped into micro-batches, each within a token budget that also includes padding, that is, the filler tokens that equalise the lengths. The gradients of several micro-batches are then accumulated, so that each optimiser step sees about 16 answers, or about 110 decisions. In this way every step contains some contradicted sentences, despite the rarity of the class. Gradient checkpointing, which recomputes some activations during the gradient computation instead of storing them, keeps memory usage within the 16 GB of the laptop. During training the model is evaluated periodically on the dev set, and the checkpoint, that is, the copy of the weights, with the highest three-class balanced accuracy, averaged over

[2]The *scaling factor* multiplies the update computed by the adapter before it is added to the output of the frozen layer. *Dropout* randomly zeroes, during training, a fraction of the adapter's inputs (here 5 %), so as to reduce the risk that the model memorises the training set.

[3]The *cross-entropy* of a prediction is the negative logarithm of the probability the model assigns to the correct class: it is zero when this probability is one and grows without bound when the model is confidently wrong.

[4]During the *warm-up* the learning rate grows gradually to the chosen value, so as to avoid abrupt updates at the start of training. The *cosine schedule* then reduces it following half a period of a cosine: slowly at first, faster in the middle and slowly again towards the end.

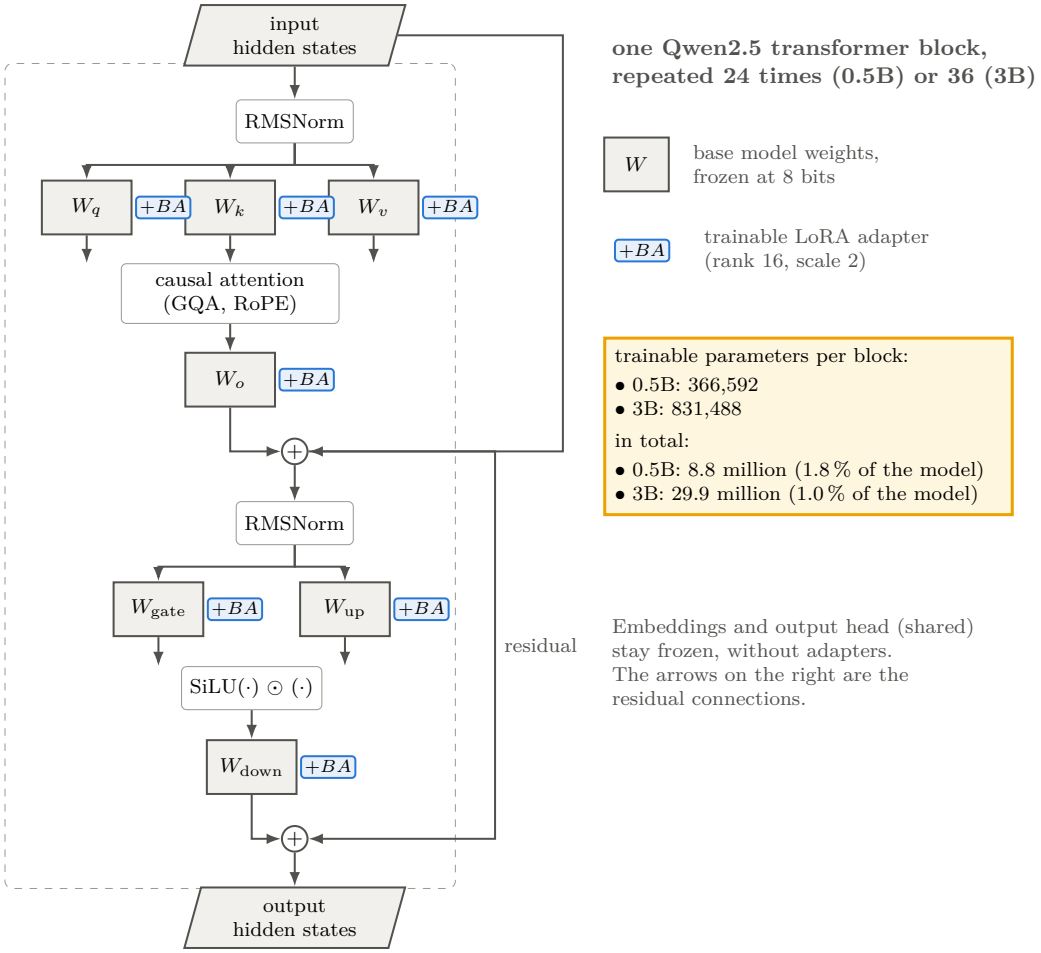


Figure 4.4: Where GroundOne applies LoRA. Each transformer block of Qwen2.5 contains seven linear projections: four in the attention (W_q, W_k, W_v, W_o) and three in the MLP block (W_{gate}, W_{up}, W_{down}). Their weights remain frozen at 8 bits, and each is paired with a rank-16 adapter (in blue) that corrects its output following the scheme of Figure 3.3. The block is repeated 24 times in the 0.5B and 36 in the 3B; the embedding and the output head have no adapter.

the three tasks of RAGTruth (question answering, summarisation and data-to-text), is kept. Section 6.1 reports the settings of each run.

4.4 Calibration

The class weights train the model as if the three classes were equally frequent, so its raw probabilities do not reflect the real class frequencies. A post-hoc transformation is therefore fitted on the dev set and applied to the logits after training, without changing the weights. The three logits are divided by a temperature T and shifted by a per-class bias, $\mathrm{softmax}(z/T + b)$, with the bias of the supported class fixed at zero. The parameters T and b are chosen by minimising the negative log-likelihood (NLL) of the dev-set labels[5], that is, the mean cross-entropy [90]. The bias plays the role of the contextual calibration proposed for language models queried through prompts [92]; unlike that correction, however, here it is fitted on labelled data. Temperature alone changes the probabilities but neither the most probable class nor the ranking of the sentences. The bias, instead, can change the most probable class and, with three classes, can also slightly reorder $P(\text{supported})$ from one sentence to another. For this reason the classification metrics and the AUROC[6] are computed on the raw output. The calibrated probabilities are used instead wherever a probability has to correspond to a frequency: in the calibration metrics, in the thresholds and in selective prediction (Section 7.7).

Calibration during or after training. GroundOne calibrates its probabilities after training. Jev, the decision model that introduced the category, takes the opposite route: according to the vendor, its probabilities become reliable already during training, thanks to a reinforcement learning phase with rewards designed to reward calibration (*Reinforcement Learning with Calibrated Decisions*, RLCD) [14]. The recipe is not public, and a direct comparison was not possible: Jev is a remote,

[5]The *negative log-likelihood* of a set of n predictions is the mean, with the sign changed, of the logarithm of the probability assigned to the correct class: $\mathrm{NLL} = -\frac{1}{n}\sum_{i=1}^{n} \log p_i(y_i)$, where y_i is the correct class of prediction i. It is zero when the model always assigns probability one to the right class and grows rapidly when it assigns a low probability to the right class, that is, when it is confidently wrong: this is why it penalises both errors and overconfidence.

[6]The *AUROC* (area under the ROC curve) measures how well a score separates two classes, here supported from unsupported sentences. Given a threshold, the sentences with a higher score are accepted as supported: the share of supported sentences accepted is the true positive rate, the share of unsupported sentences accepted by mistake is the false positive rate. The ROC curve plots these two quantities for all possible thresholds, and the AUROC is the area under it. It equals the probability that a randomly chosen supported sentence receives a higher score than a randomly chosen unsupported sentence: it is 0.5 for a random score and 1 for perfect separation.

closed service, and evaluating it on the test sets of this thesis would have required sending the data to an external service. The comparison therefore concerns the approaches, on the basis of the independent evaluations (Table 4.1).

These evaluations show that calibration obtained during training holds in aggregate, but not on the individual workload. Over 31 benchmarks the ECE of Jev is 0.047 when computed on all the data together, but 0.168 as the median of the individual benchmarks [74]. Its binary probabilities rank the examples well, but they are slightly underconfident, so the fixed threshold of 0.5 ends up poorly placed: the mean probability of an affirmative answer is 0.465, against an observed frequency of 0.518 [20]. For this reason the studies recommend fitting thresholds and temperatures on the target workload [75]. Even the openly released RLCD recipe offers no measurable advantage: in a controlled comparison it remains 2.5 to 3 points below plain cross-entropy, with a variability across seeds up to four times larger, and after temperature scaling no training objective turns out to be better calibrated than the others [100].

GroundOne starts from here. A specialist works on a single domain, so it can fit its own calibration on exactly the distribution in which it will be used, with a dev set and a few parameters. On T1 the binary ECE falls from 0.30 to 0.019 for the 0.5B and from 0.18 to 0.019 for the 3B. The price is the same one that the studies find for Jev, seen from the other side: the calibration fitted on RAGTruth does not transfer to VitaminC (Section 7.7). The advantage is that the procedure is open, reproducible and bound to the model that produced it, and that the decision threshold is part of the calibration instead of being left to the user.

Calibration is measured mainly with the Expected Calibration Error (ECE) [91, 90], which measures how far, on average, the confidence stated by a model departs from the frequency with which its predictions turn out to be correct. The n predictions are divided, according to their confidence, into M intervals of equal width, with $M = 15$ in this thesis. For each interval B_m the mean confidence of its predictions, $\mathrm{conf}(B_m)$, is compared with the fraction of them that turned out to be correct, $\mathrm{acc}(B_m)$. The ECE is the mean of the gaps, weighted by the size of the intervals:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \Big|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\Big|. \tag{4.1}$$

A perfectly calibrated model has an ECE of zero; an ECE of 0.05, for example, indicates that the stated confidence departs on average by five percentage points from the observed frequency. In the binary variant the confidence is $P(\text{supported})$ and the observed frequency is the share of supported sentences in the interval. In the three-class top-label variant, instead, the confidence is the maximum probability and the observed frequency is the share of correct predictions. The ECE does not compare the distribution of the predicted classes with that of the test set. Rather, it checks whether, among the sentences to which the model assigns a given

Table 4.1: Two approaches to the calibration of a decision model. The values for Jev come from independent evaluations on benchmarks different from those of this thesis; those for GroundOne are measured on the T1 test set.

	Jev	GroundOne
When	During training: SFT followed by RL with calibration rewards (RLCD).	After training: temperature and per-class bias fitted on the dev set.
Scope	A single model for all tasks.	A single domain, the one in which the model is used.
Recipe	Not public (vendor's statement).	Open and bound to the model fingerprint.
Threshold	Left to the user; it must be fitted for each workload.	Fitted on the dev set and part of the output rule.
Evidence	ECE of 0.047 in aggregate and 0.168 per individual benchmark; underconfident binary probabilities.	Binary ECE on T1 from 0.30 to 0.019 (0.5B) and from 0.18 to 0.019 (3B); does not transfer to VitaminC.

confidence, that confidence matches reality. The measure has two limits, however: it depends on the choice of intervals, and a constant predictor that always returns the base rate obtains an ECE close to zero while discriminating nothing. For these reasons the ECE is always reported together with the negative log-likelihood and reliability diagrams, the plots that show, for each interval, the observed frequency as a function of the mean confidence. The discriminative ability of the probabilities is measured separately instead, with two measures. The first is the AUROC, described below, which measures how well the probability of support separates supported from unsupported sentences. The second is the AURC (area under the risk–coverage curve), that is, the area under the risk–coverage curve of Section 3.4 (Figure 3.2): it measures the mean error rate obtained by accepting the decisions from the most to the least confident, and it is smaller the more the errors are concentrated among the least confident decisions, which are those a system would defer.

Calibration and AUROC answer two different questions, which is why the thesis always reports both. The ECE asks whether a probability of 0.8 really corresponds to a sentence that is supported eight times out of ten. The AUROC asks instead whether supported sentences receive higher scores than unsupported ones, whatever the scale of the scores (Figure 4.5). The two properties are independent. A constant predictor is perfectly calibrated with respect to the base rate, but has an AUROC

of 0.5; conversely, a model can rank the sentences almost perfectly, with an AUROC close to 1, and still assign overly extreme probabilities. The AUROC, moreover, depends neither on any threshold nor on the class frequencies, and it does not change if the scores are transformed monotonically, as temperature does. For this reason it is the main measure by which GroundOne is compared with verifiers that return a binary score, such as the specialised encoders, while the choice of the threshold and its consistency with the probabilities shown to the user are treated separately.

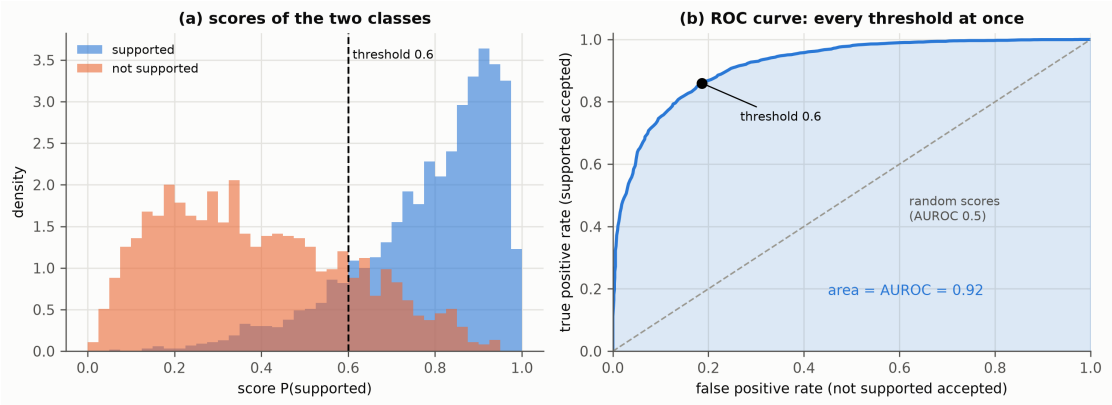


Figure 4.5: How the AUROC arises (a sketch with synthetic scores, not a result). (a) Distribution of $P(\text{supported})$ for supported and unsupported sentences: the two distributions overlap, and every threshold accepts part of each. (b) Moving the threshold from 1 to 0 yields the ROC curve; the black dot corresponds to the threshold of panel (a). The area under the curve is the AUROC: the more separated the two distributions, the closer the curve gets to the top-left corner and the area to 1. The diagonal corresponds to random scores.

From probabilities to returned labels. A user of the verifier sees, for each sentence, a label and a probability, which must be consistent with each other. This thesis evaluates the raw decision, that is, the most probable class according to the uncalibrated output. The raw decision, however, can contradict the calibrated probability shown next to it: a sentence can be labelled "not mentioned" while having a calibrated $P(\text{supported})$ slightly above one half. The returned output therefore derives every element from the calibrated probabilities with a single rule. A sentence is labelled supported if and only if $P(\text{supported}) \geq \tau$; otherwise it receives the more probable of the two remaining classes. The strict policy flags every unsupported sentence, the lenient one only contradicted sentences. The threshold τ is fitted on the dev set so as to maximise the three-class balanced accuracy. The user can change it in every request, thereby choosing a different

operating point without retraining the model. Figure 4.6 summarises the path from the raw logits to the returned labels.

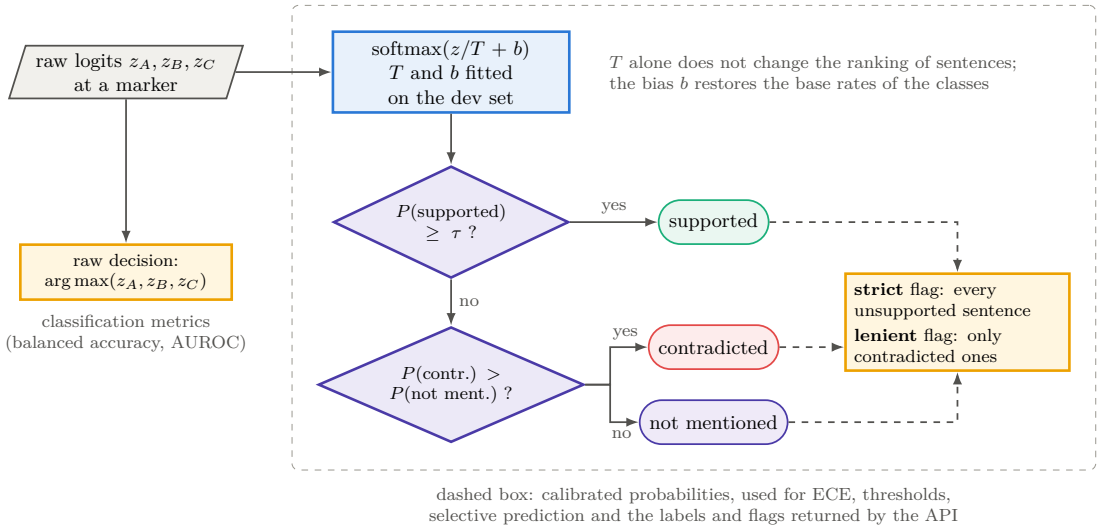


Figure 4.6: From logits to returned labels. The raw decision, that is, the argmax of the logits, serves the classification metrics of Chapter 7. The calibrated probabilities, obtained with temperature and bias fitted on the dev set, feed instead the single rule of the returned output: supported if $P(\text{supported}) \geq \tau$, otherwise the more probable of contradicted and not mentioned. The strict and lenient flags derive from this.

4.5 Software

GroundOne is implemented as an installable Python package based on MLX [3, 58], the array framework designed for Apple silicon. All the training and inference phases described so far run with MLX on the same laptop. The package comprises the dataset builder with its checks, the training loop, calibration, evaluation on the frozen test sets, the baselines and a memory-protection mechanism. The latter runs long processes one at a time and stops a process before the system runs out of memory.

The verifier is exposed through a command-line interface and an HTTP server. The server offers a native endpoint and an endpoint compatible with the System One interface of Jev, already used by other decision models, so that existing clients can adopt GroundOne by changing only the address. The native endpoint receives a document and an answer. It segments the answer with the same rule used to build the data and runs the single forward pass of Section 4.2. For each sentence

it reports whether it was scored, the three calibrated probabilities, the returned label and the flags of the strict and lenient policies, together with the decision rule and the threshold that produced them. It also reports the number of input tokens, the number of generated tokens, always zero, and the time taken. The compatible endpoint, instead, answers generic closed questions about a shared context, with the one-question-per-sentence format. All the model computation takes place on a single long-lived worker thread, because the Metal context of the GPU, that is, the state through which the program uses it through Metal, Apple's graphics API, must not move from one thread to another. Finally, nothing leaves the machine: models, data and requests stay local. Figure 4.7 summarises the architecture of the package.

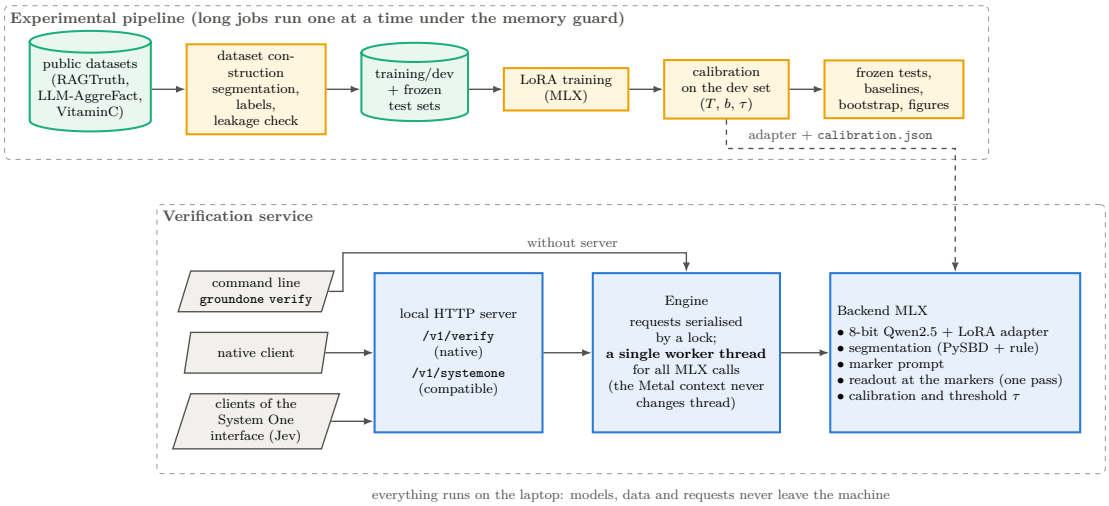


Figure 4.7: Architecture of the software. At the top, the experimental pipeline, from the public datasets to the frozen tests, run under the memory guard; at the bottom, the verification service, in which the command line and the HTTP server (native and compatible endpoints) share the same Engine, which serialises the requests and runs every MLX call on a single worker thread. The adapter and the calibration file produced by the pipeline are the only artefacts the service reads.

Chapter 5

Data

GroundOne learns a single decision, so the data must teach exactly that decision and allow it to be measured reliably. The chapter first defines the decision and its labels. It then describes the three public datasets from which the labels come and the way they are turned into the training set, the dev set and the test sets. The last sections show how the reliability of the test sets was verified without new annotations, what an exploratory analysis of the built dataset reveals and how leakage between training and test data was ruled out.

5.1 Task and labels

The input consists of a retrieved document and an answer, written by a language model and split into sentences. For every sentence the model decides whether, *with respect to the document*, it is *supported*, *contradicted* or *not mentioned*. A sentence is supported when the document states, or directly implies, everything it asserts. It is contradicted when some part of it conflicts with the document. It is not mentioned when it adds information that the document does not contain but does not contradict either. When a sentence contains both a conflicting part and an unverifiable part, it is labelled as contradicted, because a contradiction is the more serious error. Only the document counts: a statement that is true in the real world but absent from the document is still not mentioned.

The number of decisions required by an answer equals the number of its scored sentences, denoted K. This quantity is central to the thesis. A decision model reads the document and the answer only once, so each additional decision costs it little, only the sentence's tokens; an LLM judge, by contrast, has to produce text for every sentence. For this reason the comparison with LLM judges is carried out as a function of K, and the data must include long answers as well as short ones.

For every sentence the model returns three probabilities, and every other quantity

is derived from them instead of being learned separately. The binary groundedness score, used for the comparison with published binary benchmarks, is $P(\text{supported})$. Flags, instead, depend on the notion of error adopted. Under a *strict* policy, anything the document does not state is an error, and a sentence is flagged when $P(\text{supported})$ falls below a threshold τ. Under a *lenient* policy, only contradictions are errors, and a sentence is flagged only when it is judged contradicted. In this way the user chooses the notion of error and the threshold at inference time, without retraining the model. Section 4.4 gives the exact rule with which the verifier produces the labels it returns.

5.2 Source datasets

The dataset is deliberately built from as few sources as possible: one for training and three for testing (Table 5.1). With fewer sources, the definition of the labels and the text format remain unique, and there are fewer points at which the construction can introduce errors.

Table 5.1: The source datasets: licence, role in GroundOne and type of the original labels.

Dataset	Licence	Role	Labels
RAGTruth [8]	MIT	Training set, dev set and test T1	Character spans of LLM answers, annotated with four hallucination types (conflict or baseless information, evident or subtle).
LLM-AggreFact [12, 89]	CC BY-ND 4.0	Test T2 only	One binary label per claim: supported or unsupported.
VitaminC [80]	CC BY-SA 3.0	Test T3 only	Three classes per claim: *supports*, *refutes* and *not enough information*.

RAGTruth. RAGTruth [8] contains 17,790 answers written by six language models (GPT-3.5, GPT-4, Llama-2 7B, 13B and 70B, and Mistral-7B) in three RAG tasks, which differ in the type of context. In question answering the context is a set of passages retrieved for an MS MARCO question [101]. In summarisation it is a news article, taken from CNN/DailyMail [102, 103] or from recent news. In data-to-text generation, where the model describes structured data in prose, the context is the record of a Yelp business in JSON format [104]. In each answer, two

trained annotators identified the character spans not supported by the context[1], with a third review in cases of disagreement. They then assigned each span one of four types: *conflict*, evident or subtle, and *baseless information*, evident or subtle. The paper reports an inter-annotator agreement of 91.8 % at the answer level and 78.8 % at the span level. RAGTruth is the only public dataset that combines natural multi-sentence answers, produced by different models, with the distinction between contradicted and unsupported content, and for this reason it is the only training source. The official split separates 450 test sources from the remaining ones, which are intended for training.

LLM-AggreFact. LLM-AggreFact [12, 89] aggregates eleven manually annotated fact-checking datasets, each made of (document, claim) pairs. The datasets cover three areas: summarisation (AggreFact-CNN and XSum [105], TofuEval [106], with its two subsets MediaS and MeetB), retrieval-based or long-form question answering (RAGTruth [8], ClaimVerify [107], ExpertQA [108], LFQA [109]) and fact checking proper (WiCE [110], REVEAL [111], FactCheck-GPT [112]). The benchmark has a public leaderboard, which makes the results comparable with those of published systems. Its licence, however, forbids using the data to train models [113]. In this thesis the dataset therefore serves only for evaluation: no training, calibration or selection takes place on any of its splits.

VitaminC. VitaminC [80, 114] is a fact-verification dataset built from Wikipedia revisions that change a fact. For each revision, trained annotators wrote claims supported by one version of the sentence and refuted by the other. When no contradiction could be formulated, the pair was labelled as having not enough information. The two versions of the evidence are almost identical, so to decide, a model must look at the precise fact rather than at word overlap. On a re-annotated sample of 2,000 pairs the authors report a Fleiss' κ of 0.71[2]. In this thesis VitaminC fills a precise gap. RAGTruth does not publish the inter-annotator agreement on the hallucination *type*, so on its own it cannot show that the model learns the boundary between contradicted and not mentioned. VitaminC provides exactly this boundary, with labels of known reliability and in a different domain.

Several other candidates were examined and discarded. NLI corpora such as ANLI [79] have short premises, far from typical RAG contexts, and the same

[1] A *span* is a contiguous stretch of text, identified by the positions of its first and last characters. A span-level annotation marks the exact portion of the answer that contains the error, instead of judging the answer as a whole.

[2] Fleiss' κ measures the agreement among several annotators net of the agreement that would be obtained by pure chance: it equals 1 when agreement is perfect and 0 when it does not exceed chance.

three-class distinction on factual text is already covered by VitaminC. Synthetic or binary-only training data, such as the data released with MiniCheck [12], lack three-class labels. Finally, datasets with labels generated by a language model and datasets that reuse RAGTruth test answers were excluded: in this way the labels remain human and the test sets remain independent.

5.3 Construction

Sentence segmentation. Answers often contain lists and headings, so they are first split at line breaks. Each line is then divided into sentences with PySBD [115], a rule-based segmenter that handles abbreviations and decimal numbers. Leading list markers, such as "3." or "*", are removed, because they concern layout and not content. The same segmenter is used in training, evaluation and inference, so that the model always sees sentences cut in the same way. Figure 5.1 summarises the whole construction process, from the sources to the frozen test sets.

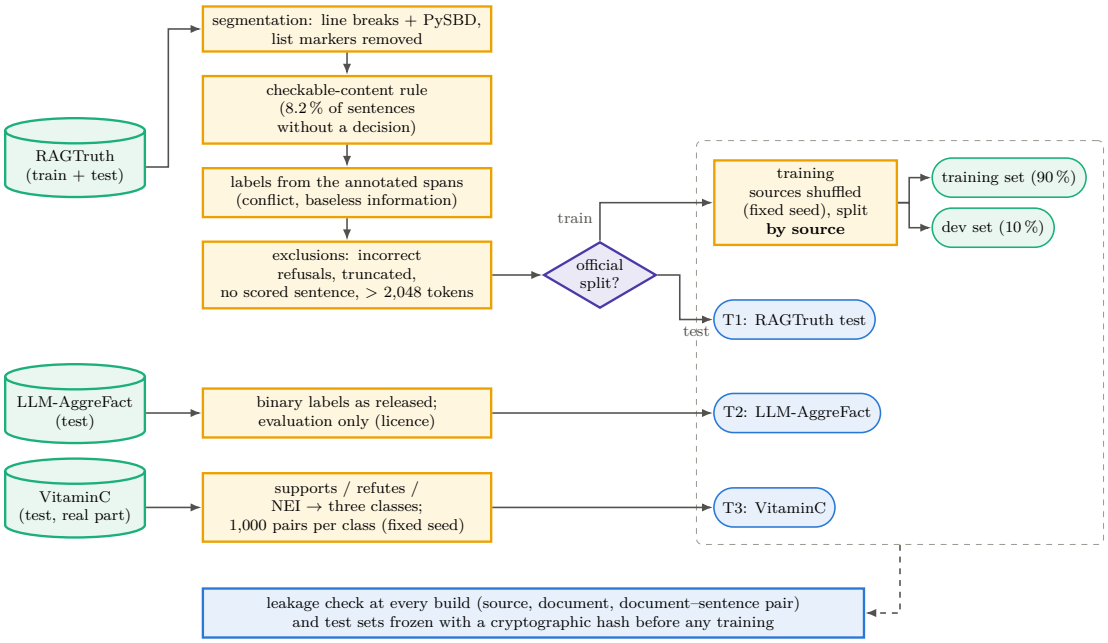


Figure 5.1: Construction of the dataset. RAGTruth answers are segmented, filtered of sentences without checkable content, labelled from the spans and cleaned of exclusions; the official test split becomes T1, while the training sources are divided by source into training set and dev set. LLM-AggreFact and VitaminC provide only test sets (T2 and T3). An automatic leakage check accompanies every build, and the test sets are frozen with a hash before any training.

Sentences without checkable content. Some sentences assert nothing about the document: greetings, headings that introduce a list, lines containing only a citation, such as "(Passage 2)", and refusal formulas. These sentences remain in the numbered answer as context, but they are not decisions and carry no label. A short rule identifies them, and it applies to 8.2 % of all RAGTruth sentences, mostly list headings. The rule considers only formatting, that is, list markers, citation-only lines and conversational formulas, and it never changes a label. Its coverage was verified against the independent segmentation of LLM-AggreFact, described in Section 5.5.

Labels. In RAGTruth the label of a scored sentence depends on the spans it overlaps. The sentence is contradicted if it overlaps a conflict span, not mentioned if it overlaps only baseless-information spans, and supported in all other cases. Only 27 of the 14,226 annotated spans (0.19 %) touch no scored sentence, so the conversion loses almost no annotation. For LLM-AggreFact, instead, the binary label is kept as released. VitaminC labels are converted directly: *supports* becomes supported, *refutes* contradicted and *not enough information* not mentioned. Figure 5.2 summarises the rule applied to RAGTruth.

Exclusions and splits. Three categories of answers are removed: those that RAGTruth flags as incorrect refusals or as truncated, those without any scored sentence and those in which document and answer together exceed 2,048 tokens. This last length limit is imposed by the memory of the 16 GB laptop. The official RAGTruth test split becomes test set T1. The training sources, instead, are shuffled with a fixed seed and divided into two parts: 10 % forms the dev set and the rest the training set. Since the division is made by source, answers about the same source never appear in both sets. Six training answers about a news article that also appears in the LLM-AggreFact test set are removed as well. Finally, the real (non-synthetic) part of the VitaminC test split is unbalanced across classes, so T3 is formed by sampling from it, with a fixed seed, 1,000 pairs per class.

5.4 The resulting dataset

Table 5.2 summarises the five files of the dataset. The training set contains 13,203 answers and 90,795 labelled sentences. Each record corresponds to an answer and contains one label for each sentence: in this way the same data can feed different ways of reading the K decisions of an answer.

RAGTruth is strongly imbalanced: about 88 % of the labelled sentences are supported, and contradictions are only 3–4 % (Figure 5.3). This is the natural frequency of errors in model answers, and it is preserved. The test set thus reflects

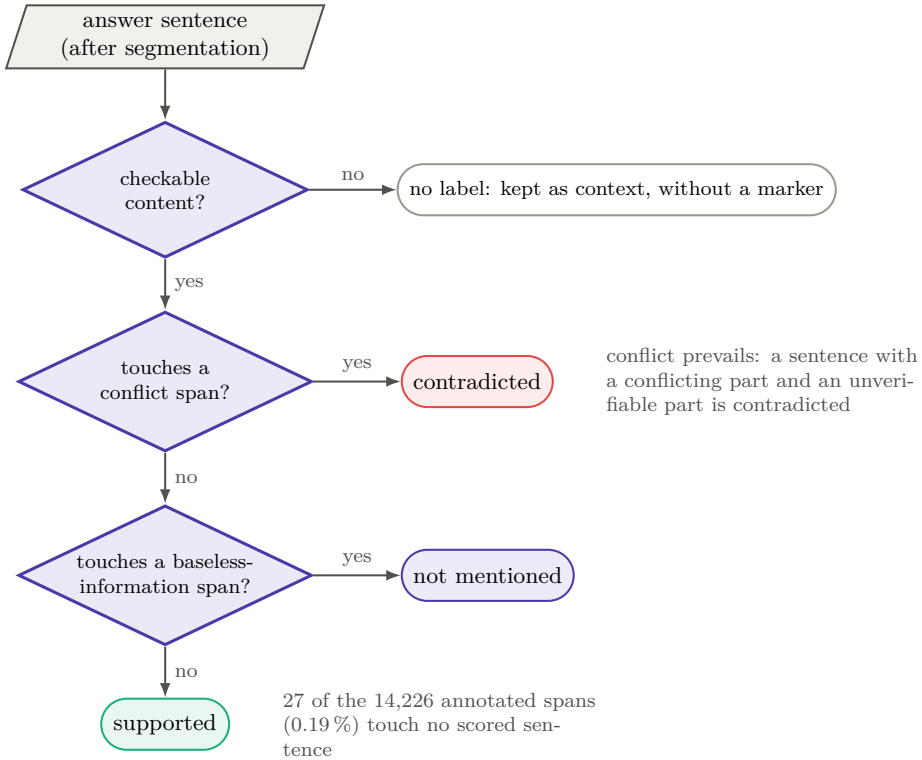


Figure 5.2: Rule by which the span-level annotations of RAGTruth become sentence labels. Sentences without checkable content remain in the prompt as context but are not decisions; among the others, conflict prevails over baseless information.

Table 5.2: The GroundOne dataset: number of records and of decisions by split and by class. A decision is a labelled sentence. T2 has binary labels, so its contradicted and not-mentioned sentences are merged into the unsupported class.

Split	Source	Records	Decisions	Supported	Contradicted	Not mentioned
Train	RAGTruth	13,203	90,795	79,898	4,039	6,858
Dev	RAGTruth	1,468	9,996	8,827	410	759
T1	RAGTruth test	2,647	17,284	15,745	584	955
T2	LLM-AggreFact test	29,320	29,320	22,832	6,488 unsupported	
T3	VitaminC test	3,000	3,000	1,000	1,000	1,000

what a verifier meets in practice, while the imbalance is handled during training. T3 compensates by offering a test balanced across the three classes.

Answers contain on average 6.9 scored sentences in the training set and 6.5 in T1, with a maximum of 28 (Figure 5.4). To analyse the results as a function of length, T1 is divided into four bands of K: 756 answers with 1–4 sentences, 1,288 with 5–8, 497 with 9–13 and 106 with 14 or more. When a results cell contains fewer than 30 sentences of a minority class, as happens for contradicted sentences in question answering, the value is reported but marked as indicative.

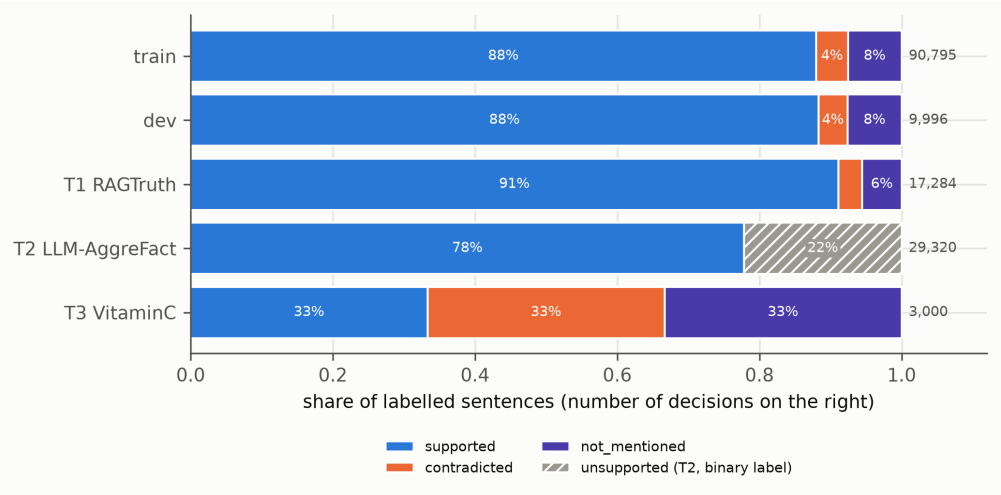


Figure 5.3: Class distribution among the labelled sentences of each split. In the RAGTruth splits (train, dev and T1) about 88–91 % of the sentences are supported. T2 (LLM-AggreFact) has binary labels, so its contradicted and not-mentioned claims form a single unsupported class (hatched). T3 is sampled with 1,000 pairs per class.

5.5 Reliability of the test sets

The test sets were verified without new annotations, using evidence independent of the construction described above (Table 5.3). All test labels come from the human annotators of the published datasets. The two choices that could introduce errors are segmentation and the conversion from spans to sentence labels. Both were compared with LLM-AggreFact, whose authors independently converted the same RAGTruth test answers into sentence-level claims. The segmenter of this thesis recovers exactly 96.3 % of their claims, and only 8.5 % of the scored sentences are absent from their set. These are mostly sentences removed by their filter of “non check-worthy” claims, which is undocumented. On the 15,492 sentences

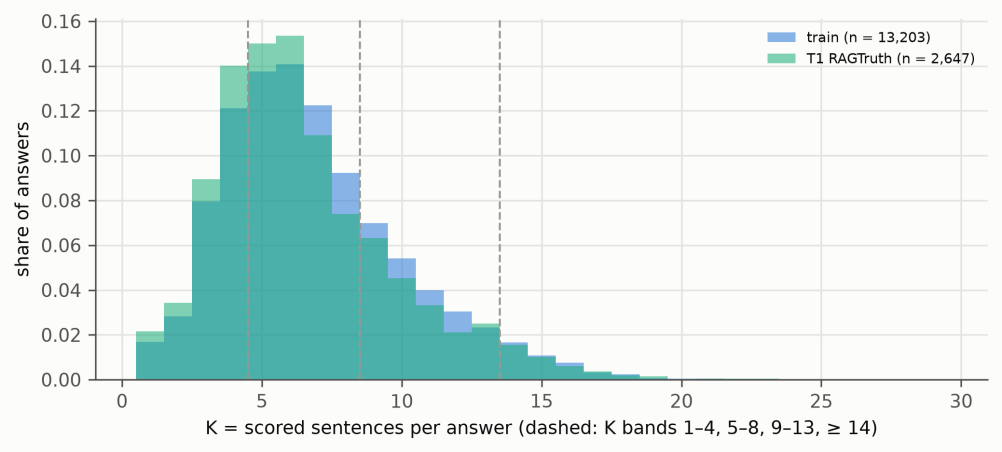


Figure 5.4: Distribution of K, the number of scored sentences per answer, in the training set and in T1. Dashed lines delimit the bands of K used to report the results.

present in both sets, the binary labels agree in 98.75 % of cases. Moreover, all 193 disagreements go in the same direction: the span-based rule marks as unsupported a sentence that LLM-AggreFact considers supported. Finally, an encoder-based baseline, evaluated on T2 without changes, reproduces the score published by its authors (balanced accuracy of 74.0 measured, against 73.5 published), which validates the evaluation code.

Table 5.3: Reliability checks run before the test sets were frozen, with the pass criterion and the outcome of each.

Check	Criterion	Outcome
Label provenance	Every test label comes from human annotators.	Passed
Segmentation	At least 90 % of the LLM-AggreFact claims recovered among the segmented sentences, and at most 10 % of the segmented sentences absent from LLM-AggreFact.	96.3 % and 8.5 %
Label derivation	Agreement of at least 97 % with the binary labels of LLM-AggreFact.	98.75 %
Reference baseline	MiniCheck-RoBERTa on T2 within 1.5 points of the published score.	74.0 against 73.5
Leakage	No source, document or (document, sentence) pair shared between training and test.	Passed

These checks validate the binary side of T1, that is, the distinction between

supported and unsupported sentences. The distinction between contradicted and not mentioned rests instead on the RAGTruth annotators, whose agreement on the hallucination type is not published. In addition, a recent re-annotation of the RAGTruth test split found 1.68 times more hallucinated answers than the original labels [116], which suggests that T1 underestimates the two minority classes. For these reasons the three-class results on T1 are always read together with those on T3.

Once these checks were complete, and before any model was trained, the three test files were frozen with cryptographic hashes[3]. From that moment on, the program that generates the dataset from the raw sources (the build command of the GroundOne package, Section 4.5) compares every new test file with its fingerprint and refuses to overwrite it if the content has changed. Moreover, rerunning the build from scratch yields exactly the frozen files: the test sets therefore cannot be modified, not even by mistake, after the results have been seen.

5.6 Exploratory analysis

An exploratory analysis of the sources and of the built dataset brought to light four properties that affect how the results are read.

Errors depend on the task and on the model. Data-to-text answers contain hallucinations two to four times as often as those of the other two tasks. The generator matters too: the answers of Llama-2 and Mistral account for most of the hallucinations, while those of GPT-3.5 and GPT-4 are almost always faithful to the document (Figure 5.5). For this reason the results are reported task by task: a single average would mostly reflect data-to-text.

The test answers are cleaner than the training answers. In every task and for every generator, the share of contradicted and not-mentioned sentences is lower in T1 than in the training set. In question answering, for example, 0.6 % of the T1 sentences are contradicted, against 1.6 % of those in the training set. As a consequence, the metrics that depend on the base rate of the classes, such as precision or F1 on the minority classes[4], are not comparable between dev set and

[3] A cryptographic *hash* is a fixed-length fingerprint computed from the content of a file. Even a minimal change to the file produces a completely different fingerprint, so comparing fingerprints is enough to detect any change.

[4] The *precision* of a class is the share of the predictions of that class that turn out to be correct, and for this reason it changes with the frequency of the class even when the model stays the same. F1 is the harmonic mean of precision and recall.

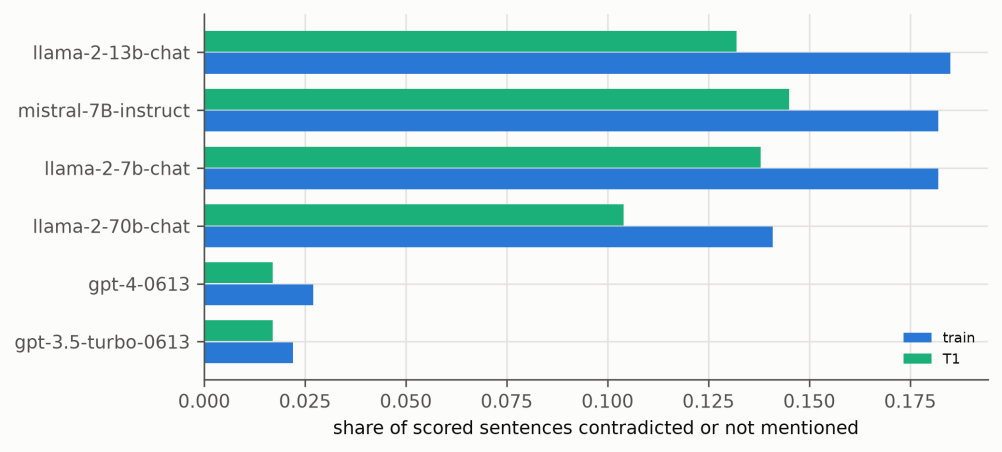


Figure 5.5: Share of contradicted or not-mentioned sentences by the model that wrote the answer, in the training set and in T1.

test set, because the dev set, drawn from the training set, preserves its distribution. The primary metrics are therefore balanced accuracy, per-class recall and AUROC, which do not depend on the base rate.

Contradictions are mostly evident. About 85 % of the contradicted sentences of the training set come from spans annotated as *evident* conflicts, for example wrong numbers or names, while only 147 come from *subtle* conflicts. The contradicted class therefore consists mainly of blatant contradictions, and subtle contradictions cannot be evaluated separately.

Surface cues exist, and they define the baselines. Two simple cues predict the labels without verifying the sentence against the document. The first is word overlap, that is, the share of the sentence's content words that also appear in the document; content words are those that remain after excluding function words, such as articles, prepositions and auxiliary verbs. In all three test sets this share is higher for supported sentences than for contradicted and not-mentioned ones (Figure 5.6). Used on its own as a score, it separates supported from unsupported sentences with an AUROC of 0.74 on T1 and 0.70 on T3. The second cue is the wording of the sentence. In data-to-text, a classifier that sees only the sentence reaches a three-class balanced accuracy of 0.66, because in Yelp descriptions some details are often invented, such as the number of reviews or the availability of Wi-Fi or valet parking. In question answering and summarisation, instead, the same classifier stays at chance level (0.36 and 0.35), and the same happens on T3 (0.33), where the contrastive construction of the dataset removes the cue. A

verifier is useful only to the extent that it beats these cues, so both are reported as baselines, task by task.

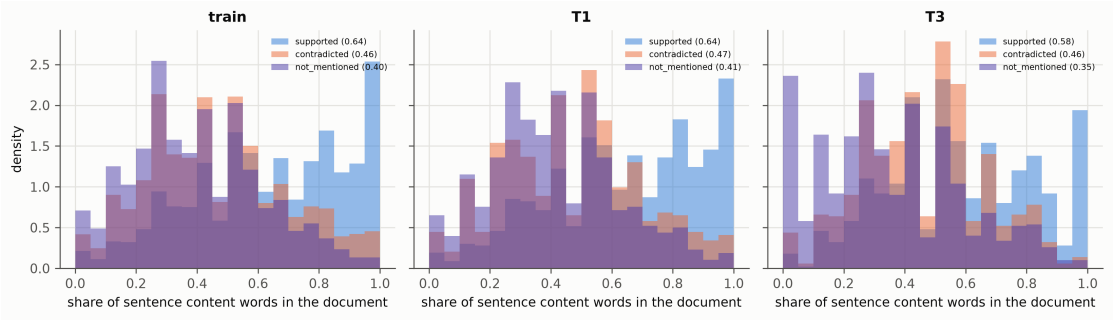


Figure 5.6: Word overlap between a sentence and its document, that is, the share of the sentence's content words present in the document, by class, in the training set, in T1 and in T3. The mean is given in brackets.

Finally, unsupported sentences are not concentrated at the end of the answer. The first sentence is rarely unsupported, while the share of unsupported sentences peaks between the third and the fifth position and stays between 8 and 11 % in later positions (Figure 5.7). Long answers, too, therefore contain errors towards the end, and accuracy at high values of K remains relevant.

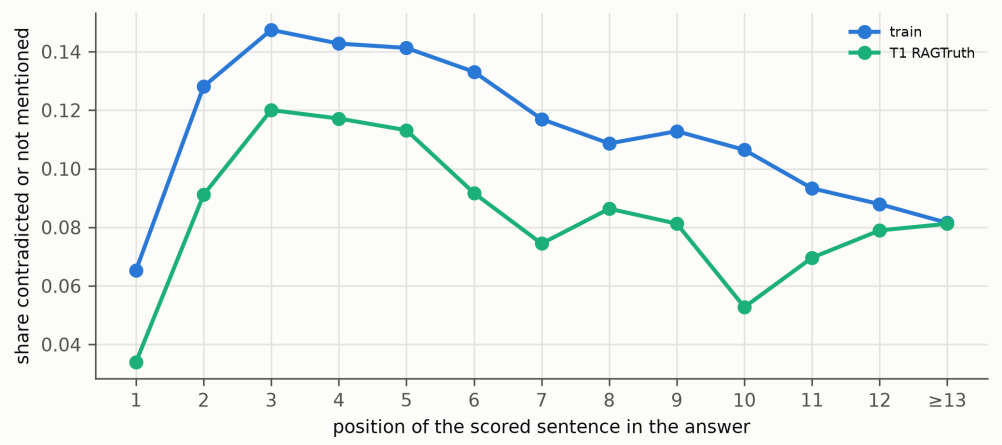


Figure 5.7: Share of contradicted or not-mentioned sentences by the position of the sentence in the answer.

5.7 Leakage control

Leakage, that is, the presence of test-set information among the training data, would make all the metrics of the following chapters optimistic. In this dataset the risk is concrete: RAGTruth contains, for every document, several answers written by different models. If these answers ended up in different splits, the model would see in training the very document on which it is later evaluated. For this reason the splits are defined by source and never by sentence, so that all answers about the same source fall into the same split. In addition, an automatic check runs at every build of the dataset, and it fails in two cases: when a source, a document or a (document, sentence) pair of the training set or of the dev set appears in any of the test sets, and when the dev set and the training set share a source. On the final dataset the check found no overlap of this kind. Without blocking the build, the check also lists the sentences that appear with the same words in both training and test data, but refer to different documents. There are 788 of them, and they are all standard data-to-text sentences, such as "The business has a rating of 4.0 stars.": the generator model repeats them for different businesses, so their presence in the training data does not anticipate any test label, which always depends on the document the sentence refers to. One aspect remains that cannot be verified: whether the public datasets used here were part of the pre-training data of Qwen2.5. This is a limitation common to most work on public benchmarks.

Chapter 6

Experimental Setup

This chapter presents the models that were trained, the systems they are compared with, the metrics, the statistical protocol and the way time and energy are measured. All experiments run on a single machine: a MacBook Pro with an Apple M4 chip and 16 GB of unified memory, with a 10-core CPU, a 10-core GPU and a memory bandwidth of 120 GB/s [45]. Every result therefore reflects what a verifier can do on the same device on which a local assistant runs. All the baselines, including the encoders and the two general-purpose decision models, also run on this laptop, and no result depends on a remote service.

6.1 Models

Both GroundOne models start from the instruction-tuned version of Qwen2.5 [23], converted to MLX with 8-bit weights, and both are trained with the procedure described in Section 4.3. The two models differ in only three respects: their size, the number of training answers and the composition of the micro-batches, which depends on the available memory.

The 0.5B model is trained on the whole training set of 13,203 answers. One epoch of 802 optimiser steps took 5.5 hours of wall-clock time[1] and used at most 2.6 GB of memory; every 150 steps the model was evaluated on the whole dev set. The 3B model, by contrast, is about six times slower per token. It is therefore trained over one night on a random subset of 6,500 answers, corresponding to about 45,000 decisions: 403 steps in 12.25 hours, with a peak of 5.9 GB. A complete evaluation on the dev set would take about an hour, so the 3B is evaluated every

[1] *Wall-clock* time is the real time elapsed from the start to the end of an operation, as a clock would measure it, waiting included. It differs from CPU time, which counts only the time in which the processor is actually working.

80 steps on a fixed subset of 200 answers. In both cases the checkpoint with the highest three-way balanced accuracy on the dev set is kept and then calibrated as described in Section 4.4; for the 3B this is step 320.

The seed determines the initialisation of the adapters and the order of the batches, and the outcome of fine-tuning is known to vary with both factors [97]. To estimate how much the results depend on the seed, the 0.5B procedure is therefore repeated with a second seed. This second run was completed and is evaluated on all three test sets exactly like the first. A further run with a third seed, instead, collapsed early to a constant prediction and was stopped at about 60 % of the epoch. Being incomplete, it is excluded from the tables, and it is discussed in Section 7.2.

6.2 Baselines

The baselines serve to isolate the origin of any advantage of GroundOne, and each of them answers a specific question. Figure 6.1 summarises what each baseline changes with respect to GroundOne.

Cues that never read the document as a whole. Two elementary predictors set the minimum level that any verifier must clear. The first scores a sentence by the share of its content words that also occur in the document. The second is a bag-of-words classifier[2] trained on the training sentences alone, without the document. Both are described in Section 5.6.

Untrained readout (B1). The same Qwen2.5 models, without fine-tuning, are asked the three-way question for each sentence, and the logits of the answer letters are read. An untrained model has no reason to interpret the `Verdict:` marker as a question. This baseline therefore adopts the format with one explicit question per sentence (Section 4.2). In this way it measures what the decision-model mechanism offers before any specialisation.

LLM judges (B0a, B0b). The untrained models are also used in the way that is most common in practice, that is, as LLM judges. They receive the document, the numbered answer and the list of sentences to check, and they write a verdict for each sentence with greedy decoding[3]. The *terse* LLM judge produces one line

[2]A *bag-of-words* classifier represents a text by the words, or the short sequences of words, that it contains and by their frequencies, ignoring their order.

[3]In *greedy decoding* the model chooses the most probable token at every step, without sampling: for the same input, the output is therefore always the same.

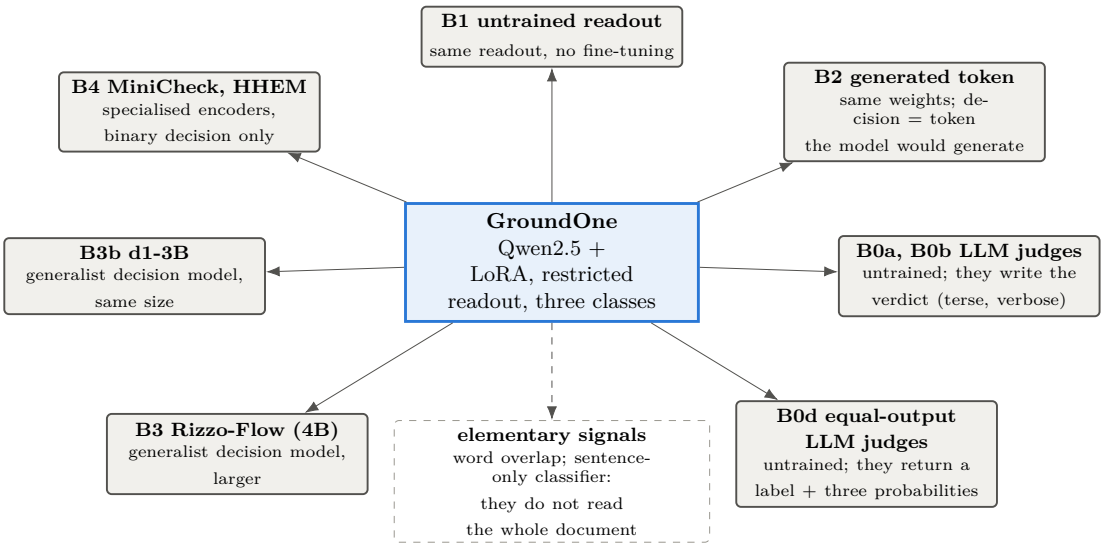


Figure 6.1: Map of the baselines. Each one differs from GroundOne in one specific respect and therefore answers a different question: how much training matters (B1), how much the restricted readout matters compared with the generated token (B2), how much the way the verdict is produced matters (LLM judges B0a, B0b, B0d), how much specialisation matters compared with a general-purpose decision model (B3, B3b), how much the architecture matters (encoders B4) and how far one gets without reading the document (elementary cues).

"`<number>: <letter>`" per sentence. The *verbose* LLM judge, instead, writes a short justification for each sentence followed by the verdict. When a verdict cannot be parsed, a format failure occurs: in the computation of accuracy it is counted as an error, and it is also reported separately.

Equal-output judges (B0d). The terse and verbose LLM judges return less than GroundOne: one letter per sentence and no probability. They therefore allow neither calibration, nor an adjustable threshold, nor the choice of a decision policy. For this reason two further LLM judges are asked for exactly what GroundOne produces, namely a label and three probabilities per sentence. The first writes the label and the probabilities in a JSON object, expressing the probabilities as text: this is what a developer would obtain from the prompt alone. The second is the terse LLM judge, whose three probabilities are read from the logits at the position of the letter it generates. It thus obtains the same content at the cost of the terse LLM judge, and it is the strongest generative opponent. Both are calibrated with

the same transformation used for GroundOne, a temperature and a bias, fitted for each LLM judge on 200 answers of the dev set, so that their probabilities receive the same treatment. The JSON prompt was chosen on twelve answers of the dev set, with attention to format validity alone. The models copied verbatim a numeric example contained in the prompt, which was therefore replaced by placeholders. Parsing is lenient, in the LLM judges' favour: the complete entries of a truncated object are kept, and a missing label is derived from the probabilities. Decoding, finally, is not constrained by a grammar [88], so that the protocol remains that of the other LLM judges. This choice weighs mainly on the format failures of the smallest model.

Readout versus generation (B2). The fine-tuned GroundOne models are run with the same prompt and the same single forward pass, but with a different decision rule. The verdict at each marker is not chosen among the three letters alone, but is the token with the highest probability over the whole vocabulary, that is, the one that greedy generation of a single token would produce. The three letters, with or without a leading space, correspond to the three classes, and any other token is a format failure. This control leaves weights, prompt and cost unchanged and changes only one aspect: whether or not the decision is restricted to the admissible options.

A general-purpose decision model: Rizzo-Flow (B3). The open project Rizzo-Flow [16] implements the decision-model idea with a model of 4B parameters, fine-tuned on generic typed decisions. It runs in its own server, based on llama.cpp [2], and is queried over HTTP. The document and the numbered answer are passed to it as shared state, that is, as a context common to all the questions, together with one three-way choice question for each sentence. The model quantised to 4 bits is used, because the 8-bit one does not fit in memory alongside the operating system.

A general-purpose decision model of the same size: d1-3B (B3b). Liquid AI's d1-3B [18] is an open decision model of 3.1B parameters, post-trained for typed decisions on a wide variety of tasks. It reads the decision from restricted answer letters and shares the state across questions. It therefore belongs to the same readout family as GroundOne and has the size of its larger model, but, unlike GroundOne, it is general-purpose. The comparison with d1-3B thus isolates the value of specialisation with respect to a decision model that was already trained to decide, rather than with respect to an untrained model. The model is run zero-shot with its own inference code, in PyTorch on the GPU of the laptop and without network access. It receives the same state and the same questions as Rizzo-Flow, with at most six questions per call on the same state. On LLM-AggreFact the

documents longer than 1,500 tokens are split, for d1-3B as for GroundOne, into chunks of 1,500 tokens overlapping by 200, and for each sentence the highest probability of support over the chunks is kept. A practical detail matters for reproducibility. On the Apple GPU, PyTorch keeps the compiled kernels, that is, the programs executed by the GPU, for every new input length, and never releases their memory. The evaluation therefore runs in worker processes that are restarted when the memory occupied by the GPU driver exceeds 11 GB; the restarts have no effect on the outputs.

Specialised encoders (B4). MiniCheck-RoBERTa [12] and HHEM-2.1-open [13] are the strongest small verifiers available today. They score each (document, sentence) pair independently and return a single probability of support. As in MiniCheck, long documents are split into chunks of about 400 words, and for each sentence the highest probability of support over the chunks is kept. The encoders have no three-way output, so they are compared on the binary decision only. They run in PyTorch on the GPU of the laptop.

Published results. For LLM-AggreFact, the scores of larger systems from the public leaderboard, such as Qwen2.5-7B used zero-shot, are also reported. These scores are taken as published, without re-running the systems.

The slower systems are evaluated on subsamples of the test sets, mostly stratified[4]. Rizzo-Flow is evaluated on 300 answers of T1, 600 pairs of T3 and 1,100 items of T2, of which it scored 1,093, because seven exceeded its context. The d1-3B model is evaluated on a random subsample of 4,000 items of T2, the untrained 3B on 600 answers of T1 and the LLM judges on the 200 answers of the latency protocol described below. GroundOne and MiniCheck-RoBERTa are instead evaluated on the complete test sets, as are, on T1 and T3, the untrained 0.5B, HHEM and d1-3B; the untrained 3B is also evaluated on the whole of T3. Every comparison with a subsampled system is made on the items scored by both systems.

6.3 Metrics

The share of hallucinated sentences differs between the dev set and T1 (Section 5.6), so the primary metrics are chosen so as not to depend on the base rate of the classes. For the three-way decision the main metric is the balanced accuracy, that is, the mean recall over the three classes. Alongside it, the recall of each class and

[4]A subsample is *stratified* when it is drawn separately from each subgroup (stratum) of the original set, with a fixed number of elements for each stratum, rather than from the whole set in a purely random way.

the share of predictions assigned to each class are reported; the latter reveals a model that has collapsed onto a single answer. For the binary decision two metrics are reported: the balanced accuracy at a threshold of 0.5, as in the LLM-AggreFact leaderboard, and the AUROC of $P(\text{supported})$, which does not depend on any threshold. Calibration is measured with the ECE defined in Section 4.4, with the Brier score and with the negative log-likelihood. Selective prediction is measured with risk–coverage curves and with the AURC (Section 4.4) [96]. All metrics on T1 are also reported per task and per band of K.

6.4 Statistical protocol

Every difference between two systems is accompanied by a 95 % confidence interval, obtained with a paired bootstrap with $B = 2{,}000$ resamples [117, 118, 119]. The bootstrap recomputes the difference on many samples drawn with replacement from the test set, and the spread of the values obtained measures its uncertainty. The resampling unit, however, is not the sentence but the group used in the leakage check: the source document in RAGTruth, the document in LLM-AggreFact and the Wikipedia page in VitaminC. The sentences of the same answer, and the answers about the same document, are in fact strongly correlated, and resampling them independently would lead to intervals that are too narrow. On LLM-AggreFact the groups are resampled within each of the eleven datasets, because the macro average gives every dataset the same weight. The bootstrap is *paired* because the two systems of a comparison are evaluated on the same resamples. The reported p-value is two-sided: it is twice the share of resamples in which the difference has the opposite sign to the observed one, with a minimum of $1/B$, where B is the number of resamples. Figure 6.2 outlines the procedure.

Two limits of the protocol must be stated in advance. The first is that about sixty comparisons are made without any correction for multiple comparisons[5]. The conclusions therefore rest on intervals that lie well away from zero, and no difference with a p-value close to 0.05 is considered established. The second is that resampling the data does not capture the variability due to training, which the two runs of the 0.5B document separately (Section 7.2).

[5]When many statistical tests are carried out, the probability that some differences turn out significant by pure chance increases. Corrections for multiple comparisons, such as the Bonferroni correction, compensate for this effect by making the threshold of each test stricter.

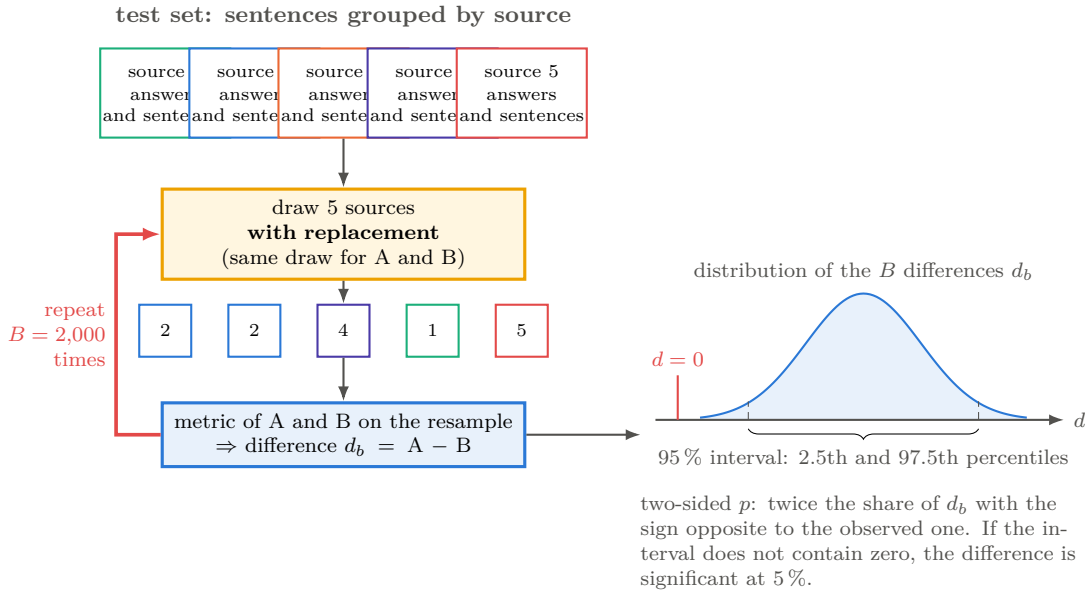


Figure 6.2: Paired cluster bootstrap. The sentences of the test set are grouped by leakage group (source, document or page); at each repetition the groups are drawn with replacement, the metric of the two systems is computed on the same drawn sources and their difference is recorded. After $B = 2{,}000$ repetitions, the 2.5 and 97.5 percentiles of the differences give the 95 % interval, and the share of differences with the opposite sign to the observed one gives the p-value.

6.5 Latency, energy and hardware

Latency is measured on 200 answers of T1, 50 from each band of K, so that long answers are as well represented as short ones. GroundOne, the readout with one question per sentence and all the LLM judges run in process on MLX, that is, inside the same process that invokes them, and they verify one answer at a time, as a verifier integrated in a local assistant would. The first three answers serve as warm-up and are excluded from the measurement. For GroundOne and for the readout with one question per sentence, each answer is timed three times and the median is kept. The generative LLM judges, instead, are timed only once, because greedy decoding is deterministic and each run lasts much longer. The reported time is the wall-clock time of the model call for one answer, from which the time per decision is derived.

Times were also recorded for the encoders, for Rizzo-Flow and for d1-3B, but under conditions that do not allow a fair comparison. The encoders ran in PyTorch and scored each sentence separately, re-reading the document every time, which

penalises them. Rizzo-Flow ran in a different inference engine, with 4-bit weights and behind an HTTP interface. The d1-3B model, finally, ran in PyTorch with its own code and with the worker restarts described above. For these systems, therefore, no conclusion about latency is drawn.

Energy is measured with `powermetrics`, the macOS tool that reports every 200 milliseconds the power drawn by the CPU, the GPU and the Neural Engine, the NPU of Apple processors. For each system, the idle power with the model already loaded is first measured for 60 seconds. Then 100 answers of the latency subset, 25 per band of K, are verified once. The energy of a verification is the mean power during the run, net of the idle power, multiplied by the duration of the run and divided by the number of answers. The measurement covers the computing units of the chip, but not the memory, the display or the storage. It was taken at night, with the machine otherwise idle.

Three measurements concern locality directly. The first checks independence from the network: a complete evaluation of T1 is repeated with the libraries in offline mode and inside a macOS sandbox that denies the process any network access, and its predictions are compared with those of the original run. The second measures the effect of quantisation: the same adapter of the 0.5B model is applied to the original 16-bit (bf16) weights and evaluated on T1 and T3. The third measures co-residence: a single process loads the generator of the demonstration application (Chapter 8), Qwen2.5-3B-Instruct quantised to 4 bits, and a GroundOne verifier; for 30 documents of T1, ten per task, the generator writes an answer of at most 300 tokens and GroundOne verifies it. The peak memory allocated by MLX and the generation and verification times are recorded. The document is given directly, as in RAGTruth, so the retriever is not included. These measurements, too, run entirely inside the network-denying sandbox.

In every experiment only one model runs at a time, and a memory guard watches every long job, recording the free memory and the swap throughout its duration. Results are written once and never overwritten, and each run stores its own configuration. In this way every number in Chapter 7 can be traced back to the run that produced it.

6.6 Licences and terms of use

The independence that the thesis pursues also concerns the legal level: a local verifier is of little use if one of its components can be used only through an external service or under a paid licence. Table 6.1 reports the licence of every model, dataset and software component used in the thesis, verified on the official page of each.

Three consequences follow. The first is that all the components can be downloaded and run locally under licences that permit their use for research, without

Table 6.1: Licences of the components used in the thesis, verified on the official pages on 22 October 2026. The quantised versions in MLX format inherit the licence of the original model.

Component and role	Licence	Relevant conditions
Models		
Qwen2.5-0.5B-Instruct: base of GroundOne 0.5B	Apache 2.0	Free use, including commercial use, with attribution.
Qwen2.5-3B-Instruct: base of GroundOne 3B and generator of the application	Qwen Research License [120]	Research and evaluation only; commercial use requires a licence from Alibaba Cloud.
MiniCheck-RoBERTa-Large (B4)	MIT	Free use.
HHEM-2.1-open (B4)	Apache 2.0	Free use, with attribution.
Rizzo-Flow, Spark-X2.5 4B with LoRA (B3)	Apache 2.0	Free use, with attribution.
d1-3B (B3b)	LFM Open License 1.0 [121]	Commercial use permitted below 10 million dollars of annual revenue.
bge-small-en-v1.5: embedding model of the application	MIT	Free use.
Data		
RAGTruth annotations	MIT	The contexts come from MS MARCO, restricted to non-commercial research [122], from news articles and from the Yelp Open Dataset, intended for educational use [104].
LLM-AggreFact	CC BY-ND 4.0	The access condition allows its use as an evaluation benchmark and forbids its use for training [113].
VitaminC	CC BY-SA 3.0	Sharing under the same licence.
Software		
MLX, mlx-lm, MLX Swift	MIT	Free use.
PySBD, llama.cpp	MIT	Free use.
PyTorch, Transformers, swift-transformers	BSD, Apache 2.0	Free use.

depending on any remote service: the independence of the solution also holds at the level of licences. The second concerns the reuse of the trained models. The adapter of GroundOne 0.5B applies to a base released under the Apache 2.0 licence, whereas that of GroundOne 3B applies to a base with a research licence, which restricts its use to research and evaluation; the same holds for the generator of the application of Chapter 8. A commercial deployment would therefore require the 0.5B model or a base with a permissive licence. The third concerns the training data: the RAGTruth annotations are released under the MIT licence, but the contexts on which they were written come in part from sources intended for non-commercial research or for educational use. For this reason too, the trained adapters should be regarded as research artefacts; a legal assessment of their commercial use is beyond the scope of this thesis. LLM-AggreFact, finally, is used only as a test set, in compliance with its access condition (Section 5.2).

Chapter 7

Results

This chapter reports the experimental results. It starts from how GroundOne learns the task and from how stable its training is. The results on the three frozen test sets follow, together with the comparison with the baselines and the contribution that the decision-model readout makes by itself. The chapter then examines the quality of the probabilities, also in comparison with LLM judges that produce the same output, the time and energy needed to verify an answer, the locality measurements (offline execution, quantisation and co-residence) and, finally, the errors that remain. The intervals, given in parentheses in the form "a to b", are 95 % confidence intervals obtained with a cluster bootstrap, computed as described in Section 6.4. Figure 7.1 gathers in a single plot the paired differences discussed in the chapter.

7.1 Training

Both models learned the task. Before training, a model that receives the marker format does not know what the marker asks of it, and its verdicts are at chance: the 0.5B answers "contradicted" for 88 % of the sentences of the dev set and reaches a three-way balanced accuracy of 0.32. In the completed runs, instead, the training loss fell within the first hundred steps below the value of a model that knows only the class frequencies ($\ln 3 \approx 1.10$), and the score on the dev set then rose steadily (Figure 7.2).

The 0.5B was trained on all 13,203 answers and reached its best score at step 600 of 802, with a balanced accuracy averaged over the three tasks of 0.633; from that point on the score levelled off. The 3B, trained for 12.25 hours on 6,500 answers, reached 0.705 at step 320 of 403, measured on its own subset of the dev set. On the whole dev set the three-way balanced accuracy is 0.731 for the 0.5B and 0.784 for the 3B. In both runs, moreover, all three classes remained present among the

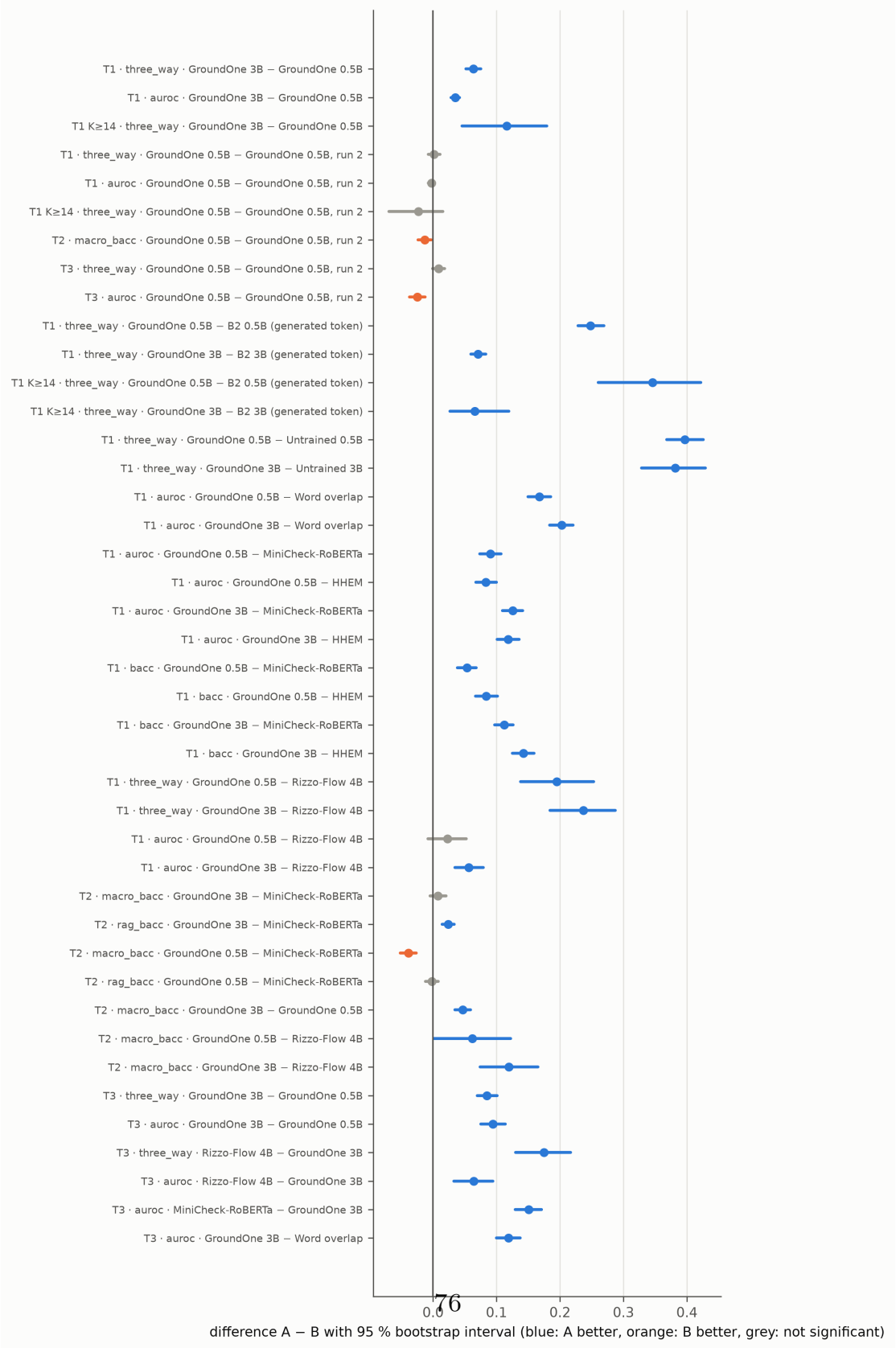


Figure 7.1: All paired differences between systems on the frozen test sets, with the 95 % cluster-bootstrap interval. In blue the differences in favour of the first system (A), in orange those in favour of the second (B), in grey the non-significant ones, whose interval contains zero.

predictions throughout training.

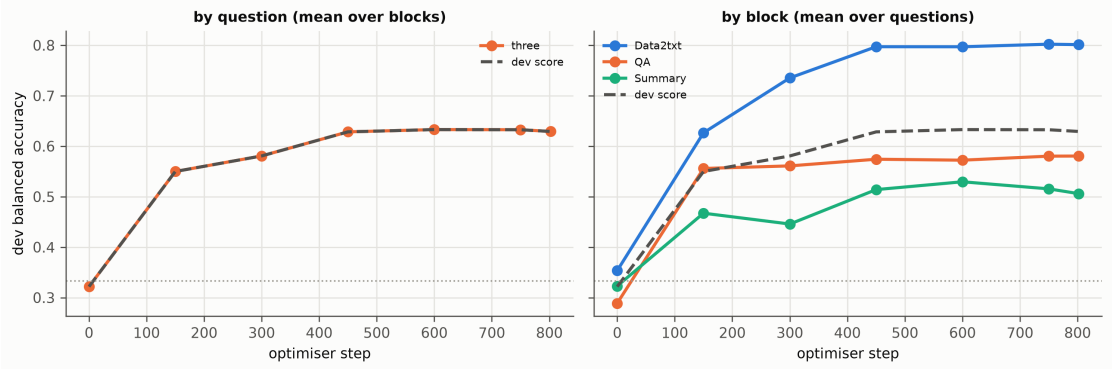


Figure 7.2: Three-way balanced accuracy on the dev set during training of the 0.5B model: overall value (left) and value per task (right). The dotted line marks chance level. The 3B shows the same trend on its own subset of 200 answers of the dev set.

7.2 Variability across seeds

To understand how much the results depend on the seed, the 0.5B procedure was repeated with a second seed, which changes the initialisation of the adapters and the order of the batches. When training converges, it converges to the same point (Figure 7.3). On T1 the second run reaches a three-way balanced accuracy of 0.767, against 0.769 for the first, and an AUROC of 0.907 against 0.905. On LLM-AggreFact the macro-averaged balanced accuracy is 71.5 against 70.1, and on VitaminC the three-way balanced accuracy is 0.434 against 0.442. On T1 the paired difference between the two runs, computed as first minus second, is +0.001 (−0.008 to +0.012) and is not significant.

The differences emerge in the details. First, the errors are distributed differently across the classes: the recall of supported sentences and that of not-mentioned sentences shift by about 0.04, in opposite directions. Moreover, the smaller slices of the data fluctuate more. The balanced accuracy of the band with fourteen or more decisions varies by 0.023, the AUROC on VitaminC by 0.025, and single datasets of LLM-AggreFact vary by up to ten points, as WiCE does (67.9 against 78.0).

Two consequences follow from these results. The first concerns the intervals: the bootstrap resamples the test data, and therefore does not capture the variability due to training. On LLM-AggreFact and on VitaminC, indeed, the two runs of the same procedure differ by more than their intervals would allow (first minus second run: −1.3 points on the macro average of T2, −0.025 on the AUROC of

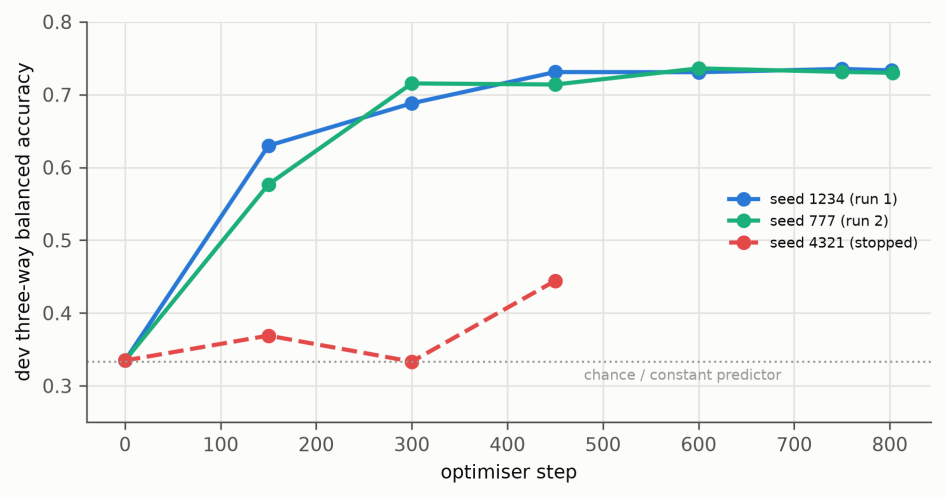


Figure 7.3: Three-way balanced accuracy on the dev set during training of the 0.5B model, with the same procedure and three different seeds. Two runs converge to the same level; the third collapsed to a constant predictor and was stopped.

T3). For this reason differences between systems smaller than the spread between seeds, that is, about 1.5 points on the macro average of LLM-AggreFact and 0.025 in AUROC on VitaminC, are not interpreted. The same applies to single datasets of LLM-AggreFact when the gap does not exceed about five points.

The second consequence is that training does not always converge. A run with a third seed remained close to chance level. Its score on the dev set was 0.369 at step 150 and 0.333 at step 300, when the model predicted "supported" for every sentence, and the training loss remained close to that of a model without information. At step 450 the score had climbed back to 0.444, and the run was stopped shortly afterwards, at about step 470 of 801. It is therefore not possible to know whether it would have recovered by the end of the epoch and, being incomplete, it is excluded from the tables. The procedure has two features that are known to make fine-tuning fragile: it applies no gradient clipping, and the optimiser works without bias correction. The absence of bias correction is the default setting of the framework and has been identified as one of the main causes of failed fine-tuning runs [99, 98]. A single collapse in three runs is not enough to estimate a failure rate, but it shows that the procedure is not fully robust to initialisation.

7.3 Accuracy on the RAG test set (T1)

On the 17,284 sentences of T1 the 0.5B model reaches a three-way balanced accuracy of 0.769 and the 3B of 0.832 (Table 7.1). The difference between the two, +0.063 (+0.051 to +0.076), is the clearest effect of model size. On the binary decision the 0.5B obtains a balanced accuracy of 0.815 and an AUROC of 0.905, while the 3B obtains 0.874 and 0.940 respectively. Both are far above the cues that never compare a sentence with its document: word overlap reaches an AUROC of 0.737, and the classifier that sees only the sentence reaches a three-way balanced accuracy of 0.570.

Table 7.1: GroundOne on T1, on the whole test set, by task and by number of decisions per answer (K). For each model the three-way balanced accuracy, the recall of contradicted sentences and the AUROC of P(supported) are reported. *Indicative value: the cell contains fewer than 30 contradicted sentences.

		GroundOne 0.5B			GroundOne 3B		
Subset	Sentences	Bal. acc.	Contr.	AUROC	Bal. acc.	Contr.	AUROC
All	17,284	0.769	0.68	0.905	0.832	0.76	0.940
Question answering	4,876	0.589	0.03*	0.916	0.699	0.30*	0.948
Summarisation	4,538	0.544	0.13	0.775	0.629	0.30	0.873
Data-to-text	7,870	0.812	0.84	0.929	0.873	0.90	0.955
$K = 1$–4	2,434	0.606	0.18	0.857	0.677	0.25	0.921
$K = 5$–8	8,012	0.758	0.67	0.899	0.829	0.78	0.936
$K = 9$–13	5,201	0.824	0.82	0.926	0.863	0.86	0.945
$K \geq 14$	1,637	0.723	0.50	0.908	0.839	0.68	0.957

Three aspects emerge from the table. The first is that accuracy does not collapse with the length of the answer. Answers with fourteen or more decisions reach a balanced accuracy of 0.839 with the 3B, among the best, and of 0.723 with the 0.5B, slightly below the intermediate bands. The hardest band is instead that of the shortest answers, which mostly contains question-answering and summarisation answers. The finding matters because it is precisely on long answers that a decision model saves the most time; on these answers the 3B gains +0.116 (+0.053 to +0.186) over the 0.5B.

The second aspect is that contradictions are recognised mainly in data-to-text, the task in which RAGTruth concentrates them. In question answering and summarisation the 0.5B separates supported from not-mentioned sentences well, but finds almost no contradictions: the recall of contradicted sentences is 0.03 and 0.13, and the first value is only indicative, because in question answering there are fewer than 30 contradicted sentences. The 3B raises it to 0.30 in both tasks, so

model size compensates in part for the scarcity of these examples in the training data. The third aspect is that question answering and summarisation offer the clearest evidence that the models really read the document. In these two tasks the classifier that sees only the sentence stays at chance level (0.36 and 0.35), while the two models reach 0.59–0.70 in question answering and 0.54–0.63 in summarisation.

The same predictions can also be evaluated at the level of the whole answer. This is the criterion with which the community Decision Index evaluates general-purpose decision models on RAGTruth: an answer is hallucinated if it contains any unsupported content, and the metric is the F1 score of this class. A system that always answers "hallucinated" already reaches an F1 of 0.52 [22]. To judge an answer, GroundOne takes the minimum of the calibrated $P(\text{supported})$ over its sentences and compares it with a threshold fitted on the dev set. With this rule, on the 2,647 answers of T1 it reaches an F1 of 0.733 with the 0.5B and of 0.784 with the 3B. On the leaderboard, by contrast, no general-purpose decision model below one billion parameters exceeds 0.553, Jev reaches 0.765 and the best model, of 27B parameters, 0.803. The comparison is only indicative, because GroundOne was trained on RAGTruth, whereas the leaderboard models are evaluated zero-shot, with a different prompt and on 53 more answers. It nevertheless places a 0.5B specialist far above the general-purpose decision models of the same size.

7.4 Comparison with the baselines

Figure 7.5 compares all systems on T1. The specialised encoders reach an AUROC of 0.814 (MiniCheck-RoBERTa) and 0.821 (HHEM): more than word overlap, but less than both GroundOne models. The 0.5B exceeds MiniCheck-RoBERTa by +0.091 (+0.074 to +0.108) and HHEM by +0.083 (+0.067 to +0.099); the 3B exceeds them by +0.126 and +0.119 respectively. The comparison by task, however, qualifies this result. GroundOne's advantage comes largely from data-to-text, where it benefits from in-domain training and from the recurring structure of the templates, and where the encoders are weakest (AUROC 0.71–0.75). In summarisation, instead, MiniCheck-RoBERTa outperforms the 0.5B model (0.872 against 0.775) and is on par with the 3B (0.873). Figure 7.4 reports the AUROC per task of all systems.

The untrained models show how much of the accuracy is due to training. The untrained 0.5B is at chance level, with a three-way balanced accuracy of 0.372, because it never answers "supported". The untrained 3B, asked the same question, reaches a three-way balanced accuracy of 0.437, against 0.818 for the same model after training, on the same answers. The contribution of the restricted readout, beyond training, is measured separately in Section 7.6.

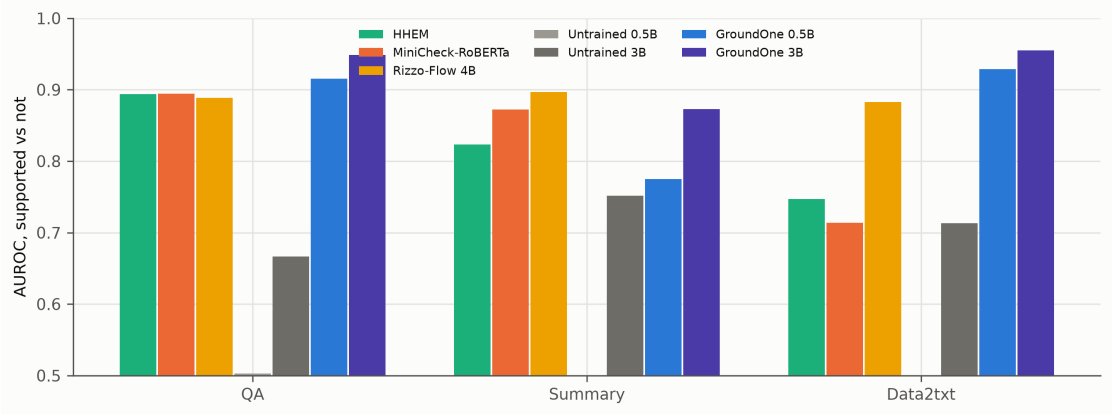


Figure 7.4: AUROC of P(supported) on T1 by task (question answering, summarisation, data-to-text) and by system. GroundOne's advantage is concentrated in data-to-text; in summarisation MiniCheck-RoBERTa outperforms the 0.5B and is on par with the 3B.

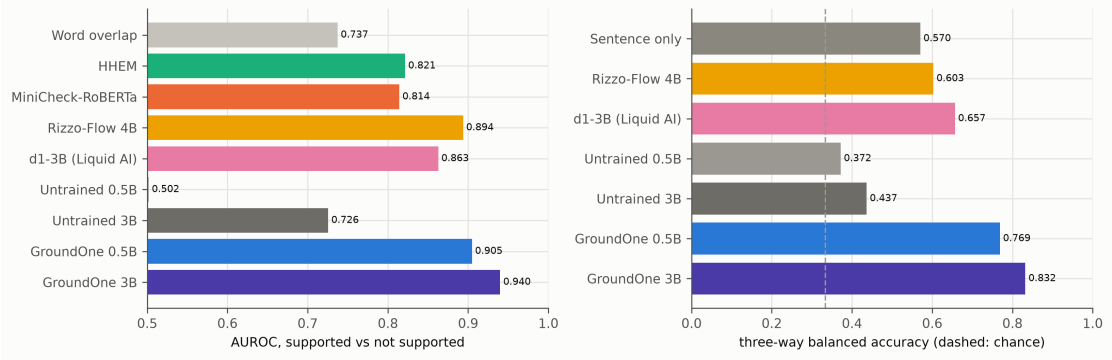


Figure 7.5: T1: AUROC of P(supported) for every system (left) and three-way balanced accuracy for the systems with a three-way output (right). Rizzo-Flow is evaluated on 300 answers and the untrained 3B on 600; all other systems, including d1-3B, are evaluated on the whole test set.

The most instructive comparison is the one with Rizzo-Flow, the larger general-purpose decision model: a 4B model fine-tuned for generic decisions. On the same 300 answers of T1, Rizzo-Flow reaches a three-way balanced accuracy of 0.603, against 0.798 for the 0.5B GroundOne and 0.840 for the 3B. The differences are large: +0.195 (+0.140 to +0.251) for the 0.5B and +0.237 (+0.184 to +0.289) for the 3B. In AUROC, by contrast, Rizzo-Flow (0.894) is close to the 0.5B, which obtains 0.916 on these answers, and the difference is not significant. Rizzo-Flow, in fact, ranks the sentences well, but predicts "supported" for 90 % of them. The 3B ranks them better than Rizzo-Flow (+0.056, +0.034 to +0.081). A specialist eight times smaller therefore outperforms the generalist on the decision it was designed for and matches it in the ability to rank sentences.

d1-3B, a general-purpose decision model of the same size as the 3B, offers the strictest test of the value of specialisation. It is in fact already trained to make typed decisions, and it differs from GroundOne mainly in the data on which it was trained. On the whole of T1 it reaches a three-way balanced accuracy of 0.657 and an AUROC of 0.863. Both GroundOne models do clearly better. The 0.5B, six times smaller, exceeds it by +0.112 (+0.092 to +0.133) in three-way balanced accuracy and by +0.042 in AUROC; the 3B exceeds it by +0.175 (+0.157 to +0.195) and by +0.077. The gap is widest on long answers: when the answer contains fourteen or more decisions, d1-3B drops to 0.527, while the 3B GroundOne stays at 0.839 (+0.312, +0.230 to +0.405). Moreover, d1-3B recognises only half of the not-mentioned sentences (recall 0.49). Within the domain of RAG answers, then, specialisation is worth far more than a generic training to make decisions.

On LLM-AggreFact (T2) the 0.5B model reaches a macro-averaged balanced accuracy of 70.1 over the eleven datasets, against 64.2 for the same model used zero-shot (the score published on the leaderboard) and 72.8 for the zero-shot 7B model reported on the public leaderboard. It remains, however, 3.9 points below MiniCheck-RoBERTa (74.0), a significant difference. On the four datasets closest to retrieval-augmented generation the two systems are on par (74.4 for GroundOne 0.5B, 74.6 for MiniCheck-RoBERTa). On the RAGTruth column GroundOne leads, 82.1 against 76.5, but the comparison favours it, because it was trained on the answers of the RAGTruth training set. Outside RAG, instead, the 0.5B is seven to nine points below the encoder on AggreFact-CNN, AggreFact-XSum, FactCheck-GPT and REVEAL: this is the expected cost of training on RAG data alone.

The 3B model, evaluated on the whole test set, closes most of this gap. Its macro-averaged balanced accuracy is 74.8: it is on par with that of MiniCheck-RoBERTa (74.0; paired difference +0.8 points, interval −0.5 to +2.0) and above the published zero-shot score of the 7B model (72.8), for which no interval can be computed. On the RAG datasets it reaches 76.9, 2.4 points above the encoder (interval +1.4 to +3.4). Its gain over the 0.5B, +4.7 points (+3.4 to +5.9), is spread across the

datasets, with the largest improvements on REVEAL, WiCE, FactCheck-GPT and XSum: model size therefore improves transfer beyond the training domain. The 3B remains clearly below the encoder only on news summarisation (AggreFact-CNN, 59.3 against 67.1). Rizzo-Flow reaches 61.5 on the 1,093 items of its subsample, and on these items the 3B GroundOne does better by 12.0 points (+7.6 to +16.4).

On this benchmark, however, the strongest systems remain above GroundOne. An independent evaluation of the closed decision model Jev, on the same test set and with the same metric, reports 78.6, and 76.9 for a 27B open model read through its answer letters. The prompts differ from those used here, and no interval can be computed [20]. On a broad groundedness benchmark, then, a 3B specialist trained only on RAG answers matches the best small encoder, but remains below the closed state of the art and below much larger open models. The general-purpose d1-3B confirms the picture when the size is the same. On a random subsample of 4,000 items it reaches 75.7, against 74.3 for the 3B GroundOne on the same items, a difference that is not significant (-1.3, -4.0 to $+1.0$), while it exceeds the 0.5B by 6.5 points. Compared with the 3B GroundOne, d1-3B is weaker on the RAGTruth column (78.9 against 87.3) and clearly stronger on WiCE and on dialogue summarisation.

7.5 Transfer to a different domain (T3)

On VitaminC the picture reverses. On the whole test set the 0.5B reaches a three-way balanced accuracy of 0.442 and the 3B of 0.527, with an AUROC of 0.722 and 0.817 respectively, while MiniCheck-RoBERTa reaches an AUROC of 0.968. On the 600 pairs evaluated with Rizzo-Flow, the general-purpose model obtains a three-way balanced accuracy of 0.712 and an AUROC of 0.909, against 0.442 and 0.744 for the 0.5B and 0.537 and 0.845 for the 3B. Figure 7.6 shows the complete comparison on T3.

Two facts help to interpret these numbers. The first is that MiniCheck-RoBERTa starts from the AlignScore model, whose training data include VitaminC [11]. For the encoder, therefore, T3 belongs to the training distribution, whereas for GroundOne it measures transfer from RAGTruth. The second is that the gain of the 3B over the 0.5B on T3 is due to model size more than to training on RAG data: the untrained 3B already reaches 0.531 and an AUROC of 0.821. d1-3B, the general-purpose decision model of the same size, makes the contrast explicit. On the whole of T3 it reaches 0.724 and an AUROC of 0.913, and exceeds the 3B GroundOne by 0.197 and 0.096: the exact opposite of what happens on T1. Specialisation on RAG answers therefore improves the RAG task substantially, but on its own it is not enough to produce a general fact checker. Section 7.12 shows which cases are missed.

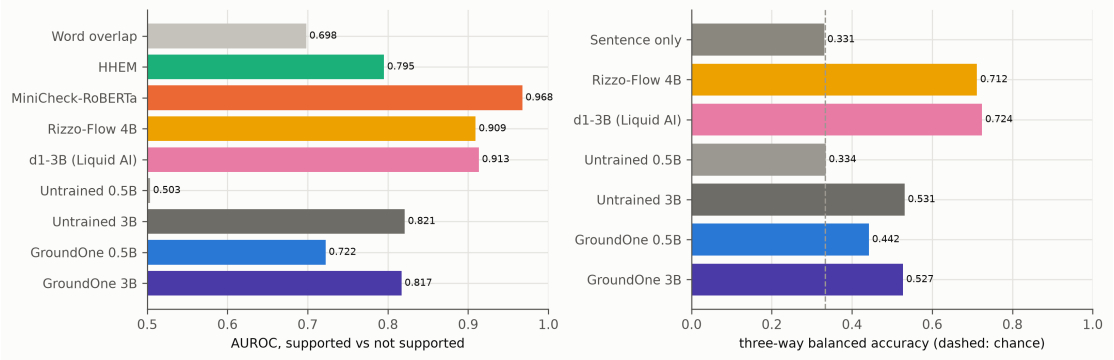


Figure 7.6: Comparison of the systems on T3 (VitaminC): on the left the AUROC of $P(\text{supported})$, on the right the three-way balanced accuracy for the systems with a three-way output. Rizzo-Flow is evaluated on a subsample of 600 pairs; the dashed line marks chance level. For MiniCheck-RoBERTa, VitaminC is part of the lineage of its training data.

7.6 Readout versus generation

The control B2 isolates the contribution of the readout. The comparison keeps the fine-tuned weights, the prompt and the single forward pass unchanged, and changes only the way in which the verdict at each marker is obtained. The verdict is either read from the three answer letters, as GroundOne does, or it is the token that greedy generation would emit (Figure 7.7). For the 0.5B the generated token gives a three-way balanced accuracy of 0.521, against 0.769 for the readout, a difference of +0.248 (+0.228 to +0.268) in favour of the readout. In a quarter of the markers (24.7 %) the model would not emit a letter at all, but mostly a bare space, that is, the beginning of a longer answer. For the 3B, instead, the generated token is almost always a letter (1.2 % format failures) and agrees with the readout in 92 % of the markers; yet the readout remains better, with 0.832 against 0.761 (+0.071, +0.059 to +0.082). When the answer contains fourteen or more decisions, the gap is largest for the small model: +0.346.

The two options have the same latency, because both require a single forward pass: only the decision rule changes. Restricting the choice to the admissible options removes format failures by construction. Moreover, since the model is trained only to order the three letters, the restricted choice also provides the more accurate decision (Section 4.2). This restriction matters all the more, the smaller the model. It is often said that reading the answer logits and generating a single answer token are equivalent operations: for a fine-tuned verifier the equivalence holds for the cost, but not for the accuracy.

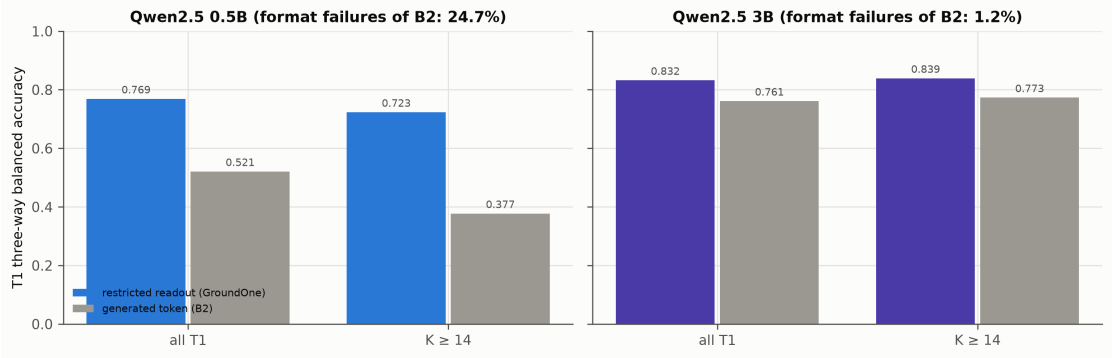


Figure 7.7: Control B2 on the whole of T1. With the same weights and the same forward pass, the verdict is read from the three answer letters (GroundOne) or taken as the token that greedy generation would emit; format failures are counted as errors.

7.7 Calibration and selective prediction

Before calibration, the probabilities underestimate support. In the reliability diagram (Figure 7.8) the curve of the raw probabilities lies above the diagonal, because the class weights train the model as if the classes were balanced. The binary ECE on T1 is 0.30 for the 0.5B and 0.18 for the 3B. The temperature and bias fitted on the dev set bring it to 0.019 for both models, and to 0.020 for the second run of the 0.5B. They also reduce the negative log-likelihood from 0.55 to 0.21 for the 0.5B and from 0.36 to 0.17 for the 3B, leaving the AUROC practically unchanged. The map fitted on the dev set therefore transfers to the test set of the same domain. Figure 7.9 shows the same diagram for the 3B.

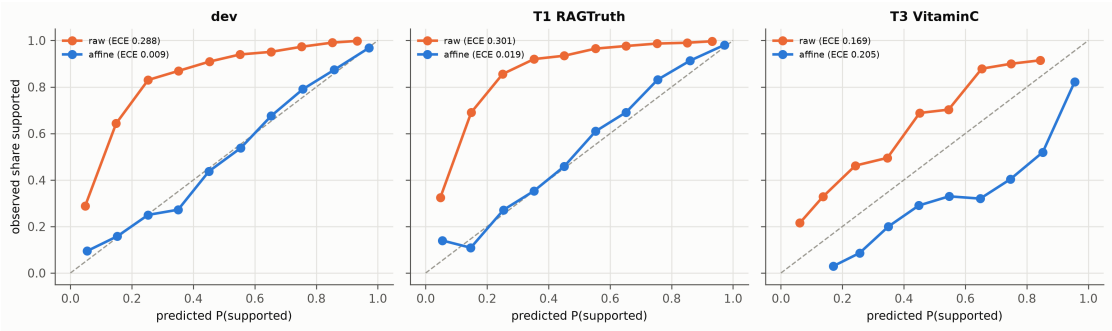


Figure 7.8: Reliability diagram of P(supported) of the 0.5B, before calibration (raw) and after the calibration fitted on the dev set, for the dev set, T1 and T3. Each point represents a probability bin containing at least 20 sentences.

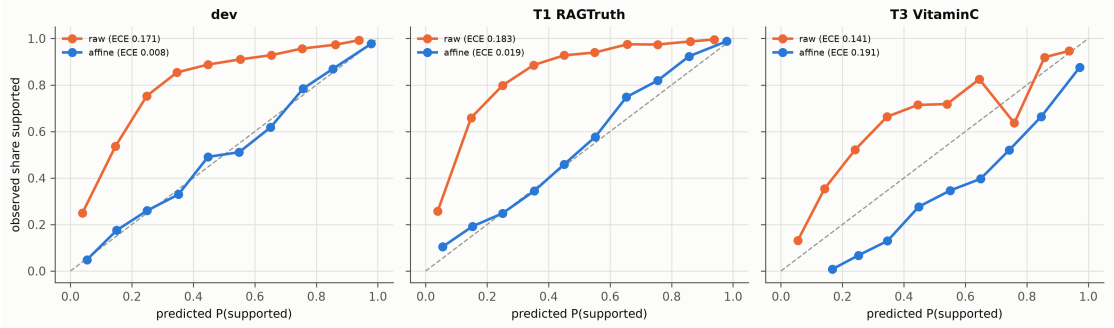


Figure 7.9: Reliability diagram of $P(\text{supported})$ of the 3B, before and after the calibration fitted on the dev set, for the dev set, T1 and T3; each point is a probability bin with at least 20 sentences. As for the 0.5B, the calibration transfers to T1 but not to T3.

Calibration, however, has a consequence that must be kept in mind. The bias restores the true frequency of each class, so the most probable class after calibration becomes "supported" more often. For the 0.5B on T1 the accuracy thus rises from 0.82 to 0.93, but the three-way balanced accuracy falls from 0.77 to 0.63. When the three kinds of error carry the same weight, therefore, choosing the most probable calibrated class is not a good decision rule. The rule with which the labels are served, described in Section 4.4, solves the problem. The rule uses a threshold τ fitted on the dev set, equal to 0.935 for the 0.5B, to 0.895 for its second run and to 0.925 for the 3B. With these thresholds it reaches on T1 a three-way balanced accuracy of 0.758, 0.769 and 0.830, within about 0.01 of the raw decision evaluated throughout the chapter, and keeps the labels consistent with the calibrated probabilities. The thresholds are high because about 88 % of the sentences of the training and dev sets are supported: a sentence is served as supported only when the model is very confident.

Used as confidences, the calibrated probabilities make selective prediction effective. Deferring the sentences on which the model is least confident quickly reduces the error rate of the binary decision: on T1 the AURC is 0.015 for the 0.5B and 0.010 for the 3B (Figure 7.10).

On VitaminC, instead, the calibration does not transfer. The binary ECE rises from 0.17 to 0.21 for the 0.5B and from 0.14 to 0.19 for the 3B, and the rule with which the labels are served loses 0.02–0.04 with respect to the raw decision. The bias, in fact, encodes the base rate of RAGTruth, whereas VitaminC is balanced by construction. A calibration holds for the distribution on which it was fitted, so every real deployment requires a small labelled sample of its own data.

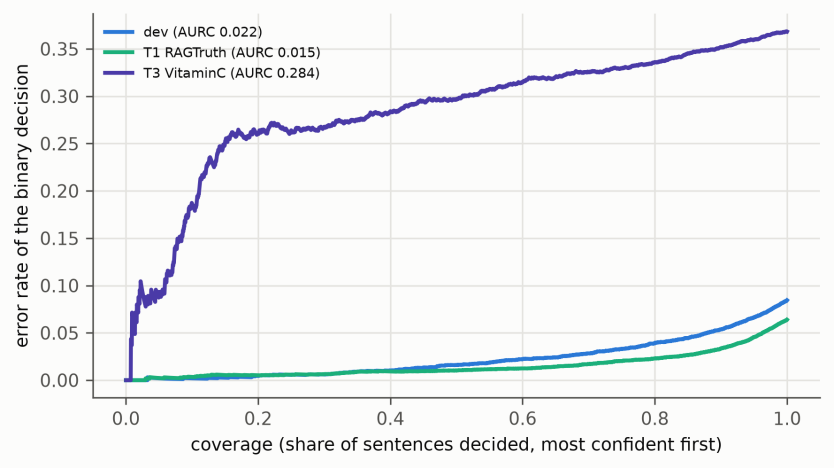


Figure 7.10: Risk–coverage curves of the binary decision of the 0.5B with calibrated probabilities: error rate as the number of least confident sentences that are deferred grows.

7.8 Probabilities at equal output

The equal-output LLM judges serve to establish whether an LLM judge of the same size, asked for the same content as GroundOne, can match the quality of its probabilities. The answer is negative (Figure 7.11). The comparison uses the 200 answers of the latency subset, with all systems calibrated in the same way on the dev set. For the 0.5B and the 3B respectively, the AUROC of P(supported) is 0.94 and 0.97 for GroundOne, 0.48 and 0.68 for the LLM judge whose probabilities are read from the logits, and 0.47 and 0.58 for the LLM judge that writes them. The AURC is 0.009 and 0.005 for GroundOne, against values from 0.048 to 0.063 for the LLM judges, that is, about seven to twelve times larger than those of GroundOne of the same size. The accuracy of the decisions follows the same pattern: 0.835 and 0.900 for GroundOne, at most 0.41 for any LLM judge.

Two further observations matter in practice. The first is that probabilities written in words carry almost no information. The 3B writes a vector with all the mass on a single class for 93 % of the sentences, and uses only sixteen distinct vectors. The 0.5B instead writes uninformative ties, such as equal probabilities for all classes: if the ties without a label are counted as errors, its balanced accuracy falls from 0.234 to 0.081. This agrees with the observation that the verbalised confidence of small models is coarse and poorly informative [95, 94]. The second observation is that the ECE alone would hide all of this. After calibration every system, LLM judges included, obtains a binary ECE between 0.03 and 0.05, because a calibration map can align the average confidence with the base rate even when

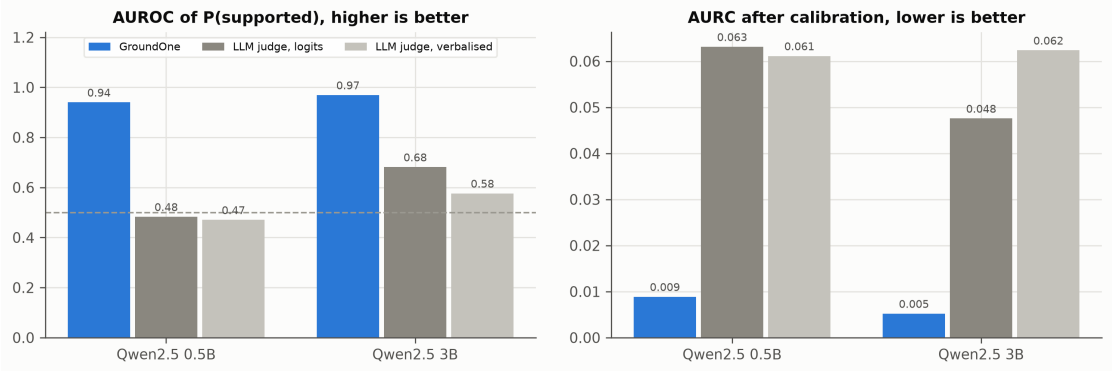


Figure 7.11: Equal-output comparison on the 200 answers of the latency subset. LLM judges of the same size are asked for GroundOne's output, with the probabilities read from the logits of the generated letter or written in JSON; all systems are calibrated on the dev set with the same map. Only the sentences with a valid verdict are considered.

the probabilities do not separate the classes. Discriminative ability must therefore be measured separately, with the AUROC and the AURC.

Asking for the probabilities, finally, has a cost. The LLM judge that reads them from the logits is as fast as the terse LLM judge, because it generates the same letters. The LLM judge that writes the JSON object, instead, needs 17.0 seconds with the 3B for an answer with fourteen or more decisions, as much as the verbose LLM judge, and is no more accurate than the terse LLM judge.

7.9 Latency as a function of K

Figure 7.12 reports the central measurement of the thesis: the time needed to verify an answer as the number of decisions grows, measured on the same 200 answers of T1. Among the systems run in process on MLX, GroundOne is the fastest at both sizes and in every band of K, and its advantage grows with K. When the answer contains fourteen or more decisions, the 0.5B GroundOne needs 0.41 seconds, against 0.80 for the terse LLM judge and 2.51 for the verbose LLM judge of the same size. It is therefore 1.95 times faster than the first (1.82 to 2.12) and 6.16 times faster than the second (5.72 to 6.66). In the same band the 3B GroundOne needs 2.57 seconds, against 4.44 and 15.86 seconds for the two 3B LLM judges, and is therefore 1.73 times (1.65 to 1.82) and 6.18 times (5.88 to 6.63) faster.

Reading all the verdicts in a single forward pass is also faster than asking one question per sentence in a batch: at the highest values of K the gain is a factor

of 1.4–1.5. The other lever is model size: the 0.5B GroundOne verifies an answer about six times faster than the 3B, with a median time of 0.33 seconds against 1.99.

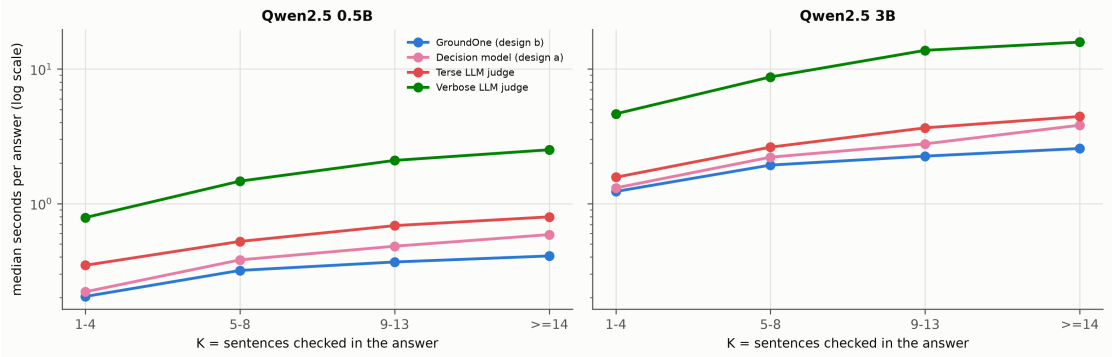


Figure 7.12: Median time per answer as a function of K on the same 200 answers of T1 (in-process MLX, one answer at a time, M4 with 16 GB). GroundOne, the decision model with one question per sentence and the terse and verbose LLM judges are compared. Logarithmic scale.

7.9.1 Reading the text and deciding

GroundOne's time grows with K, but not because of the number of decisions as such. Answers with more sentences are also longer: in the latency subset the prompt goes from about 550 tokens in the $K \leq 4$ band to about 1,140 tokens at $K \geq 14$, and the prefill time grows with the length of the text. To separate the two components, the time of every answer was split into a *reading* part, the prefill of instructions, document and answer that every approach must pay, and a *deciding* part. For GroundOne, reading is a forward pass over the same text without markers, and deciding is the difference from the served pass, that is, the cost of the markers and of the readout; the two measurements are interleaved five times with the 0.5B and three times with the 3B, and the median is kept. For the LLM judges, reading is the time to the first generated token, and deciding is the decoding of the verdicts. The measurement uses the same 200 answers and the same protocol as the previous section, and reproduces its overall times within 4 %. Table 7.2 reports the medians, Figure 7.13 their trend with K.

For GroundOne, reading takes 97 % of the time at both sizes. Deciding costs a median of 12 milliseconds per answer with the 0.5B and 67 with the 3B, that is, 1.3 and 6.7 milliseconds per sentence: the three tokens of the marker and the vocabulary projection at the marker positions only. The terse LLM judge spends instead 30 and 139 milliseconds per sentence, the verbose one 170 and 1,033. For

Table 7.2: Time per answer split into reading the text (prefill) and deciding, on the same 200 T1 answers: medians over all answers, in seconds. For GroundOne deciding is the cost of the markers and of the readout, for the LLM judges the decoding of the verdicts. The reading share is the median of the per-answer shares.

Model	System	Reading	Deciding	Total	Reading share
0.5B	GroundOne	0.330	0.012	0.342	97 %
	Terse LLM judge	0.297	0.383	0.673	46 %
	Verbose LLM judge	0.305	1.509	1.826	18 %
3B	GroundOne	1.941	0.067	2.002	97 %
	Terse LLM judge	1.753	1.196	3.102	61 %
	Verbose LLM judge	1.756	8.313	10.277	18 %

the LLM judges, deciding is a substantial part of the time, and for the verbose LLM judge the main part. Net of reading, GroundOne decides 33 times faster than the terse LLM judge with the 0.5B (30 to 38) and 18 times faster with the 3B (15 to 20); against the verbose LLM judge, 131 times (112 to 148) and 125 times (116 to 142). With fourteen or more sentences the ratios are similar: 29 and 24 times against the terse LLM judge, 141 and 151 against the verbose one.

The overall speed-up remains about two against the terse LLM judge because reading, which dominates the time, is shared by all approaches: GroundOne's advantage lies in the part that the readout replaces, not in the prefill. The split also shows a cost that the overall times hide. GroundOne's reading is about 10 % slower than that of the LLM judges, although its prompt is slightly shorter: it is the extra computation of the LoRA adapter, which stays separate from the quantised base weights. Fusing the adapter into the weights would remove this cost, but would require quantising the model again and recalibrating it.

Composition of the prefill. The prefill is almost all of GroundOne's time, but its parts do not change in the same way from one verification to the next. The prompt has four of them, in order: the instructions, the same for every request; the document, the same as long as answers on the same sources are verified; the answer, which changes with every version; the markers, three tokens per sentence (Figure 7.14a). On the 200 answers of the latency measurement the instructions are 63 tokens, 7 % of the prompt; the document has 577 at the median, 65 %; answer and markers together make up the remaining 28 %, a share that grows with the number of sentences, from 19 % below five sentences to 35 % from fourteen up. The time is split in the same way (Figure 7.14b): with the 3B, on answers with fourteen or more sentences, reading the document is 60 % of GroundOne's time, reading the

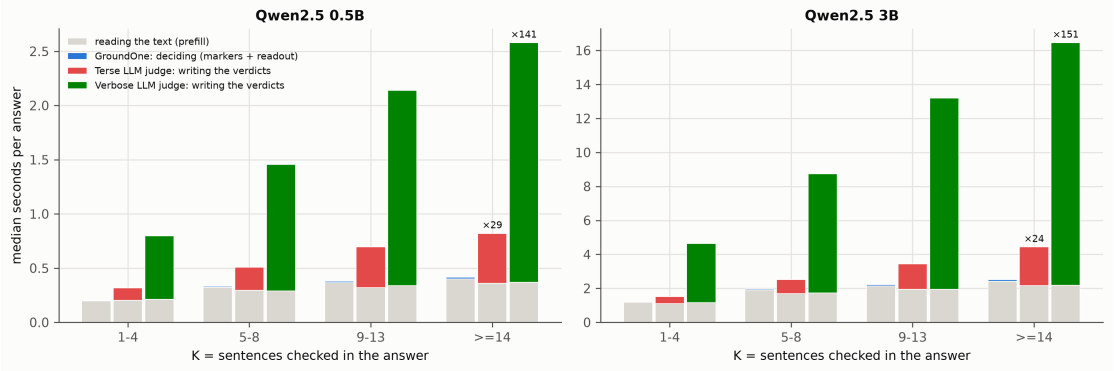


Figure 7.13: Median time per answer split into reading the text (prefill, in grey, shared by every approach) and deciding (in colour), per K band, on the same 200 T1 answers. The labels give, for $K \geq 14$, how many times longer the LLM judge takes to decide than GroundOne.

answer 34 %, the instructions 2 % and deciding 3 %. The terse LLM judge reads the same document, but half of its time goes into writing the verdicts.

This composition determines what is worth caching. The instructions can always be reused, but they weigh little: keeping them costs a single KV cache per process and saves a few percentage points. The document is the part that matters: it is the largest share of the prompt and of the time, and it stays the same whenever the same sources are verified again, that is, for a rewritten answer, for several alternative answers or for several questions on the same library. Keeping it in memory is cheap: with the 3B the KV cache takes about 36 KB per token, and a median document about 21 MB. The service therefore keeps the last four documents read and the application the last sixteen heads (instructions and document). The document, moreover, is known before the answer: it can be read while the generator writes or while the user reads the answer, so that the first verification, too, reads only the answer (Chapter 8). The answer, instead, cannot be reused as a block, because it changes with every version; only its initial part, up to the first changed sentence, can be reused, as the paragraph *Reuse of the answer prefix* shows. GroundOne's decision, finally, is already a negligible fraction of the time, while the judge's is decoding, which no cache removes: this is why caching widens the gap between the two.

Prefix reuse. The same measurement was repeated with part of the prompt already in the KV cache when the clock starts, the same for GroundOne and for the LLM judges: the instructions alone, identical for every request (63 to 81 tokens), or the instructions and the document (about 640 tokens at the median), as when

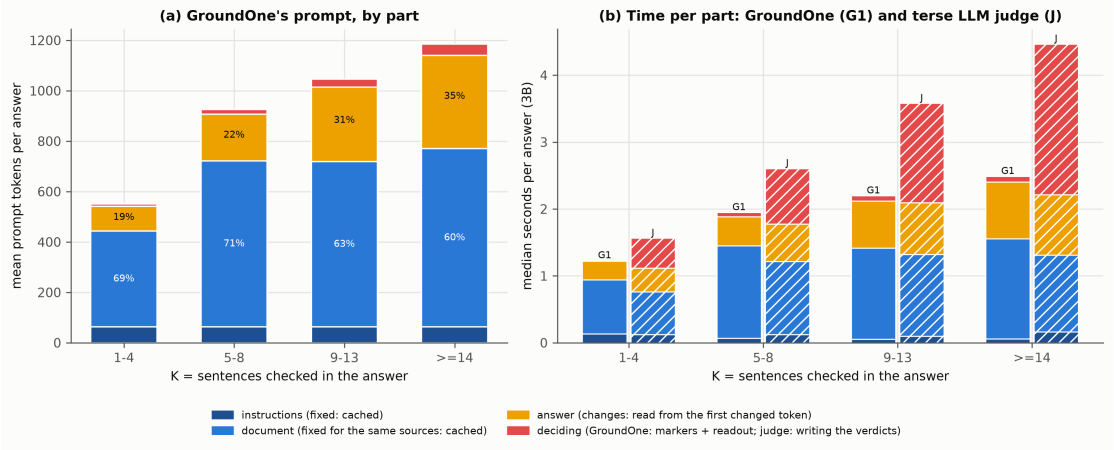


Figure 7.14: Composition of the prefill on the 200 T1 answers used for latency. (a) Tokens of GroundOne's prompt by part, mean per K band; the labels give the share of the document and that of answer and markers. (b) Time per part with the 3B, derived from the runs with nothing, the instructions only, and the instructions and the document in the cache as differences of the band medians, so that each column adds up to the median time without cache. In blue, the parts that stay the same across verifications of the same sources, the only ones worth caching.

a rewritten answer is verified again on the same sources (Section 4.2).[1] Table 7.3 reports the overall times.

As the composition of the prefill anticipates, caching the instructions alone saves 3 to 5 % of the time, little more than the measurement noise, while the document makes a clear difference. With the document cached, GroundOne verifies an answer 3.2 times faster with the 0.5B (0.11 seconds at the median) and 3.4 times faster with the 3B (0.58 seconds). The LLM judges gain much less, 1.3 and 1.7 times for the terse one and 1.1 and 1.2 for the verbose one, because the cache removes only the reading and leaves the decoding of the verdicts intact, which cannot be reused. As a result, GroundOne's advantage over the terse LLM judge rises from about 2 to 4.8 times with the 0.5B (4.1 to 5.2) and from 1.6 to 3.1 times with the 3B (2.8 to 3.3), and over the verbose LLM judge to about 15 times. On answers

[1] In the MLX version used for the measurements (0.29.3), the fused attention kernel with the implicit causal mask returns wrong results for some shapes when many queries follow cached keys, for example 501 tokens read after 523 cached ones. The paths with a cached prefix therefore use an explicit causal mask, which matches a float32 reference; after this correction the logits with and without the cache differ by at most 0.07 and the probabilities by 0.014, that is, within bfloat16 rounding. The measurements without a cache, and so all the other results, do not take this path. The Swift version of MLX used by the application does not show the error.

Table 7.3: Overall time per answer with part of the prompt already in the KV cache, on the same 200 T1 answers: medians in seconds. The same cache applies to GroundOne and to the LLM judges. The last column is the ratio between the median times without a cache and with the document cached.

Model	System	No prefix	Instructions cached	Document cached	Saving with the document
0.5B	GroundOne	0.342	0.333	0.108	3.2×
	Terse LLM judge	0.673	0.652	0.520	1.3×
	Verbose LLM judge	1.826	1.844	1.683	1.1×
3B	GroundOne	2.002	1.913	0.583	3.4×
	Terse LLM judge	3.102	3.002	1.795	1.7×
	Verbose LLM judge	10.277	9.998	8.742	1.2×

with fourteen or more sentences, the band in which Section 7.9 measures the 1.7 to 2 speed-up, the advantage over the terse LLM judge becomes 3.6 times with the 0.5B (3.5 to 3.8) and 3.4 times with the 3B (3.2 to 3.4). With the document cached, 90 % of GroundOne's time is the reading of the answer alone and the rest is deciding: verifying a rewritten answer therefore costs little more than its length.

Reuse of the answer prefix. A rewrite usually changes one or a few sentences, and everything before the first changed token is identical to the version already verified. The cache can therefore keep the whole sequence of the last verification, answer included: the new pass starts at the first changed token, and the verdicts of the earlier markers, which causal attention makes independent of what follows them, are taken unchanged from the previous pass. To measure it, one scored sentence of each of the 200 answers was changed, the first, the middle or the last one, by appending a few words to it (", as reported."), and the new version was verified once with only the document cached and once with the previous version of the answer cached as well. The change stands in for a rewrite: the time depends only on where the first changed token falls and on how many tokens follow it, not on what the sentence says. Table 7.4 reports the medians.

For GroundOne the time follows the tokens left to read. When the last sentence changes, 8 tokens are read at the median instead of 249, and the verification takes 0.02 seconds with the 0.5B and 0.09 with the 3B, five and six times less than with the document alone; a change in the middle sentence halves the time; a change in the first sentence leaves almost the whole answer to read, and the gain is 13 to 15 %. Over the three positions together, the gain is 2.6 times with the 0.5B and 2.7 times with the 3B. The terse LLM judge gains 3 to 29 %: it reads less as well, but all its verdicts come after the whole answer, so a change in any sentence invalidates

Table 7.4: Verification of an answer with one sentence changed, on the same 200 answers of T1: median times in seconds with only the document cached and with the previous version of the answer cached as well, and in parentheses the ratio between the two. The LLM judge reuses the same prefix but writes all its verdicts again.

Model	System	Document cached	First sentence changed	Middle sentence changed	Last sentence changed
0.5B	GroundOne	0.106	0.095 (1.1×)	0.051 (2.1×)	0.021 (5.0×)
	Terse LLM judge	0.512	0.516 (1.0×)	0.485 (1.1×)	0.452 (1.1×)
3B	GroundOne	0.589	0.515 (1.2×)	0.285 (2.1×)	0.093 (6.3×)
	Terse LLM judge	1.828	1.717 (1.1×)	1.549 (1.2×)	1.412 (1.3×)

every one of them, and their decoding, most of its time, is repeated in full. With the previous version cached, GroundOne's advantage over the terse LLM judge therefore rises from 4.8 to 12 times with the 0.5B (11.2 to 12.9) and from 3.1 to 7.1 times with the 3B (6.6 to 7.8). The service and the application reuse the answer prefix in the same way (Chapter 8), and the same tests that check the document cache check that the reused verdicts match those of a pass without cache.

Speed is not the only difference (Figure 7.15). The untrained LLM judges are also much less accurate, and they do not always respect the output format. The 0.5B verbose LLM judge produces a parsable verdict for only 3 % of the sentences, because it mostly copies the answer back, while the 0.5B terse LLM judge fails on 23 % of the sentences. The 3B LLM judges respect the format much better (0.1 % format failures for the terse LLM judge and 7.6 % for the verbose one), but stop at a three-way balanced accuracy of about 0.40, against 0.90 for the 3B GroundOne on the same answers. The 3B verbose LLM judge, finally, writes on average 248 tokens per answer, all of which must be generated before the verdict can be used; GroundOne, instead, generates none.

7.10 Energy per verification

The energy measurement adds a qualification that latency alone hides (Figure 7.16). Among the measured systems, that is, GroundOne and the LLM judges, GroundOne is the one that uses the least energy at both sizes: 5.5 joules per answer with the 0.5B and 33 with the 3B, against 8.9 and 43 joules for the terse LLM judge and 22.5 and 112 joules for the verbose LLM judge. Its advantage in energy, however, is smaller than its advantage in time. Compared with the terse LLM judge, GroundOne uses 1.6 times less energy with the 0.5B and 1.3 times less with

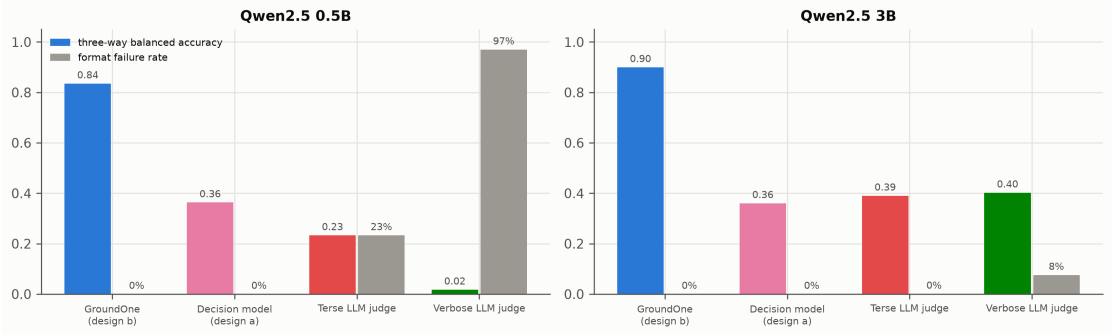


Figure 7.15: LLM judges on the latency subset: three-way balanced accuracy, with format failures counted as errors, and share of sentences whose verdict could not be parsed.

the 3B; compared with the verbose LLM judge, 4.1 and 3.4 times less.

The reason is the power drawn during the computation. GroundOne spends its time in a single long prefill, which keeps the GPU fully busy and draws about 16 W above the idle power. The autoregressive decoding of the LLM judges, instead, is limited by memory bandwidth and draws 11–14 W, but for a much longer time. This is the behaviour expected from the two phases of transformer inference [42], and it implies that a speed-up should never be read as an equivalent saving of energy [67, 68].

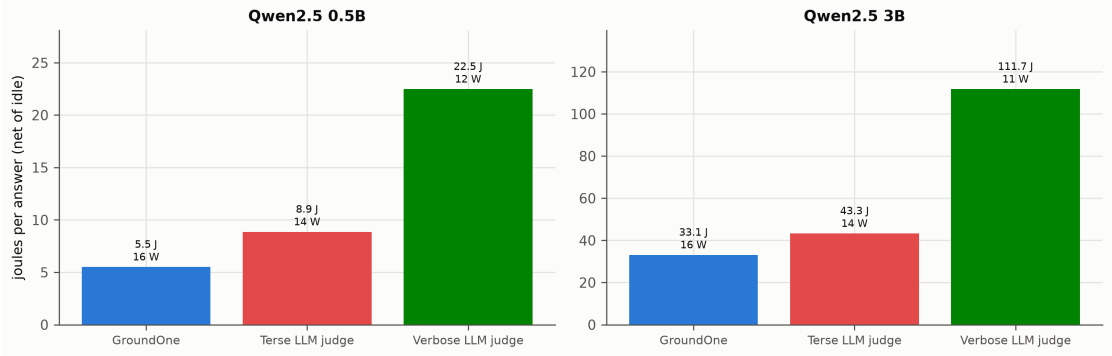


Figure 7.16: Energy per verified answer on 100 answers of the latency subset, measured with `powermetrics` on the CPU, GPU and Neural Engine net of the idle power, together with the mean net power during the run.

In absolute terms the values are small. A consumption of 5.5 joules corresponds to about 0.0015 watt-hours: with the energy of one watt-hour, therefore, the 0.5B GroundOne could verify about six hundred answers, entirely on the laptop.

7.11 Locality: offline execution, quantisation and co-residence

Three measurements complete the picture of the on-device cost (Section 6.5). The first concerns independence from the network. The complete evaluation of T1 with the 0.5B model, repeated inside a sandbox that denies the process any network access, produces predictions identical bit for bit to those of the original run on all 17,284 sentences. Loading the model, inference and evaluation therefore require no connection at all. The test is stronger than a disconnected machine, because the process cannot reach the network even when the machine is connected.

The second concerns quantisation. When the same adapter of the 0.5B is applied to the original bf16 weights instead of the 8-bit ones, the probabilities change on average by 0.011 on T1 and by 0.007 on T3, and the predicted class agrees on 98.3 % and 98.9 % of the sentences. The metrics remain unchanged: the three-way balanced accuracy is 0.769 with the 8-bit base and 0.768 with the bf16 one on T1, 0.442 in both cases on T3, and the AUROC on T1 is 0.905 against 0.904. The 8-bit base, which halves the memory of the weights, therefore costs no measurable accuracy. The measurement, however, concerns the sensitivity to the precision of the base of an adapter trained on the 8-bit base, not a model trained entirely in bf16.

The third concerns co-residence with the generator. In the same process, the 4-bit Qwen2.5-3B-Instruct generator and GroundOne verify 30 answers generated from T1 documents, with a median of 169 generated tokens and seven scored sentences per answer (Table 7.5). With the 3B verifier the peak memory allocated by MLX is 6.1 GB, with the 0.5B 3.3 GB: both well below the 16 GB of the machine. Verification takes 30 % of the total time per answer with the 3B and 6.7 % with the 0.5B. The small model therefore adds a modest fraction to the generation time, whereas the 3B, comparing the medians of the two times, lengthens it by about a third in exchange for higher accuracy. During the run with the 3B, with the usual desktop applications open, the free memory of the whole system dropped for a few moments to about 1.6 GB: the pair fits comfortably in the process, but on a 16 GB laptop it leaves little headroom for heavy desktop use. The retriever is not included in this measurement; the application of Chapter 8 adds it.

7.12 Error analysis

The weak result on VitaminC and the missed contradictions on T1 have a common cause, which word overlap brings to light. On VitaminC, when a supported claim shares few words with its evidence (overlap below 0.4), the 0.5B recognises it in only 2 % of cases. When the overlap exceeds 0.7, instead, it recognises it in 64 %

Table 7.5: Co-residence of generator and verifier in a single process: 4-bit Qwen2.5-3B-Instruct as the generator, 30 T1 documents, greedy generation up to 300 tokens. Memory allocated by MLX; median times per answer; the share of verification is the median of the per-answer shares.

Verifier	Memory after loading	Peak memory	Generation	Verification	Share of verification
GroundOne 3B	5.0 GB	6.1 GB	4.75 s	1.68 s	30 %
GroundOne 0.5B	2.3 GB	3.3 GB	4.65 s	0.28 s	6.7 %

of cases. Moreover, the 0.5B never recognises a contradiction in a claim without digits. The models therefore rely in part on lexical overlap: a paraphrased claim looks "not mentioned", while a small change to a copied sentence, often a number, goes unnoticed.

The same weakness is found on T1, although with less weight. When the lexical overlap with the document is low or intermediate (up to 0.7), the three-way balanced accuracy of the 0.5B is 0.77. For sentences that take more than 70 % of their words from the document, instead, it falls to 0.54, with a recall of 0.24 for contradicted sentences and of 0.43 for not-mentioned ones. The 3B reduces this blind spot without removing it: on the same sentences its recall is 0.36 and 0.56, and its balanced accuracy 0.63. Hallucinations hidden in sentences that otherwise copy the source thus remain the main limitation of both models. VitaminC is not harder in absolute terms: it consists largely of precisely this kind of case, that is, paraphrased support and minimal edits, which are less numerous in RAGTruth.

Appendix A, finally, shows the output of every system on the same example answer.

Chapter 8

GroundOne Studio: a local application and a case study

The previous chapters measure GroundOne as an isolated component. This chapter puts it to work inside a complete application, GroundOne Studio, a RAG assistant for macOS in which document retrieval, answer generation and verification take place in the same process, on the same 16 GB laptop used for the experiments. The aim is twofold. On the one hand, the application completes the argument for locality (Chapter 2): it is not enough for the verifier to be able to run locally; it must also be shown that a usable system can be built around it without any external service. On the other hand, it makes visible, on real answers, the properties measured in Chapter 7: the verification of every sentence in a single forward pass, the calibrated probabilities, the comparison with a generative LLM judge. The measurements collected in the application form a case study and are kept separate from the results of Chapter 7: they describe the behaviour of the complete system and do not establish new records.

8.1 Goals and requirements

The application was designed around five elements that the user must be able to see, each tied to a claim of the thesis (Table 8.1). One constraint applies to all of them: no data leaves the machine. Models, document index, conversations and logs reside on the local disk, and the application works without a network connection.

Table 8.1: What the application shows and which claim of the thesis it supports.

What the user sees	Claim supported
Every sentence of the answer coloured according to the GroundOne verdict, with the calibrated probabilities.	The probabilities are usable and consistent with the label shown (RQ3).
The comparison, on the same answer, with an LLM judge run locally: verdicts and times side by side.	Reading all the verdicts in a single forward pass, without generating text, is more accurate than an LLM judge that writes the verdicts, and faster, increasingly so as the answer grows longer (RQ2).
A panel with the generation and verification times and the memory in use.	Generator and verifier coexist in 16 GB, and verification costs little compared with generation (RQ4).
The rewriting of the flagged sentences from the documents, verified again.	Verification is cheap enough to sit inside the generation loop.
For every sentence, the passage of the document that supports or contradicts it, highlighted.	Attribution to the passage can be obtained with the same model; it is an unevaluated prototype (Section 10.2).

8.2 Platform choices

The application is written in Swift, with a SwiftUI interface, and runs the models with MLX Swift [123, 124], the Swift version of the same library used to train GroundOne. Three alternatives were considered: a Python application around the existing package, with a web interface; a Rust application, with unofficial bindings to MLX [125] or with another runtime; a port to llama.cpp with weights converted to the GGUF format. The first would have shown that the experiment can be served locally, but as a server and a page in the browser, not as an installable application. The last two would have required demonstrating numerical equivalence with the results anew, and in the case of llama.cpp recalibrating the model after a new quantisation. MLX Swift, by contrast, loads the same weights and the same adapter and relies on the same MLX backend: if a conformance test confirms it (Section 8.4), the numbers of the thesis also hold for the application, without retraining or recalibrating anything. The price is portability: the interface is tied to Apple platforms, and compiling the MLX shaders requires Xcode.

8.3 Architecture

The application is divided into three layers (Figure 8.1). The first is the interface. The second is a core that does not depend on the engines: it manages the session (answer, verification, rewriting), applies the policy and saves conversations and logs. The core knows only five abstract interfaces, called *ports*: retriever, generator, verifier, LLM judge and attributor. The third layer contains the engines that implement them. Thanks to this separation the core was tested with fake, deterministic engines, just as the Python package is tested with a fake backend, and a different engine, for example a generator on llama.cpp, could replace the current one without touching the rest.

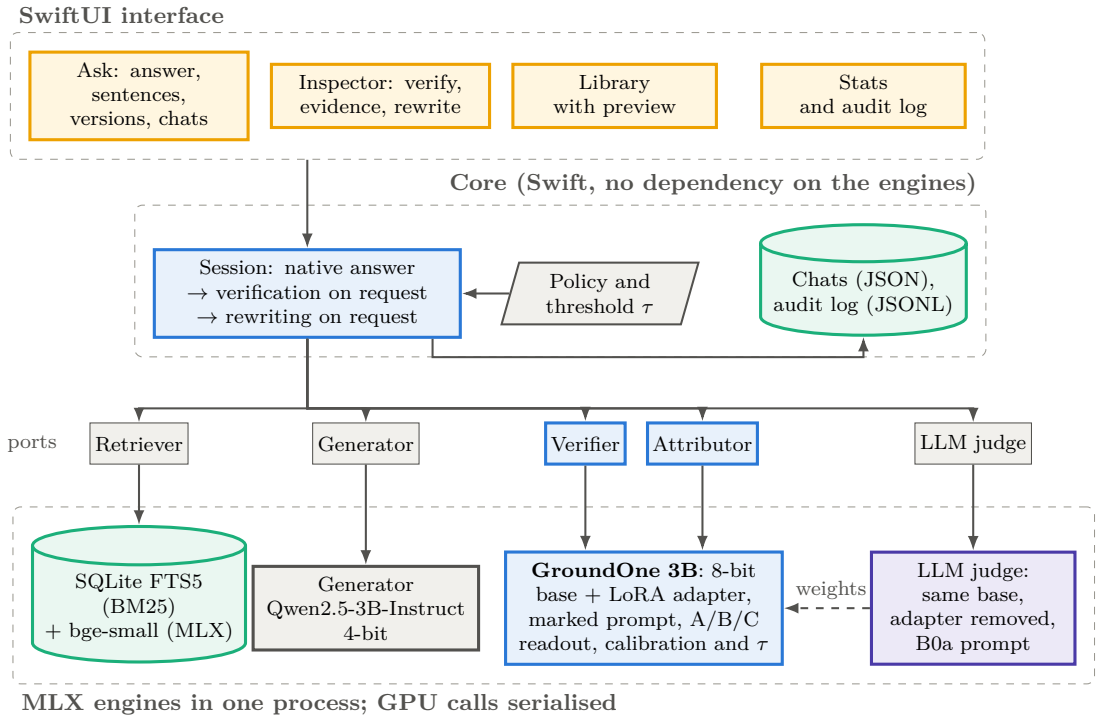


Figure 8.1: Architecture of GroundOne Studio. The core knows only the ports; the engines, all in the same process, are the retriever, the generator, GroundOne (verifier and, passage by passage, attributor) and the LLM judge, which uses the same weights as GroundOne with the adapter removed. The components in which GroundOne is involved are shown in blue.

There are four engines. The generator is Qwen2.5-3B-Instruct quantised to 4 bits, the same one used in the co-residence measurement of Section 7.11. The verifier is GroundOne 3B, that is, the 8-bit base with the LoRA adapter, the

marker prompt, the readout of the letters A, B and C and the calibration fitted on the dev set. It is the default model because it is the most accurate, and in the application verification happens only on request, in less than two seconds for a typical answer; a selector allows switching to the faster 0.5B. The LLM judge used for the comparison is the same base model as GroundOne with the adapter removed at runtime and the prompt of the terse LLM judge B0a (Section 6.2): it is exactly the LLM judge measured in the thesis, it takes no additional memory and it does not leave the machine. The retriever, finally, combines a full-text index and a small embedding model (Section 8.5).

All calls to the models go through the model containers of MLX Swift, which serialise access to them, and are executed one at a time on the GPU, as happens with the single worker thread of the Python service (Section 4.5). Generation and verification therefore do not compete for resources, and the measured times are not confounded by concurrent executions. With the three models loaded, the memory allocated by MLX is about 5.3 GB and reaches a peak of 6.2 GB during one question and of 6.9 GB after a session of twelve questions, when the KV cache also keeps the sources of the earlier ones. The first value is in line with the co-residence measurement of Table 7.5, which did not include the retriever. The embedding model therefore adds little: it takes about 130 MB on disk.

8.4 Porting GroundOne to Swift

Four parts of GroundOne were rewritten in Swift; the rest, namely weights, adapter and calibration, is loaded as it is. The first is prompt construction. As in training, header, numbered sentences, markers and line breaks are tokenised separately and then concatenated, because tokenising the joined string would produce different identifiers at the boundaries between the pieces. The tokeniser is that of swift-transformers [126]. The second is the readout: the model computes the hidden states of the whole sequence, but the output head is applied only at the marker positions, and at each of them only the logits of the three letters are read. The third is calibration: temperature, bias and threshold τ are read from the same file produced in Python. The fourth is sentence segmentation: PySBD does not exist for Swift, and the segmentation rule of Section 5.3 was reproduced with the sentence tokeniser of the operating system [127], adding the same rules for lists and for checkable content.

Conformance test. Before the interface was built, the port was compared with Python on 200 answers of T1, stratified by number of sentences like the subset used for latency, for a total of 1,747 markers. For every answer, Python exports the token identifiers, the marker positions, the logits, the calibrated probabilities and

the labels; Swift must reproduce them (Table 8.2). The token identifiers and the marker positions coincide in all 200 answers with both models. With the 3B, the default model of the application, the maximum difference between the calibrated probabilities is 0.005 and all the labels coincide: for the same input, the results of the thesis therefore also hold for the verifier of the application. With the 0.5B, three sentences out of 1,747 exceed a difference of 0.01, with a maximum of 0.031, and the labels coincide in 99.71 % of the cases; every disagreement concerns a sentence whose probability of support falls within 0.01 of the threshold. A plausible but unverified cause is a different order of summation in reduced precision, which weighs more heavily on the small model. The same test compares in Swift the cached path, which reuses the sources or the beginning of an answer already verified after a one-token change, with the pass without cache: the probabilities differ by at most 0.007 with the 3B and 0.019 with the 0.5B, the same order of magnitude observed in Python (Section 7.9.1).

Table 8.2: Conformance test between the Swift and the Python implementations on 200 answers of T1 (1,747 markers). The logits are computed in Swift on the same token identifiers produced by Python.

Check	GroundOne 3B	GroundOne 0.5B
Identical token identifiers and marker positions	200/200	200/200
Maximum difference between the logits	0.070	0.172
Maximum difference between the calibrated probabilities	0.005	0.031
Sentences with a probability difference below 0.01	100 %	99.83 %
Identical served labels	100 %	99.71 %
Maximum difference between the probabilities with and without the cache	0.007	0.019

Speed is also almost the same. On the same 200 answers, verification takes on average 2.13 seconds in Swift against 2.04 in Python with the 3B, about 4 % more, and 0.34 against 0.33 seconds with the 0.5B; these are means, whereas Chapter 7 reports medians. Segmentation, finally, produces the same sentences as PySBD in 98.4 % of the 2,647 answers of T1. The remaining differences mostly concern abbreviated titles and quotations that span several sentences. Without three additional rules, which do not split the sentence after initials and common abbreviations or before a lower-case word, and which attach list bullets to the following sentence, the agreement would fall to 95.2 %.

8.5 Library and document retrieval

The library of the application consists of the sources of T1: the 147 CNN and Daily Mail news articles of the summarisation task and the 450 MS MARCO web passages of question answering, linked to the 150 real questions they come from. The structured Yelp records, in JSON format, are excluded, because they are not documents to converse with. The documents thus belong to the domain in which GroundOne was trained and calibrated, and the probabilities shown to the user should largely retain their meaning. This was not measured: generator, prompt and a context made of several passages differ from those of RAGTruth in any case.

Every document is split into passages of about 140 words, aligned with sentence boundaries, which keep the source file and the position in the text. Retrieval is hybrid. A full-text index in SQLite [128] ranks the passages with BM25[1], while a small embedding model, bge-small-en-v1.5 [130], also run with MLX, ranks them by semantic similarity. The two rankings are merged with reciprocal rank fusion[2], and every document can contribute at most three passages, so that a comparison question reaches different sources. The six retrieved passages, preceded by the question, form the document that GroundOne reads, in the same form as the question answering of RAGTruth.

On the 150 real MS MARCO questions, hybrid retrieval returns all the original passages of the question among the six passages, with BM25 alone as with fusion. The result is, however, optimistic by construction: in the library every passage is preceded by its own question, used as a title, and the question asked is identical. The measurement confirms that the index works, not that retrieval is robust to freely formulated questions.

8.6 Using the application

The usage flow is summarised in Figure 8.2. The user asks a question, or picks one of the examples offered on the start screen. The generator receives the retrieved passages and writes an answer of several paragraphs, which appears exactly as it was generated, without any intervention (Figure 8.3). Only at this point does the user decide whether to verify it, and how: with GroundOne alone, or with GroundOne

[1] BM25 is a classic scoring function of information retrieval: it rewards passages that contain the terms of the query, giving more weight to terms that are rare in the collection and damping the effect of repetitions and of passage length [129].

[2] *Reciprocal rank fusion* assigns to every passage the sum, over the rankings to be combined, of $1/(60 + r)$, where r is the position of the passage in each ranking: a passage placed well in both outranks one that appears at the top of only one [131].

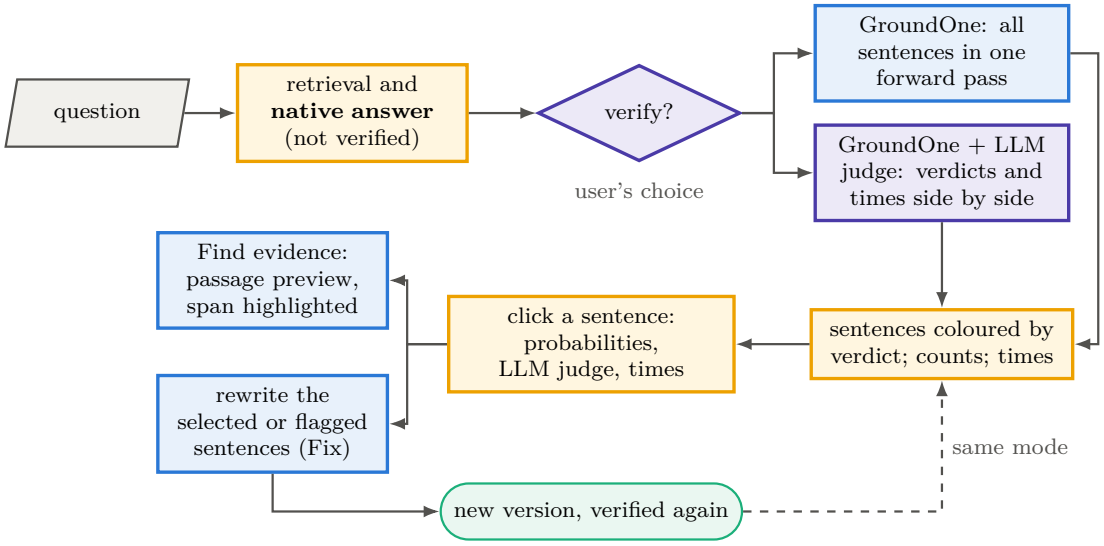


Figure 8.2: The usage flow. The answer appears as the generator wrote it; verification, with GroundOne or with GroundOne and the LLM judge, and rewriting happen only on request, and every rewriting produces a new version that is verified again in the same mode.

and the LLM judge, to compare verdicts and times. The separation is deliberate: by showing the native answer first, the effect of verification remains visible, instead of delivering an already filtered answer straight away. The separation also has a cost advantage. The sources are known before the answer, so, as soon as the generator has finished, GroundOne reads the instructions and the sources into its KV cache in the background while the user reads the answer, and the verification, if requested, reads only the answer. On the 200 answers of the latency measurement, with the 3B the first verification drops at the median from 2.11 to 0.59 seconds (3.6 times), while the read-ahead, 1.41 seconds, uses the GPU when it would otherwise be idle; with the 0.5B the verification drops from 0.35 to 0.10 seconds. If the user asks for the verification before the read-ahead has finished, the verification waits for it instead of repeating it, and costs as much as without read-ahead (2.11 seconds at the median with the 3B). The Python service offers the same mechanism through a dedicated call (`/v1/prefetch`).

GroundOne verification, in both modes, reads the whole answer in a single forward pass: all the sentences receive their verdict together, and an answer with many sentences does not require more passes. After verification the sentences are coloured: light green for supported sentences, red for contradicted ones, orange for those not mentioned by the document; sentences without checkable content remain grey (Figure 8.4). The side panel reports the counts, the threshold τ, the times of

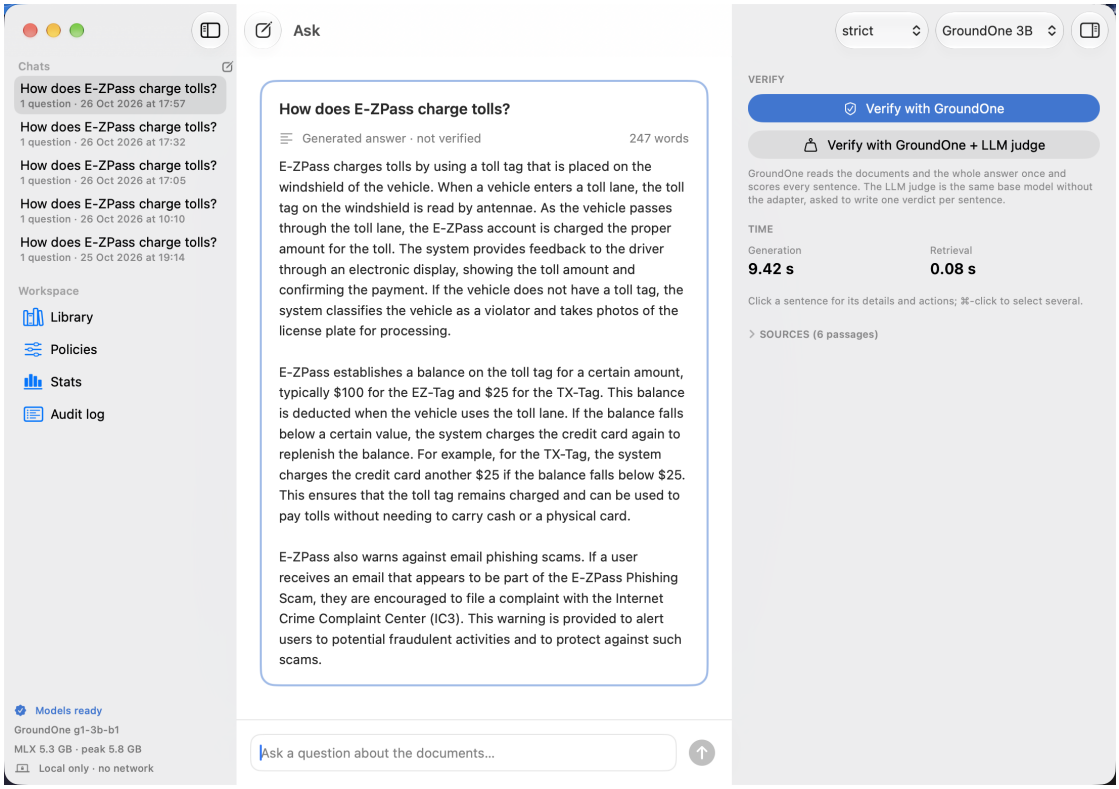


Figure 8.3: The native answer of the generator to the question "How does E-ZPass charge tolls?", before any verification. The two verification modes are on the right.

generation, of verification and, if requested, of the LLM judge, and the sentences on which GroundOne and the LLM judge disagree.

Every sentence can be selected. A click shows, in the panel, the three calibrated probabilities, the verdict of the LLM judge and the times; with the Command key several sentences can be selected (Figure 8.5). From here the user can request the supporting evidence, or the rewriting of the selected sentences. When nothing is selected, a button rewrites all the flagged sentences according to the chosen policy. Every rewriting produces a new version of the answer, verified again in the same mode, and a selector allows the versions to be compared.

Evidence. GroundOne indicates whether a sentence is supported, but not by which passage (Section 9.2). The application reconstructs this information in two steps. In the first, it verifies the answer against each retrieved passage separately, with one forward pass per passage, and picks the one in which the sentence obtains the highest probability of support, or of contradiction if the sentence is contradicted. In the second, within the chosen passage, it locates the sentence closest to the

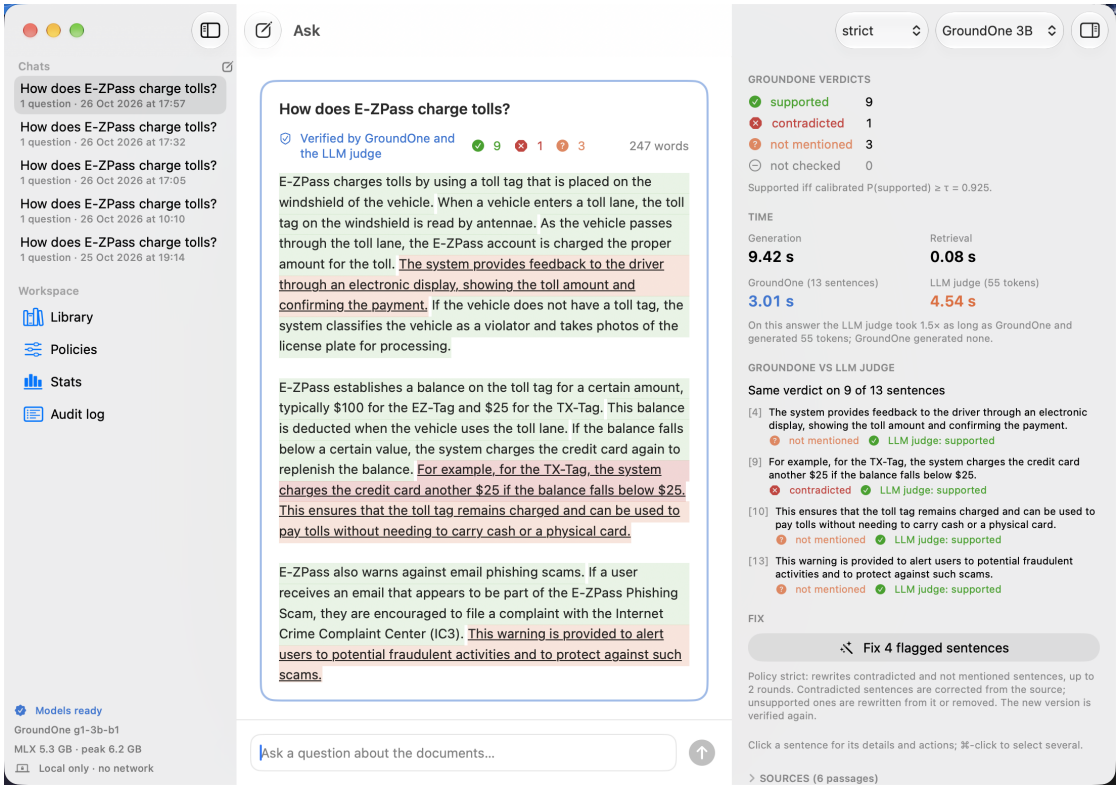


Figure 8.4: The same answer after verification with GroundOne and the LLM judge. The sentences are coloured according to the GroundOne verdict; the panel reports counts, times and disagreements with the LLM judge.

one in the answer, weighting the shared words by their rarity in the passage. The document preview opens with that span highlighted (Figure 8.6). This is the first prototype of the readout per passage listed among the future developments (Section 10.2). It has not been evaluated, because RAGTruth does not indicate which passage supports a sentence, and the application presents it as evidence suggested by GroundOne. The computation runs only on request and is cached: with six passages it costs six forward passes, that is, a few seconds with the 3B, and it does not change the times of the main verification.

Targeted rewriting. Rewriting follows a principle of the verify-and-revise loops that have proved effective in the literature: an external critic, or retrieved evidence, flags the errors and the generator changes only what has been flagged [132, 133], whereas spontaneous self-correction, without an external signal, does not reliably improve the answers [134]. GroundOne is the critic, and a cheap one: verifying again costs a single prefill, and not even a full one. The instructions and the

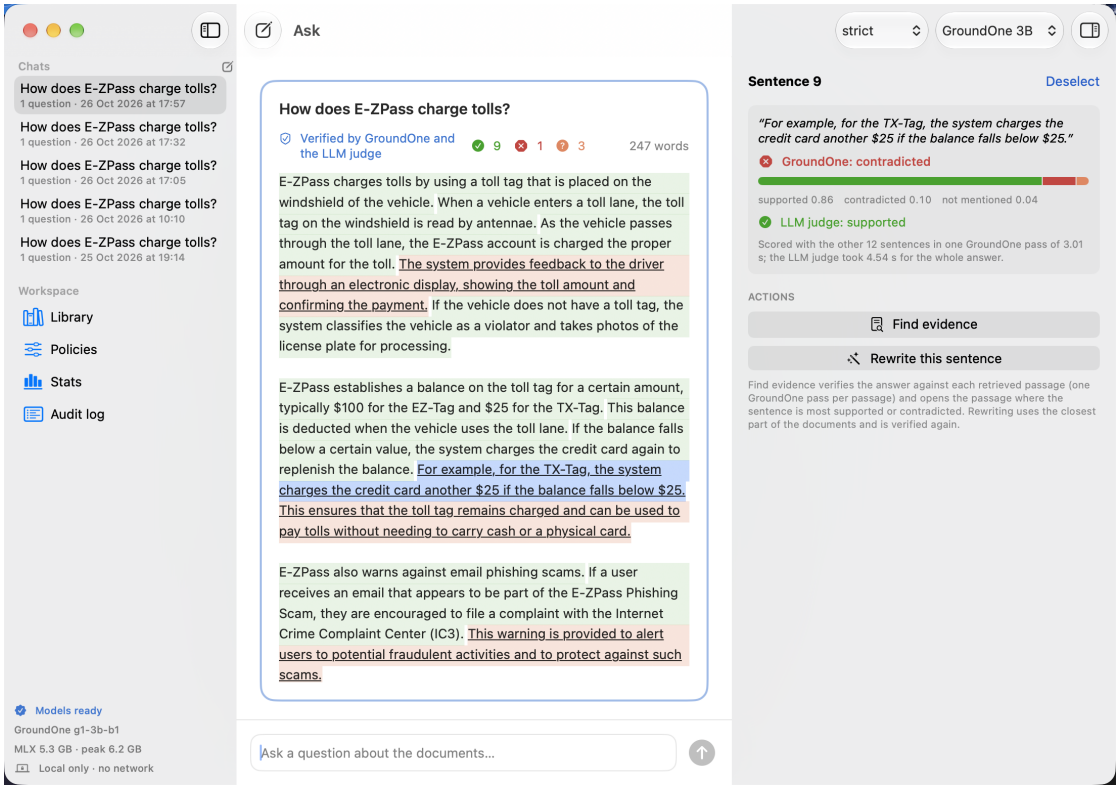


Figure 8.5: The panel of a selected sentence: calibrated probabilities of GroundOne, verdict of the LLM judge, times and available actions.

sources, already read ahead or at the first verification, stay in the KV cache (Section 4.2), so the new version only requires reading the rewritten answer, which in the measurements of Section 7.9.1 makes the verification 3.4 times faster with the 3B. The part of the answer before the first rewritten sentence is also kept, together with the verdicts of its sentences, so the reading restarts at the first change: when the last sentence is rewritten, the verification is about six times faster than with the sources alone (Table 7.4). The LLM judge, when requested, reuses the cache of the sources, but writes all its verdicts again; the timing panel shows how many tokens were reused. For every sentence to be rewritten, the generator receives a short request, with the sentence and only the excerpt of the document that concerns it, and an instruction that depends on the type of error. A contradicted sentence must be corrected with the facts of the source and cannot be deleted: if the generator does not return a usable sentence, the original one stays in place and is resubmitted in the next round. A sentence that is not mentioned must be rewritten with what the source says on the same point, or removed if the source does not address it. In both cases the instruction explicitly asks not to return the sentence unchanged. If

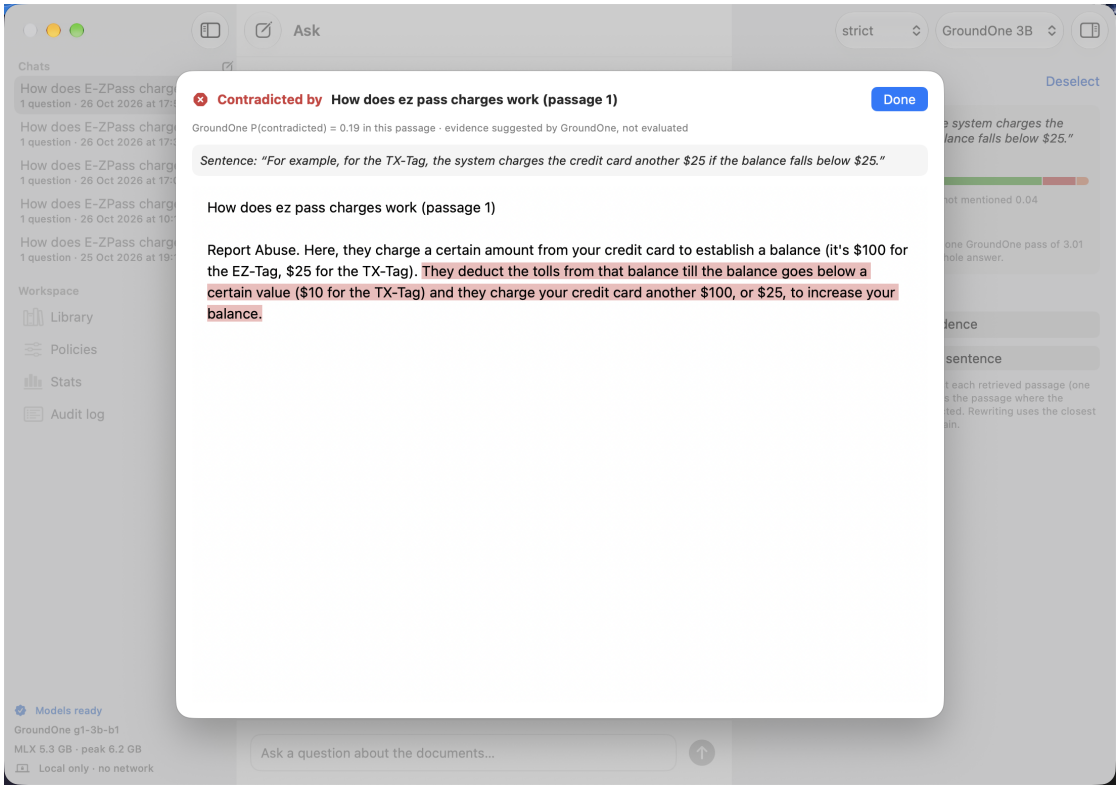


Figure 8.6: The evidence for a sentence: the passage that, according to GroundOne, most clearly contradicts the sentence, with the closest span highlighted.

the generator copies it anyway, which happens with a small model, the request is repeated once, quoting the copy and rejecting it; if the second attempt also returns the same sentence, the sentence stays as it is, flagged, and the panel shows that the generator could not correct it. It is not retried in later rounds, because with greedy decoding the reply would be the same. The other sentences remain unchanged. The maximum number of rounds and the classes to be corrected are set by the policy.

Policy and threshold. A policy is a small JSON file that specifies which verdicts are to be corrected, how many rewriting rounds are allowed, what to do if the error persists (highlight, hide the sentence or abstain) and which threshold to use: the one fitted on the dev set or one chosen by the user. The two default policies correspond to the two readings already introduced in the thesis (Section 4.4): the strict policy corrects both contradicted and not-mentioned sentences, the lenient policy only the former.

Conversations and statistics. Questions are organised into separate conversations, which can be renamed, deleted and resumed at any time. A dedicated section collects the statistics of all the verifications run in the application, including those of previous sessions (Figure 8.7): the time of GroundOne and that of the LLM judge as a function of the number of sentences, the agreement between the two on the same sentences, the distribution of the verdicts and the cost the same verification would have had with a cloud LLM judge. The cost uses the same list prices as Table 9.1: the input tokens are those of the document and the answer plus the instruction; the output tokens are those actually written by the local LLM judge or, if the LLM judge was not run, their mean number per sentence measured in the thesis. The cost of the electricity consumed locally, estimated from the verification time and from the power measured in Section 7.10, is subtracted from it.

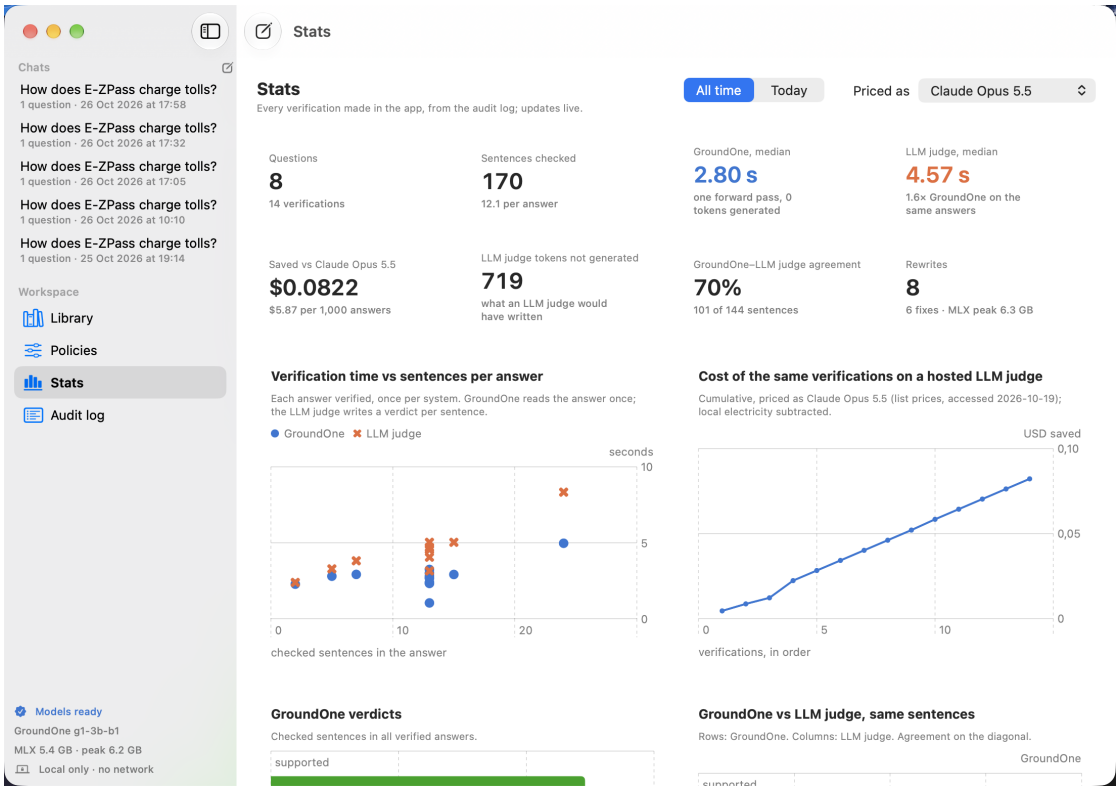


Figure 8.7: The statistics section: key indicators, verification time as a function of the number of sentences for GroundOne and for the LLM judge, and cost of the same verification with a cloud LLM judge.

Persistence. All the data of the application reside on the local disk, in the application support folder provided by macOS, without any database server (Table 8.3). The support folder was preferred to the user's document folders for a practical reason: macOS protects the Desktop and Documents folders and makes access to them conditional on an explicit permission, which an application would have to request at launch.

Table 8.3: Data persistence in the application. Everything resides on the local disk.

Data	Format	Use
Conversations	One JSON file per conversation.	Questions, answer versions, verdicts, times; resumption of conversations.
Action log	Append-only JSONL file.	One line for every generation, verification and rewriting; basis of the statistics and of the browsable log.
Library	SQLite database.	Full-text index for BM25 and passage embeddings, with the origin of each passage.
Policies	JSON files.	Selectable decision and rewriting rules.
KV cache of the sources	In memory only.	Instructions, the last documents already read (also ahead, while the user reads the answer) and the last answer verified, reused by later verifications; not saved to disk.

8.7 Case study: injected errors

Answers generated for free questions have no reference label: measuring on them how much the verify-and-rewrite loop improves the answer would require an independent judgement on every rewritten sentence. The case study therefore uses injected errors, whose position is known in advance and does not depend on GroundOne.

Protocol. From the T1 answers whose reference labels are all "supported" and which GroundOne 3B accepts in full, 100 were drawn from the three tasks: 46 from summarisation, 29 from question answering and 25 from data-to-text. A single error was injected into each, of four types, 25 per type: a number replaced with a value absent from the document, a name replaced with an invented one, a shifted

date, or an invented sentence inserted after an existing one. The answers then go through the same engine as the application: GroundOne verifies the answer, the LLM judge verifies it in turn for the comparison, and the strict policy rewrites the flagged sentences for up to two rounds. An error counts as removed when the wrong value no longer appears in the text, or when no sentence remains similar to the invented one. The case study does not use retrieval: the document is that of T1.

Table 8.4: Case study on injected errors: 100 answers of T1, one error each, GroundOne 3B and the 4-bit Qwen2.5-3B-Instruct generator in the same process, without the embedding model, with the KV cache of the sources active. Detection is computed on the 98 answers in which the sentence with the error receives its own verdict.

Measure	Value
Sentence with the error flagged by GroundOne	97 of 98
Sentence with the error flagged by the LLM judge	84 of 98
False alarms of GroundOne on the other sentences	1.8 %
Answers still containing the error: without the loop, after one and after two rounds	100 %, 8 %, 6 %
Errors removed by restoring the value of the source	42
Errors removed by rewriting the sentence	1
Errors removed by deleting the sentence	51
Remaining errors: still flagged / never flagged	3 / 3
Sentences the generator copies even at the second attempt	3
Other sentences left identical	98.0 %
Answers accepted by GroundOne at the end	94 %
Median time per answer: GroundOne, LLM judge, rewriting per round	1.74 s, 2.36 s, 0.92 s
Memory allocated by MLX: after loading and at peak	5.1 GB, 6.7 GB

Results. Table 8.4 and Figure 8.8 summarise the case study. In 98 answers the sentence with the error receives its own verdict; in the other two, the segmenter merges it with an introductory sentence without checkable content, and the sentence is not scored. GroundOne flags the error in 97 of the 98 answers, the LLM judge in 84. The loop reduces the answers that still contain the error from 100 % to 8 % after one round and to 6 % after two, according to the automatic count. The way in which errors are removed matters as much as their number. For replaced numbers and dates, in most cases the generator restores the value of the source: 19 numbers and 16 dates out of 25. Invented sentences, instead, are deleted, all 25 of

them, which is the appropriate correction. Invented names, finally, are correctly labelled as not mentioned, and for this reason they are mostly removed, 18 out of 25, rather than replaced with the original one, 7 out of 25: the sentence loses the error, but also the information. The copy check intervenes three times: in three sentences the generator returns the same sentence even at the second attempt, and the sentence stays flagged. Of the six answers that keep the error, three remain flagged and are therefore highlighted by the policy. In the other three the error is never flagged: in one, the probability of support, 0.93, is just above the threshold of 0.925; in the other two, the injected sentence does not receive its own verdict.

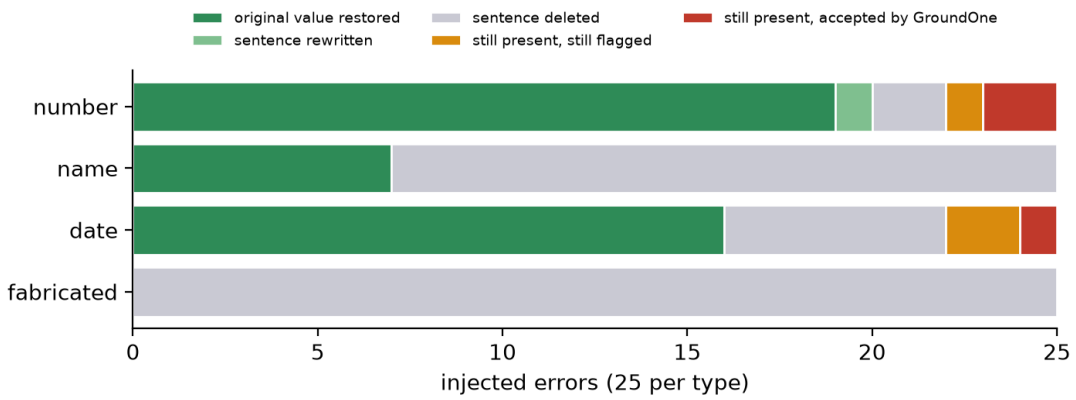


Figure 8.8: Outcome of each injected error after the loop, by error type, with GroundOne 3B: value restored from the source, sentence rewritten, sentence deleted, error still present and flagged, error accepted by GroundOne.

The type of error decides the repair. Repeating the case study with GroundOne 0.5B as the verifier shows why the three-class verdict matters, and not only the flag. The 0.5B flags the error in 97 answers out of 98, like the 3B, but labels almost all the replaced numbers and dates as not mentioned (44 out of 50), whereas the 3B recognises them as contradicted (40 out of 50). For a sentence that is not mentioned, rewriting may remove the sentence, and the generator does so: with the 0.5B, 84 errors are removed by deleting the sentence and only 12 by restoring the value. The answers that keep the error fall to 3 %, but at the price of lost information and of many false alarms: the 0.5B flags 17 % of the other sentences, which are correct, and only 84 % of them remain unchanged. The distinction between a contradicted and a not-mentioned sentence, which Chapter 4 motivates in the abstract, here decides whether the correction restores the information or eliminates it.

Times. The comparison of times confirms the picture of Section 7.9. On these answers, with a median of about 770 tokens between document and answer and of six scored sentences, the median time of the LLM judge is about 1.3 times that of GroundOne, and the LLM judge writes around twenty tokens, whereas GroundOne generates none. The difference is small because the terse LLM judge writes little and the answers are short. The comparison on detection, 97 errors flagged against 84, instead favours GroundOne by construction: the answers were chosen among those that GroundOne accepts in full, a filter that the LLM judge did not pass, and the false alarms of the LLM judge on the correct sentences were not measured.

A comparison at equal compute. On 60 new T1 answers, generated by the model for each task, the complete loop, that is, answer, verification and up to two rounds of targeted rewriting, was compared with a strategy that corrects nothing: generating up to three answers with sampling and keeping the first one that GroundOne accepts. The first answer is accepted in 52 % of the cases. At the end, the loop brings the accepted answers to 70 %, multiple generation to 62 % (42 and 37 answers out of 60, a small difference on this sample). The loop, however, generates half the tokens, 115 against 218 at the median, in a similar time, 6.3 seconds against 7.0: multiple generation also verifies every sample on the same document, and therefore reuses its KV cache. Rewriting only the flagged sentences thus gets at least as many answers accepted as regenerating them from scratch, with half the generation. Part of the answers remains flagged because the 3B generator fails to correct a contradicted sentence, which the loop cannot delete.

A larger generator. The generator can be chosen in the application: besides the 3B, Qwen2.5-7B-Instruct at 4 bits is available. On the same case study the 7B corrects every error that GroundOne flags: the answers that still contain the error fall to 3 %, that is, to the three in which GroundOne does not flag it, against 6 % with the 3B, and those accepted by GroundOne rise to 98 %. The price is time and memory: a rewrite costs 2.1 seconds per round against 0.9, and the peak of memory allocated by MLX rises to 9.2 GB against 6.7. On the twelve example questions of the library, moreover, the 7B writes answers of similar length (276 words at the median against 250) in more than twice the time (17.7 seconds against 7.6), and adds more content that the sources do not report: GroundOne considers 66 % of its sentences supported, against 85 % of those of the 3B. For this reason the 3B remains the default generator.

How to read the results. Three caveats delimit the scope of the case study. The first concerns detection. The answers were chosen among those that GroundOne accepts, and the injected errors are clear-cut: values and names absent from the document. The high share of flagged errors therefore shows how the loop works,

not the recall of GroundOne, which is measured in Chapter 7. The second concerns the rewritten sentences: they have no reference label, and their correctness is judged only by GroundOne, the same model that decides when to stop the loop. A rewriting that falls into a blind spot of the model is therefore counted as successful, and the automatic count records only the disappearance of the injected error, not any new errors introduced by the rewriting. For this reason the results are expressed as answers "accepted by GroundOne", not as correct answers, and the 94 % of accepted answers includes three answers that still contain the error. The third concerns the generator. A 3B model sometimes cannot correct a sentence and sometimes removes it: 26 of the 51 errors removed by deletion also take away correct information, so part of the accepted answers is simply shorter.

8.8 An example: a subtle numerical error

The question of Figure 8.3 shows in miniature both the value and the limits of the system. Among the thirteen sentences of the answer, the generator writes that, for the TX-Tag card, the credit card is topped up by 25 dollars when the balance falls below 25 dollars. The retrieved passage instead says that the top-up happens when the balance falls below 10 dollars. The sentence follows the source almost word for word, and it is precisely the type of error that Section 7.12 identifies as the hardest for GroundOne. GroundOne assigns it a probability of support of 0.86, below the threshold of 0.925, and indicates contradiction as the most likely error; the generative LLM judge considers it supported. GroundOne also flags as not mentioned three sentences that add details or inferences absent from the documents, for example that the display shows the toll amount, whereas the LLM judge accepts them all. Rewriting, however, fails: the 3B generator returns the same sentence, with the same wrong amount, even at the second attempt, which quotes the copy and rejects it. The application detects this and leaves the sentence flagged: the quality of the loop also depends on the generator.

8.9 Calibrated probabilities and decision threshold

The example helps to clarify a point that using the application makes evident. The probabilities of GroundOne are calibrated: on T1, among the sentences to which the 3B assigns a probability of support between 0.80 and 0.925, 93 % are actually supported. Yet all these sentences are flagged, because the threshold is 0.925. The two facts are not in contradiction. Calibration concerns the meaning of the probabilities; the threshold is a decision rule, chosen on the dev set to maximise the three-way balanced accuracy (Section 4.4). Since the great majority of sentences

are supported, about 88 % in the dev set and 91 % in T1, a metric that weighs the three classes equally pushes the threshold upwards: to find the rare errors, it accepts many false alarms (Table 8.5).

Table 8.5: Effect of the threshold on the T1 sentences with GroundOne 3B: share of flagged sentences and recall per class.

τ	Flagged sentences	Recall supported	Recall contradicted	Recall not mentioned	Balanced accuracy
0.5	7 %	0.98	0.56	0.60	0.71
0.8	15 %	0.92	0.71	0.83	0.82
0.925	25 %	0.82	0.78	0.89	0.83

Moving from 0.8 to 0.925 adds only 0.01 of balanced accuracy, but raises the flagged sentences from 15 % to 25 %, when the true errors in T1 are 9 %. On the other hand, the error on the TX-Tag card lies precisely in this band: with a threshold of 0.8 it would have gone unnoticed. The case study offers the mirror example: the only error that GroundOne lets through has a probability of support of 0.93, just above the threshold. Neither choice is wrong. A high threshold is preferable when an unflagged error is costly, a lower one when false alarms disturb the user. This is the practical advantage of calibrated probabilities: the threshold can be chosen knowingly, and the application exposes it as a parameter of the policy. The label shown for a sentence below the threshold should be read accordingly: it indicates the most likely type of error, not a certainty, which is why the panel always reports the three probabilities.

8.10 Limitations

The generator. The 3B generator writes answers shorter than requested, between about 120 and 250 words against the 300–500 indicated in the prompt, and does not always manage to correct a flagged sentence: in the example of Section 8.8 it returns the sentence unchanged. A larger generator, for example a 4-bit 7B, would still fit in the memory of the laptop and should improve both aspects, but it has not been evaluated.

The rewritten sentences. The correctness of the rewritten sentences is judged only by GroundOne. An independent check, with a second verifier such as a specialised encoder or with a human reading of a sample, would make it possible to measure how many accepted rewritings are actually correct.

The evidence. The suggested evidence has not been evaluated, because the data do not indicate which passage supports each sentence. It is a demonstration feature.

Scope. The application inherits the limits of GroundOne (Section 9.2): it verifies documents and answers in English, in the domain of the training data, and it relies on the retrieved documents, not on the truth of the world. Questions that concern the whole library, for example an overview of the documents, are beyond the reach of a system that retrieves six passages. The interface, finally, runs only on macOS, and the 3B generator and verifier are subject to the Qwen research licence (Section 6.6): the application is a research prototype, not a product.

Chapter 9

Discussion, Limitations and Threats to Validity

This chapter interprets the results in the light of the five research questions. It then states what the experiments cannot show and which choices could have biased them.

9.1 When a specialist decision model pays off

Specialisation (RQ1). What makes GroundOne accurate is training on the decision. The untrained models remain close to chance, whereas the same models, once trained, reach a three-way balanced accuracy of 0.769 (0.5B) and 0.832 (3B) on the RAGTruth test set. They thus outperform the specialised encoders in AUROC, and Rizzo-Flow, a general-purpose decision model eight times larger than the 0.5B, in three-way balanced accuracy. The advantage, however, is concentrated where the training data are. On LLM-AggreFact the 3B draws level with the best small encoder without surpassing it, and remains below the closed decision model Jev and a 27B open model, both measured independently; the 0.5B stays below the encoder. On VitaminC, finally, the trained 3B does no better than the untrained one.

The comparison with d1-3B, a general-purpose decision model of the same size, makes the trade-off explicit. d1-3B is already trained to decide, but on a broad mixture of tasks. On RAGTruth it is far behind GroundOne, especially on long answers; on LLM-AggreFact it is on par with the 3B; on VitaminC it is clearly ahead. A specialist is therefore a solid choice for the domain it was trained on, but a weak tool for fact checking in general. Within the domain, specialisation buys accuracy and robustness on long answers at the cost of transfer; outside the domain, it is model size or a broad training mixture that improves transfer.

Readout and generation compared (RQ2). The decision-model mechanism contributes in two distinct ways. The first concerns cost: a single forward pass over the answer replaces the generation of K verdicts, so the time saved grows with the length of the answer. When the answer contains fourteen or more sentences, the saving reaches a factor of about two against a terse LLM judge and about six against a verbose one. This overall ratio is bounded by reading the text, which every approach pays and which takes 97 % of GroundOne's time: net of it, GroundOne decides 18 to 33 times faster than a terse LLM judge and about 125 to 130 times faster than a verbose one (Section 7.9.1). It follows that any reduction of the prefill, such as reusing a prefix already read, increases the relative advantage of the readout. With the document already cached, as when a rewritten answer is verified again, the advantage over the terse LLM judge, with fourteen or more sentences, rises from 1.7–2 to 3.4–3.6 times: the cache removes the shared part and leaves the judge with the decoding, which cannot be reused. Keeping the previous version of the answer as well, GroundOne reads again only what follows the first rewritten sentence and reuses the verdicts of the earlier ones, while the judge must write them all again: over all answers the advantage over the terse LLM judge rises from 4.8 to 12 times with the 0.5B and from 3.1 to 7.1 times with the 3B (Table 7.4). The second contribution, which this thesis did not anticipate, concerns accuracy. With the same weights and the same cost, choosing among the three admissible letters is more accurate than taking the token the model would generate, with a gain in three-way balanced accuracy of 0.25 for the 0.5B and 0.07 for the 3B. A fine-tuned decision model is therefore better understood as a three-way classifier grafted onto a language model than as a generator that stops after one token. The distinction matters most for small models, that is, precisely for those a laptop can afford. The comparison with LLM judges, finally, also measures the effect of training: since the LLM judges are untrained, the large differences in accuracy with respect to GroundOne should be read as the joint effect of specialisation and readout. Figure 9.1 decomposes the gain on T1 into its two steps, training and the restricted readout.

Calibration (RQ3). A temperature and a bias fitted on the dev set align the probabilities with the frequencies observed on the test set of the same domain, and the calibrated probabilities make selective prediction effective. Two lessons hold beyond GroundOne. The first is that the Expected Calibration Error alone is not enough to rank systems. After the same calibration, LLM judges whose probabilities barely separate the classes reach the same ECE as GroundOne, and only the AUROC and the AURC reveal the difference. The second is that decisions and probabilities must be derived consistently. If the most probable calibrated class is chosen, the decision collapses towards the majority class; a threshold fitted on the dev set, instead, recovers the balanced decision and keeps the label consistent

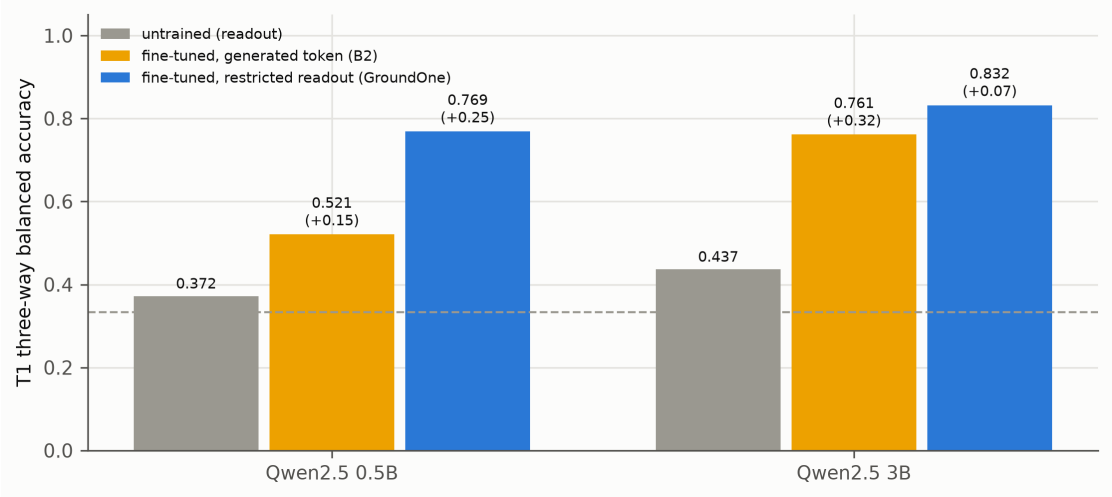


Figure 9.1: Where the accuracy of GroundOne on T1 comes from: the same base model untrained (readout, design a), trained but with the decision taken from the token it would generate (control B2, format failures counted as errors) and trained with the restricted readout. The gain over the previous step is shown in brackets. The untrained 3B is evaluated on 600 answers; the dashed line marks the chance level.

with the probability shown to the user. As the results on VitaminC show, finally, calibration holds only for the distribution on which it was fitted.

Cost on the device (RQ4). The verifier fits comfortably in the 16 GB laptop. During training the memory in use peaks at 2.6 GB with the 0.5B and at 5.9 GB with the 3B. Verifying a long answer takes 0.41 seconds with the 0.5B and 2.57 with the 3B, and the energy is on average 5.5 and 33 joules per answer. The 0.5B fits an interactive budget; the 3B, instead, is slower than continuous interactive use would require, unless verification runs on request, as in the application of Chapter 8, and it trades time for accuracy. The energy measurements qualify the speed results. A single long prefill draws more power than token-by-token decoding, so the energy saved is smaller than the time saved, and the higher speed should not be read as an equal saving of battery. The verifier also coexists with a generator in the same process: together with a 4-bit 3B generator, the peak memory is 6.1 GB with the 3B and 3.3 GB with the 0.5B. The whole evaluation runs without any network access, with results identical bit for bit, and the 8-bit weights cost no accuracy compared with bf16 ones (Section 7.11). The retriever adds little: in the application of Chapter 8, with the generator, GroundOne 3B and the embedding model loaded together, the memory allocated by MLX peaks at 6.2 GB during one

question and at 6.9 GB after a session of twelve questions, with the KV cache of the sources already read.

The economic cost should also be read with care (Table 9.1). Verifying 10,000 answers with GroundOne takes an amount of energy that, at the European average household electricity price [135], is worth less than one euro cent with the 0.5B and about three cents with the 3B. Entrusting the same verification to a hosted LLM judge, at the providers' list prices [136, 137, 138], costs instead from about one dollar with the cheapest models to more than two hundred with the top-tier ones. The ratio is thus two to four orders of magnitude, but in absolute terms cheap LLM judges remain inexpensive, and the comparison takes into account neither the purchase of the hardware nor the quality of hosted LLM judges, which this thesis did not measure. The decisive argument for local verification therefore remains independence: the documents never leave the machine and the service does not depend on an external provider.

Table 9.1: Indicative cost of verifying 10,000 RAG answers. Hosted LLM judges: standard-tier list prices, accessed on 19 October 2026, with about 900 input tokens per answer (median T1 document and answer plus the instructions) and the output tokens measured for the 3B LLM judges of this thesis (36 for the terse LLM judge, 248 for the verbose one). GroundOne: measured net chip energy multiplied by the EU average household electricity price (0.29 €/kWh, second half of 2025). Hardware cost and caching or batch discounts are excluded.

System	Input / output price (USD per Mtoken)	Terse judge	Verbose judge
Claude Haiku 5.5, gpt-6-luna	0.10 / 0.50	1.1 USD	2.2 USD
Gemini 3.5 Flash-Lite	0.30 / 2.50	3.6 USD	8.9 USD
Gemini 3.8 Flash	0.75 / 3.75	8.2 USD	16 USD
Claude Sonnet 5.5, gpt-6.1-sol, Gemini 4 Argon	2.00 / 10.00	22 USD	43 USD
Claude Opus 5.5	4.00 / 20.00	43 USD	86 USD
gpt-6-astra	10.00 / 50.00	109 USD	215 USD
GroundOne 0.5B (local)	0.015 kWh, about 0.004 €		
GroundOne 3B (local)	0.092 kWh, about 0.03 €		

Beyond a binary score (RQ5). The three-way output separates contradicted from not-mentioned sentences well in data-to-text, the task in which RAGTruth concentrates its contradictions. It separates them much less in question answering and summarisation, where contradictions are rare in the training data. The 3B

improves considerably on this front, which suggests that what the small model lacks is more varied contradictions in training rather than a different architecture. The strict and lenient policies and the threshold are all derived from the same three probabilities. The user can therefore change the notion of error without retraining the model, which no binary verifier allows. Figure 9.2 summarises these indications as a decision tree.

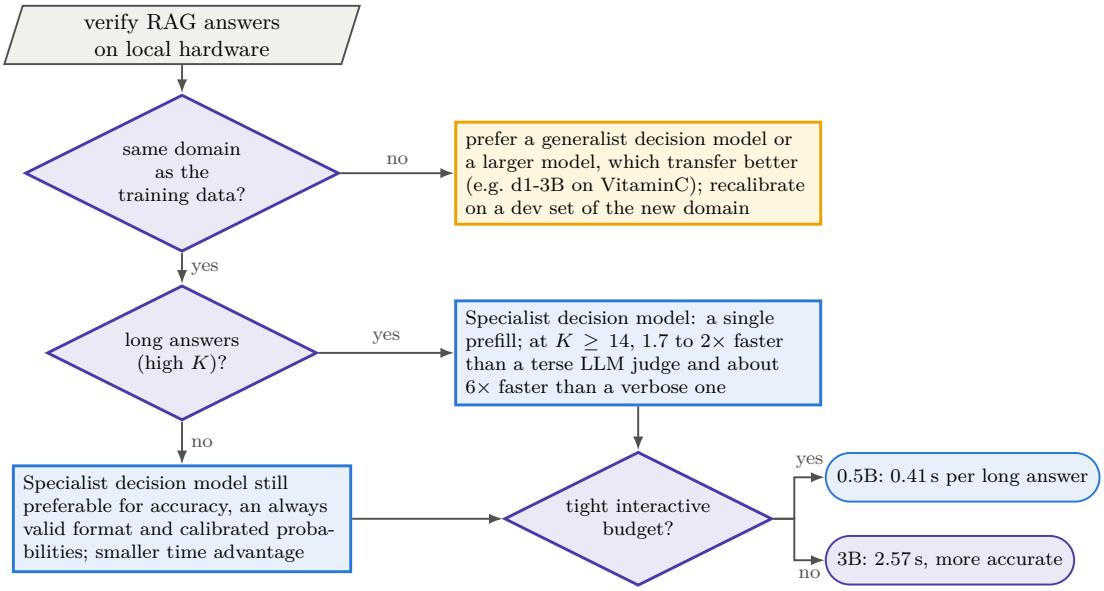


Figure 9.2: When a specialist decision model pays off, according to the results of this thesis. The summary is qualitative: the times refer to the M4 laptop and to answers with 14 or more sentences, and the out-of-domain transfer is the one observed on LLM-AggreFact and VitaminC.

9.2 Limitations

One platform and one family of models. Every result comes from a single laptop, a single runtime and a single model family. Latency and energy depend on the hardware and on the implementation: what should hold elsewhere are the ratios between methods, not the absolute values. The latency of GroundOne is compared neither with that of the encoders nor with that of the general-purpose decision models, because these ran in different runtimes and under conditions that would make the comparison unfair (see also the paragraph “One machine” in Section 9.3).

Training stability. Two completed runs of the 0.5B agree on all aggregate metrics, but a third run collapsed in its early stages and was stopped. The training recipe has no gradient clipping and uses AdamW without bias correction, two known causes of unstable fine-tuning [99, 98]. Either could explain the collapse, but neither hypothesis was tested. The 3B, moreover, was trained only once, on half of the training answers, and its variability across seeds remains unknown.

Training data. RAGTruth is the only training source. Its contradictions are concentrated in data-to-text and, outside that task, are learned only in part. The models, moreover, rely partly on lexical overlap. When support is expressed in different words, the sentence looks unsupported; when a sentence copies the source except for a small change, the change often goes unnoticed. This blind spot shrinks as the model grows, but it remains the main limitation of both.

Scope of the decision. Verification is carried out sentence by sentence and only against the retrieved document. A sentence that is true in the world but absent from the document therefore counts as not mentioned: this is the right behaviour for groundedness, but not for fact checking. Sentences without checkable content are excluded by a rule, and the segmentation itself can split or merge claims differently from a human reader. Finally, the verdict on a sentence depends only on the document and on the sentences that precede it.

Language. All training and test data are in English, so GroundOne was trained, calibrated and measured only on English documents and answers. The base model Qwen2.5 is multilingual [23], but it was not measured how far the learned specialisation transfers to other languages, nor whether the calibration remains valid: the calibration fitted on RAGTruth does not transfer even to another English domain such as VitaminC. Use with documents or answers in another language, or in different languages from each other, therefore requires a new evaluation.

Contexts made of several documents. When the answer is written from several retrieved passages, GroundOne reads them concatenated as a single document, as in RAGTruth question answering. Three limits remain, however. The first concerns length: in training, document and answer never exceed 2,048 tokens, so a longer context falls outside the distribution on which the model was measured and must be split into chunks, keeping for each sentence the highest support probability, as on LLM-AggreFact; a fact that requires combining two different chunks may then be missed. The second concerns conflicting sources: when one passage supports a sentence and another contradicts it, the expected verdict is not uniquely defined, and RAGTruth contains no such cases. The third concerns

attribution: GroundOne states whether a sentence is supported, but not by which passage; a first, unevaluated prototype is described in Section 8.6.

Statistical scope. The bootstrap intervals describe the variability of the test set, not that of training. Moreover, about sixty comparisons were tested without correction for multiple testing. For this reason the conclusions rest on effects far from zero, and differences on LLM-AggreFact and VitaminC smaller than the spread between seeds are not claimed. Several baselines, finally, were evaluated on subsamples, which widens their intervals.

9.3 Threats to validity

Labels. The distinction between contradicted and not-mentioned sentences rests on the RAGTruth annotators, whose agreement on the type of hallucination is not published. In addition, a recent re-annotation of the RAGTruth test split found considerably more hallucinated answers than the original labels indicate [116]. The test set may therefore under-count the minority classes. For this reason the three-way results on RAGTruth should always be read together with those on VitaminC, whose labels have a measured inter-annotator agreement. LLM-AggreFact, finally, unifies the labels of eleven datasets of different quality, and some of its columns are known to be noisy.

In-domain advantage. GroundOne is trained on RAGTruth. Its results on the RAGTruth test set and on the RAGTruth column of LLM-AggreFact therefore favour it with respect to systems trained elsewhere. Conversely, VitaminC is part of the data from which the training of MiniCheck-RoBERTa descends. Each comparison is reported with the relevant caveat, and the conclusions rest on the overall pattern across the three test sets rather than on a single one.

Baseline configuration. LLM judges are sensitive to their prompts. Their prompts were fixed before looking at the test results, and the prompt of the equal-output LLM judge was chosen on the dev set on the basis of format validity only. Output parsing was lenient, to the LLM judges' advantage. Decoding was not constrained by a grammar, which mostly affects the format failures of the smallest model. Neither an LLM judge fine-tuned on the same data nor a larger LLM judge was evaluated: the comparison therefore measures what an LLM judge of the same size offers without training. Rizzo-Flow was run with 4-bit weights, because the 8-bit model did not fit in memory beside the operating system, and this may understate its accuracy. d1-3B, finally, was run zero-shot and, on LLM-AggreFact,

only on a subsample. It received the questions used for all decision-model baselines, which may not be the phrasing that suits it best.

Contamination. The base models were pretrained on large web corpora that may include parts of the public test sets[1]. The same base models, however, used untrained as baselines, reach near-chance accuracy on the three-way task. Contamination therefore cannot explain the gains due to training; for the absolute values, instead, it cannot be ruled out entirely.

One machine. All measurements of time and energy come from a single laptop, an Apple M4 with 16 GB, and a single runtime, MLX. The cost model of Chapter 2 predicts that the advantage of a decision model holds on every class of local hardware, because it rests on the difference between prefill and decoding, which vendors and independent studies observe everywhere. The lower bounds on the cost of decoding computed in that chapter for other machines also follow from published specifications. The actual speed-ups on a discrete graphics card, a DGX Spark or a smartphone, however, were not measured. Their size depends on the balance between computation and bandwidth of each machine: the cost model supports the direction of the advantage, not its magnitude.

[1]Data *contamination* occurs when examples from a test set appear, even only in part, in the pretraining data of a model. A contaminated model may remember the examples instead of judging them, and its scores on that test set are then too optimistic.

Chapter 10

Conclusions and Future Work

10.1 Summary of findings

This thesis asked whether a compact language model, specialised in a single decision and used as a decision model, can verify the answers of a RAG system sentence by sentence on a laptop, and where any advantage comes from. GroundOne, a fine-tuned Qwen2.5 model that reads the verdicts of a whole answer in a single forward pass, gives a positive answer, albeit with some reservations.

Specialisation works where it was aimed. On the RAGTruth test set GroundOne reaches a three-way balanced accuracy of 0.769 with 0.5B parameters and 0.832 with 3B. It is far above the same models untrained, ranks supported sentences better than the specialised encoders and is clearly above, in three-way balanced accuracy, Rizzo-Flow, a general-purpose decision model eight times larger than the 0.5B. On LLM-AggreFact the 3B is on par with the best small encoder. On VitaminC, instead, the gain disappears: a specialist is not a general fact-checking tool. d1-3B, a general-purpose decision model of the same size trained to decide on a broad mixture of tasks, confirms the trade-off: GroundOne is far ahead of it on RAGTruth, with the 3B it matches d1-3B on LLM-AggreFact, and it is far behind on VitaminC.

The decision-model mechanism pays off on two fronts. The first is speed. Reading all the verdicts in a single forward pass makes the verification of long answers about two times faster than that of a terse LLM judge and six times faster than that of a verbose LLM judge of the same size, with output that is always valid. The second is accuracy: restricting the decision to the admissible answers is, by itself, more accurate than letting the same model generate the verdict. Moreover, even when asked for the same output, LLM judges cannot provide probabilities of comparable

quality. Their probabilities barely separate correct from incorrect decisions, and the verbalised ones are almost always all-or-nothing.

The probabilities of GroundOne can be calibrated with a temperature and a bias fitted on the dev set. A threshold chosen on the same data then turns them into labels consistent both with the probabilities and with the evaluated decision. The Expected Calibration Error alone, however, does not distinguish useful probabilities from useless ones: discrimination must be measured separately.

The whole system runs on a 16 GB laptop. A long answer is verified in 0.41 seconds with the 0.5B and in 2.57 with the 3B, with an energy of 5.5 and 33 joules on average per answer; when the document is already in the KV cache, as in the verification of a rewritten answer, 0.11 and 0.58 seconds suffice. Compared with an LLM judge the energy saving is real, but smaller than the time saving, because a single long prefill draws more power than decoding.

The GroundOne Studio application shows that a complete RAG assistant can be built around the verifier, in a single process and without network access: retrieval, generation, verification and comparison with an LLM judge require at most 6.9 GB of memory allocated by MLX. In the injected-error case study GroundOne flags the error in 97 of the 98 answers in which the sentence receives a verdict, against 84 for the LLM judge, and targeted rewriting reduces the answers that still contain the error from 100 % to 6 %, according to the automatic count: numbers and dates are mostly restored from the source, fabricated sentences deleted. The case study also shows that the type of error matters: a contradicted sentence is corrected, a not-mentioned one often merely removed. The answers that pass the loop are accepted by GroundOne, not independently verified.

The main limitations are two. The first is the narrowness of the training data, because of which contradictions outside data-to-text and hallucinations hidden in copied sentences are learned only in part. The second is a training recipe that is not fully robust to the choice of the seed.

10.2 Future work

Several lines of work follow directly from these limitations. The training recipe should be made robust by enabling bias correction and gradient clipping, logging the gradient norms and repeating the runs over several seeds, including for the 3B. Training data with more varied contradictions, with paraphrased support and with minimal edits of the kind VitaminC contains would then be needed: they should address both the low recall outside data-to-text and the blind spot on copied sentences. Finally, training the readout on the letter that the model would spontaneously write after the marker could align readout and generation, with effects on accuracy that remain to be measured.

On the systems side, the cost model of Chapter 2 should be verified on other hardware. A port of the readout to llama.cpp would allow GroundOne to be measured on a discrete graphics card, on a DGX Spark or an AMD Strix Halo machine, and on a smartphone, whose neural processing units accelerate precisely the prefill on which a decision model relies. Latency comparisons with the other families of verifiers also require fair conditions: the encoders must be run in batches, while GroundOne and the general-purpose decision models must be queried through the same HTTP interface. One change would also shorten the reading of the text, which is now almost all of GroundOne's time: fusing the LoRA adapter into the base weights, which would remove the roughly 10 % of extra computation measured in the prefill, at the price of a new quantisation and a new calibration.

Another direction concerns language. The first step is to measure how well the adapter trained in English already works on documents and answers in another language, for example by translating part of T1 into Italian. If the transfer is not sufficient, a model this small may be better served by a solution other than a single multilingual adapter: a twin adapter for each language, trained separately on the same base, so that the languages do not have to share the little capacity available. Adapters take up little space and can be loaded and unloaded at runtime on top of a single copy of the base model, with a language identifier choosing the right one. Three problems remain to be solved: labelled data are needed in every language, because translating RAGTruth breaks the alignment of the spans and requires the translated labels to be checked; temperature, bias and threshold must be fitted again for each language; and the frequent case of a document in one language and an answer in another falls within no single adapter. The comparison between per-language adapters and a multilingual adapter is therefore an experiment to be run, not a conclusion.

For contexts made of several documents, a natural extension is attribution: reading, besides the verdict, which passage supports or contradicts each sentence, for example with a readout per passage, so as to show the user the source of every claim and to handle conflicting sources explicitly.

The application of Chapter 8 contains a first prototype of this attribution, which remains to be evaluated with data indicating which passage supports each sentence. Three further directions follow from the application. A larger generator, such as the 7B at 4 bits already selectable in the application, corrects the flagged sentences better, but is slower and adds more content that the sources do not report: choosing the generator remains a trade-off to be studied on real questions. The rewritten sentences should be checked independently, with a second verifier or with a human reading of a sample, because their correctness is currently judged by GroundOne alone. Verification, finally, could proceed while the answer is being generated, flagging each sentence as soon as it is complete: the marker-based design and prefix reuse make this use natural, because the document can be read while the generator

writes, and at the end of generation only the last sentence remains to be read.

Appendix A

Output of every system on one answer

This appendix shows what each system returns for the same answer of T1, so that the numbers of Chapter 7 can be related to concrete outputs. The answer (identifier `rt-7784`) is a data-to-text answer written by a language model from a structured description of a restaurant: a JSON record with its name, address, categories, opening hours (Tuesday to Sunday, 11:00–21:00), attributes, an average rating of 4.5 stars and two customer reviews. It was chosen because every system scored it and because it contains all three classes; it was not selected for the performance of any system. Its six sentences and their reference labels are:

1. *Mesa Verde is a popular business located in Santa Barbara, CA.* — supported.

2. *It offers a variety of services including Garage Door Services, Restaurants, Live/Raw Food, . . . , Vegan, and Salad.* — supported.

3. *The business is open seven days a week from 11 am to 9 pm.* — contradicted (it is closed on Mondays).

4. *Mesa Verde has a rating of 4.5 stars on Google and has received multiple positive reviews.* — not mentioned (the record gives no source for the rating and contains two reviews, one of them negative).

5. *One customer praised the restaurant's fresh, delicious, and inventive cuisine, while another was disappointed by the taste and price of the food.* — supported.

6. *Despite this, many customers have had positive experiences at Mesa Verde and recommend it for its fusion and vegan food options.* — supported.

A.1 Verdicts sentence by sentence

Table A.1 lists the verdict of every system. For GroundOne the label is the served one, derived from the calibrated probabilities with the threshold fitted on development data ($\tau = 0.935$ for the 0.5B and 0.925 for the 3B), and the number is the calibrated P(supported). For the encoders the number is their probability of support and the decision is taken at 0.5. S, C and N stand for supported, contradicted and not mentioned; a dash marks a format failure.

Table A.1: Verdicts on answer `rt-7784`. Errors with respect to the reference are in bold.

System	1	2	3	4	5	6
Reference	S	S	C	N	S	S
GroundOne 0.5B (served, P_{sup})	S 0.96	S 0.98	C 0.50	N 0.05	S 0.98	S 0.97
GroundOne 3B (served, P_{sup})	S 0.96	S 0.99	C 0.58	N 0.21	S 0.99	**N 0.85**
B2 0.5B (generated token)	S	S	C	N	S	–
B2 3B (generated token)	S	S	C	–	S	S
Untrained 0.5B (readout)	**C**	**C**	C	**C**	**C**	**C**
Terse LLM judge 0.5B	S	S	**S**	**S**	S	S
Terse LLM judge 3B	**C**	S	**S**	**S**	**N**	S
Verbose LLM judge 0.5B	–	–	–	–	–	–
Verbose LLM judge 3B	–	–	–	–	–	–
Equal-output LLM judge 3B, logits	**C**	S	**S**	**S**	**N**	S
Equal-output LLM judge 3B, verbalised	S	**C**	**N**	**S**	**C**	**N**
MiniCheck-RoBERTa (P_{sup})	0.78	0.90	0.09	0.03	**0.43**	**0.03**
HHEM-2.1-open (P_{sup})	0.75	0.95	0.01	0.09	0.73	**0.24**
Rizzo-Flow 4B	S	S	**S**	**S**	S	S

The 0.5B GroundOne gets all six sentences right. The contradiction in sentence 3 is caught with a calibrated P(supported) of 0.50: well below the threshold, but far from certain, which is what a calibrated probability should express for a sentence that is correct except for one day of the week. The 3B GroundOne also catches both errors but serves sentence 6, a generalisation over the two reviews, as not mentioned: its P(supported) of 0.85 is below the high threshold that the base rate of RAGTruth imposes, a case where a user who prefers fewer flags would lower the threshold. With the same weights, the generated token (B2) gives the right letter on five of the six sentences, but on the remaining one it produces a space (0.5B) or the word “We” (3B) instead of a verdict. The binary encoders separate the two errors well but also reject sentences 5 or 6, which summarise the reviews in different words; Rizzo-Flow accepts every sentence. The LLM judges fail in the ways that Chapter 7 quantifies, as the raw outputs below show.

A.2 Raw outputs

GroundOne 0.5B (served JSON, abridged). One forward pass over 1,193 input tokens, no generated token, 0.47 seconds (2.6 seconds for the 3B); in the JSON the sentences are numbered from zero:


```
{"design": "b", "threshold": 0.935, "n_decisions": 6,
 "usage": {"input_tokens": 1193, "output_tokens": 0},
 "sentences": [
  {"index": 2, "text": "The business is open seven days a week ...",
   "scored": true, "labels": {"three": "contradicted"},
   "answers": {"three": {"supported": 0.502, "contradicted": 0.462,
                         "not_mentioned": 0.036}},
   "flag_strict": true, "flag_lenient": true},
  {"index": 3, "text": "Mesa Verde has a rating of 4.5 stars ...",
   "scored": true, "labels": {"three": "not_mentioned"},
   "answers": {"three": {"supported": 0.051, "contradicted": 0.003,
                         "not_mentioned": 0.946}},
   "flag_strict": true, "flag_lenient": false}, ...]}
```


Terse judge 3B. One line per sentence, as requested, but with the wrong letters for four sentences:

```
1: B
2: A
3: A
4: A
5: C
6: A
```

Verbose judge 3B (first three verdicts). The reasoning restates the sentence and cites the document as support even for the contradicted and the unsupported sentence, and every verdict is "A". The verdicts are written on a separate line instead of on the same line, as the instruction asked, so the protocol counts them as format failures:

```
1: Mesa Verde is located in Santa Barbara, CA, as stated in the document.
Verdict: A

2: Mesa Verde offers a variety of services including Garage Door Services,
   [...], as mentioned in the categories provided.
Verdict: A

3: The business is open seven days a week from 11 am to 9 pm, as indicated
   by the hours provided.
Verdict: A
```

Verbose judge 0.5B. It copies the six sentences back, numbered, without any verdict.

Equal-output judge 3B, verbalised. The probabilities are all one-hot, and for sentences 3 and 6 the stated label (C) contradicts the stated probabilities, which put all the mass on A:

```
{"1": {"label": "A", "p": [1.0, 0.0, 0.0]},
 "2": {"label": "B", "p": [0.0, 1.0, 0.0]},
 "3": {"label": "C", "p": [1.0, 0.0, 0.0]},
 "4": {"label": "A", "p": [1.0, 0.0, 0.0]},
 "5": {"label": "B", "p": [0.0, 1.0, 0.0]},
 "6": {"label": "C", "p": [1.0, 0.0, 0.0]}}
```

Equal-output judge 0.5B, verbalised. The same vector for every sentence:

```
{"1": {"label": "A", "p": [0.3, 0.3, 0.3]},
 "2": {"label": "A", "p": [0.3, 0.3, 0.3]},
 ...
 "6": {"label": "A", "p": [0.3, 0.3, 0.3]}}
```

Bibliography

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020 (cit. on pp. 1, 35).

[2] Georgi Gerganov and ggml-org contributors. *llama.cpp: LLM inference in C/C++*. 2023. URL: `https://github.com/ggml-org/llama.cpp` (cit. on pp. 1, 26, 29, 34, 68).

[3] Apple. *MLX: An array framework for Apple silicon*. 2023. URL: `https://github.com/ml-explore/mlx` (cit. on pp. 1, 26, 51).

[4] Tom Gunter, Zirui Wang, Chong Wang, Ruoming Pang, et al. *Apple Intelligence Foundation Language Models*. 2024. arXiv: `2407.21075` (cit. on pp. 1, 13, 17).

[5] Venkat Somala and Luke Emberson. *Frontier AI performance becomes accessible on consumer hardware within a year*. Epoch AI. Aug. 15, 2025. URL: `https://epoch.ai/data-insights/consumer-gpu-model-gap` (visited on 2026-10-11) (cit. on pp. 1, 17).

[6] Stanford Institute for Human-Centered Artificial Intelligence. *The 2025 AI Index Report: Technical Performance*. 2025. URL: `https://hai.stanford.edu/ai-index/2025-ai-index-report/technical-performance` (visited on 2026-10-11) (cit. on pp. 1, 14, 17).

[7] NYU Shanghai RITS. *OpenAI claims a Navier–Stokes proof, amid a dispute over credit*. Sept. 9, 2026. URL: `https://rits.shanghai.nyu.edu/ai/openai-navier-stokes-proof-credit-dispute/` (visited on 2026-10-11) (cit. on pp. 1, 11).

[8] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, et al. "RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. on pp. 2, 4, 35, 54, 55).

[9] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, et al. "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions". In: *ACM Transactions on Information Systems* (2025). DOI: 10.1145/3703155 (cit. on pp. 2, 35).

[10] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, et al. "Efficiently Scaling Transformer Inference". In: *Proceedings of Machine Learning and Systems (MLSys)*. 2023 (cit. on pp. 3, 18, 19, 42).

[11] Yuheng Zha, Yichi Yang, Ruichen Li, and Zhiting Hu. "AlignScore: Evaluating Factual Consistency with a Unified Alignment Function". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023 (cit. on pp. 3, 36, 83).

[12] Liyan Tang, Philippe Laban, and Greg Durrett. "MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents". In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2024 (cit. on pp. 3, 36, 54–56, 69).

[13] Vectara. *HHEM-2.1-Open: Hughes Hallucination Evaluation Model*. 2024. URL: https://huggingface.co/vectara/hallucination_evaluation_model (cit. on pp. 3, 36, 69).

[14] TypeSafe. *Introducing System One Models and Jev*. Blog post. 2026. URL: https://typesafe.ai/blog/introducing-system-one-models-and-jev (cit. on pp. 3, 32, 47).

[15] TheoLeeCJ. *SemIf: open option-logit System One baseline*. 2026. URL: https://github.com/TheoLeeCJ/SemIf (cit. on pp. 3, 4, 33).

[16] Simone Rizzo. *Rizzo Flow: typed decisions from an LLM, without generating a single token*. 2026. URL: https://github.com/Rizzo-AI-Academy/rizzo-flow (cit. on pp. 3, 4, 33, 68).

[17] Hugging Face. *Models tagged decision-model*. URL: https://huggingface.co/models?other=decision-model (visited on 2026-10-10) (cit. on pp. 4, 33).

[18] Liquid AI. *d1-3B: a decision model built on LFM2.5-VL-3B (model card, LiquidAI/d1-3B, revision 051bcc4)*. URL: https://huggingface.co/LiquidAI/d1-3B (visited on 2026-10-10) (cit. on pp. 4, 34, 68).

[19] Cloudflare. *Clef: open decision models (model card, Cloudflare/clef)*. URL: https://huggingface.co/Cloudflare/clef (visited on 2026-10-10) (cit. on pp. 4, 34).

[20] Tobias Deußer, Lorenz Sparrenberg, and Rafet Sifa. *Evaluating and Benchmarking the System One Model Jev*. 2026. arXiv: 2609.37647 (cit. on pp. 4, 33, 48, 83).

[21] Gowthamkumar Nandakishore. *Laya as a Typed Probabilistic Assessor: An Independent Reproduction and a Preregistered Study of Calibration and Selective Escalation*. 2026. arXiv: 2609.33843 (cit. on pp. 4, 33).

[22] multimodalart. *Jev Decision Index: a leaderboard of System One decision models (Hugging Face Space, v0.2.1)*. URL: https://huggingface.co/spaces/multimodalart/jev-decision-index (visited on 2026-10-10) (cit. on pp. 4, 34, 80).

[23] Qwen Team. *Qwen2.5 Technical Report*. 2024. arXiv: 2412.15115 (cit. on pp. 4, 14, 38, 44, 65, 122).

[24] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations (ICLR)*. 2022 (cit. on pp. 4, 38, 44).

[25] Guanqiao Qu, Qiyuan Chen, Wei Wei, Zheng Lin, Xianhao Chen, and Kaibin Huang. "Mobile Edge Intelligence for Large Language Models: A Contemporary Survey". In: *IEEE Communications Surveys & Tutorials* 27.6 (2025), pp. 3820–3860. arXiv: 2407.18921 (cit. on pp. 11, 12).

[26] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)*. Official Journal of the European Union, L 119, 4 May 2016, pp. 1–88. 2016. URL: https://eur-lex.europa.eu/eli/reg/2016/679/oj (cit. on p. 11).

[27] Jeremy Kahn. *OpenAI says it cracked a Navier–Stokes math grand challenge; a mathematician raises accusations (Eye on AI)*. Fortune. Sept. 8, 2026. URL: https://fortune.com/2026/09/08/openai-says-it-cracked-navier-stokes-math-grand-challenge-buckmaster-accusation-cheating-intimidation-tao-lament/ (visited on 2026-10-11) (cit. on p. 11).

[28] Larissa G. Capella. *'Why would you ruin your career?': OpenAI claims to have cracked one of math's greatest unsolved problems, but mathematicians allege it played dirty*. Live Science. Sept. 9, 2026. URL: https://livescience.com/physics-mathematics/mathematics/why-would-you-ruin-your-career-openai-claims-to-have-cracked-one-of-maths-greatest-unsolved-problems-but-mathematicians-allege-it-played-dirty (visited on 2026-10-11) (cit. on p. 11).

[29] Apple. *Private Cloud Compute: A new frontier for AI privacy in the cloud*. June 10, 2024. URL: https://security.apple.com/blog/private-cloud-compute/ (visited on 2026-10-11) (cit. on pp. 12, 13).

[30] Senyao Li, Haozhao Wang, Wenchao Xu, Rui Zhang, Song Guo, Jingling Yuan, Xian Zhong, Tianwei Zhang, and Ruixuan Li. *Collaborative Inference and Learning between Edge SLMs and Cloud LLMs: A Survey of Algorithms, Execution, and Open Challenges*. 2025. arXiv: 2507.16731 (cit. on pp. 12, 24).

[31] Open Source Initiative. *The Open Source AI Definition, version 1.0*. Oct. 28, 2024. URL: https://opensource.org/ai/open-source-ai-definition (visited on 2026-10-11) (cit. on p. 14).

[32] Artificial Analysis. *Introducing the Artificial Analysis Openness Index*. Dec. 1, 2025. URL: https://artificialanalysis.ai/articles/announcing-artificial-analysis-openness-index (visited on 2026-10-11) (cit. on p. 14).

[33] Rishi Bommasani, Alice Hau, Kevin Klyman, and Percy Liang. *Foundation models under the EU AI Act*. Stanford Center for Research on Foundation Models. Aug. 1, 2024. URL: https://crfm.stanford.edu/2024/08/01/eu-ai-act.html (visited on 2026-10-11) (cit. on p. 14).

[34] Epoch AI. *Open-weight models lag the closed frontier by about four months on the Epoch Capabilities Index*. May 29, 2026. URL: https://epoch.ai/data-insights/open-closed-eci-gap (visited on 2026-10-11) (cit. on pp. 14, 15).

[35] Artificial Analysis. *Artificial Analysis Intelligence Index*. Leaderboard, version 4.3.2; snapshot of 20 October 2026. URL: https://artificialanalysis.ai/leaderboards/models (visited on 2026-10-20) (cit. on pp. 14, 16).

[36] JustVugg. *colibri: run frontier MoE models on hardware you already own, experts streamed from disk*. URL: https://github.com/JustVugg/colibri (visited on 2026-10-13) (cit. on pp. 15, 26, 28).

[37] Venkat Somala and Luke Emberson. *What's the best model you can run on a single consumer GPU?* Epoch AI. Sept. 9, 2025. URL: https://epochai.substack.com/p/whats-the-best-model-you-can-run (visited on 2026-10-11) (cit. on p. 17).

[38] Zhenyan Lu, Xiang Li, Dongqi Cai, Rongjie Yi, Fangming Liu, Xiwen Zhang, Nicholas D. Lane, and Mengwei Xu. *Small Language Models: Survey, Measurements, and Insights*. 2024. arXiv: 2409.15790 (cit. on p. 17).

[39] Zechun Liu et al. "MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases". In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. Vol. 235. PMLR. 2024 (cit. on pp. 17, 30).

[40] Google. *Introducing Gemma 3n: the developer guide*. May 20, 2025. URL: https://developers.googleblog.com/en/introducing-gemma-3n/ (visited on 2026-10-11) (cit. on p. 17).

[41] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. 2023 (cit. on p. 18).

[42] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, et al. "Splitwise: Efficient Generative LLM Inference Using Phase Splitting". In: *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. 2024 (cit. on pp. 18, 23, 30, 42, 95).

[43] Samuel Williams, Andrew Waterman, and David Patterson. "Roofline: An Insightful Visual Performance Model for Multicore Architectures". In: *Communications of the ACM* 52.4 (2009), pp. 65–76. DOI: 10.1145/1498765.1498785 (cit. on pp. 19, 20).

[44] Zhihang Yuan et al. *LLM Inference Unveiled: Survey and Roofline Model Insights*. 2024. arXiv: 2402.16363 (cit. on p. 19).

[45] Apple. *MacBook Pro (14-inch, M4, 2024) – Technical Specifications*. URL: https://support.apple.com/en-us/121552 (visited on 2026-10-10) (cit. on pp. 19, 24, 65).

[46] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, et al. "LLM in a flash: Efficient Large Language Model Inference with Limited Memory". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. on pp. 20, 28).

[47] Apple. *MacBook Pro – Technical Specifications (M5, M5 Pro and M5 Max)*. URL: https://www.apple.com/macbook-pro/specs/ (visited on 2026-10-11) (cit. on p. 24).

[48] Apple. *Mac mini – Technical Specifications*. URL: https://www.apple.com/mac-mini/specs/ (visited on 2026-10-11) (cit. on pp. 22, 24).

[49] Apple. *Mac Studio (2025) – Technical Specifications*. URL: https://support.apple.com/en-us/122211 (visited on 2026-10-11) (cit. on pp. 23, 24).

[50] NVIDIA. *DGX Spark User Guide: Hardware Overview*. Sept. 10, 2026. URL: https://docs.nvidia.com/dgx/dgx-spark/hardware.html (visited on 2026-10-11) (cit. on pp. 23, 24).

[51] Mark Hachman. *Framework Desktop review*. PCWorld. Aug. 7, 2025. URL: https://www.pcworld.com/article/2866400/framework-desktop-review.html (visited on 2026-10-11) (cit. on pp. 22, 24).

[52] NVIDIA. *Compare GeForce Graphics Cards: specifications*. URL: https://www.nvidia.com/en-us/geforce/graphics-cards/compare/ (visited on 2026-10-11) (cit. on p. 24).

[53] EXO Labs. *NVIDIA DGX Spark and Mac Studio: disaggregated prefill and decode*. 2025. URL: https://blog.exolabs.net/nvidia-dgx-spark/ (visited on 2026-10-11) (cit. on p. 23).

[54] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. *PowerInfer-2: Fast Large Language Model Inference on a Smartphone*. 2024. arXiv: 2406.06282 (cit. on p. 24).

[55] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. "Fast On-device LLM Inference with NPUs". In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '25)*. ACM, 2025. DOI: 10.1145/3669940.3707239 (cit. on pp. 24, 25, 30).

[56] Apple Machine Learning Research. *Exploring LLMs with MLX and the Neural Accelerators in the M5 GPU*. Nov. 19, 2025. URL: https://machinelearning.apple.com/research/exploring-llms-mlx-m5 (visited on 2026-10-11) (cit. on p. 25).

[57] Subhransu Das, Jiaming Cheng, Arnav Kumar, Sadia Afrose, Mingzhe Han, Michael Silagy, Shreya Palande, Brijesh Soni, and Rajiv Ramnath. *Large Models for Small Devices: Edge AI Deployment*. 2026. arXiv: 2608.15693 (cit. on p. 25).

[58] Apple. *mlx-lm: LLMs with MLX*. 2024. URL: https://github.com/ml-explore/mlx-lm (cit. on pp. 26, 29, 51).

[59] Varun Rajesh, Om Jodhpurkar, Pooja Anbuselvan, Mantinder Singh, Ashok Jallepali, Shantanu Godbole, Pradeep Kumar Sharma, and Hritvik Shrivastava. *Production-Grade Local LLM Inference on Apple Silicon: A Comparative Study of MLX, MLC-LLM, Ollama, llama.cpp, and PyTorch MPS*. 2025. arXiv: 2511.05502 (cit. on p. 26).

[60] Stefanos Laskaridis, Kleomenis Katevas, Lorenzo Minto, and Hamed Haddadi. "MELTing Point: Mobile Evaluation of Language Transformers". In: *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*. 2024. arXiv: 2403.12844 (cit. on pp. 26, 30).

[61] Salvatore Sanfilippo. *DwarfStar (ds4): DeepSeek 4 Flash and PRO local inference engine for Metal, CUDA and ROCm*. URL: https://github.com/antirez/ds4 (visited on 2026-10-13) (cit. on p. 26).

[62] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 35. 2022 (cit. on pp. 29, 30).

[63] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers". In: *International Conference on Learning Representations (ICLR)*. 2023 (cit. on pp. 29, 30).

[64] Ji Lin et al. "AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration". In: *Proceedings of Machine Learning and Systems (MLSys)*. 2024 (cit. on p. 29).

[65] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023 (cit. on pp. 29, 38, 45).

[66] Yaniv Leviathan, Matan Kalman, and Yossi Matias. "Fast Inference from Transformers via Speculative Decoding". In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. Vol. 202. PMLR. 2023 (cit. on p. 30).

[67] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, et al. "From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference". In: *IEEE High Performance Extreme Computing Conference (HPEC)*. 2023 (cit. on pp. 30, 95).

[68] Alexandra Sasha Luccioni, Yacine Jernite, and Emma Strubell. "Power Hungry Processing: Watts Driving the Cost of AI Deployment?" In: *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. 2024 (cit. on pp. 30, 95).

[69] Jacky Kwok, Hangoo Kang, Tarun Suresh, Jon Saad-Falcon, Marco Pavone, Christopher Ré, and Azalia Mirhoseini. *Contrastive Language Models: A System One Model for Fast and Generalizable Decision-Making*. Blog post, not peer-reviewed. 2026. URL: `https://contrastive-lm.notion.site` (cit. on p. 33).

[70] Convai Innovations. *Laya: a System One decision model (model card, convaiinnovations/laya)*. URL: `https://huggingface.co/convaiinnovations/laya` (visited on 2026-10-10) (cit. on p. 33).

[71] Seyedarmin Azizi, Erfan Baghaei Potraghloo, and Massoud Pedram. *Labels Override Definitions in Jev-Style Typed Decision Models*. 2026. arXiv: `2610.02586` (cit. on p. 33).

[72] Cho Yin Yong. *DecideBench (dataset card, choyiny/decidebench)*. URL: https://huggingface.co/datasets/choyiny/decidebench (visited on 2026-10-10) (cit. on p. 33).

[73] Perplexity. *pplx-decider-v1.1-27b (model card)*. URL: https://huggingface.co/perplexity-ai/pplx-decider-v1.1-27b (visited on 2026-10-10) (cit. on p. 34).

[74] Ruoqi Guo, Yi Liu, Gelei Deng, Yuekang Li, et al. *Just Ask Jev: Reinforcement Learning for Calibrated Decisions as a Zero-Shot Detector of AI Alignment Failures*. 2026. arXiv: 2609.29429 (cit. on pp. 34, 48).

[75] Yubo Li, Yidi Miao, Ramayya Krishnan, and Rema Padman. *JEV-as-a-Judge: Accept When Confident, Escalate When Unsure*. 2026. arXiv: 2609.26550 (cit. on pp. 34, 48).

[76] Xinpeng Wang, Bolei Ma, Chengzhi Hu, Leon Weber-Genzel, et al. ""My Answer is C": First-Token Probabilities Do Not Match Text Answers in Instruction-Tuned Language Models". In: *Findings of the Association for Computational Linguistics: ACL 2024*. 2024. arXiv: 2402.14499 (cit. on pp. 34, 44).

[77] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. "Large Language Models Are Not Robust Multiple Choice Selectors". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. on pp. 34, 44).

[78] Adina Williams, Nikita Nangia, and Samuel R. Bowman. "A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference". In: *Proceedings of NAACL-HLT 2018*. 2018 (cit. on p. 35).

[79] Yixin Nie, Adina Williams, Emily Dinan, Mohit Bansal, Jason Weston, and Douwe Kiela. "Adversarial NLI: A New Benchmark for Natural Language Understanding". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020 (cit. on pp. 35, 55).

[80] Tal Schuster, Adam Fisch, and Regina Barzilay. "Get Your Vitamin C! Robust Fact Verification with Contrastive Evidence". In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021, pp. 624–643. DOI: 10.18653/v1/2021.naacl-main.52. URL: https://aclanthology.org/2021.naacl-main.52 (cit. on pp. 35, 54, 55).

[81] Philippe Laban, Tobias Schnabel, Paul N. Bennett, and Marti A. Hearst. "SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization". In: *Transactions of the Association for Computational Linguistics* 10 (2022), pp. 163–177 (cit. on p. 35).

[82] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, et al. "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation". In: *Proceedings of EMNLP 2023*. 2023 (cit. on p. 35).

[83] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena". In: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. 2023 (cit. on p. 36).

[84] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. "RAGAs: Automated Evaluation of Retrieval Augmented Generation". In: *Proceedings of EACL 2024: System Demonstrations*. 2024 (cit. on p. 36).

[85] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems". In: *Proceedings of NAACL 2024*. 2024 (cit. on p. 36).

[86] Selvan Sunitha Ravi, Bartosz Mielczarek, Anand Kannappan, Douwe Kiela, and Rebecca Qian. *Lynx: An Open Source Hallucination Evaluation Model*. 2024. arXiv: 2407.08488 (cit. on p. 36).

[87] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. on p. 36).

[88] Brandon T. Willard and Rémi Louf. *Efficient Guided Generation for Large Language Models*. 2023. arXiv: 2307.09702 (cit. on pp. 36, 68).

[89] Liyan Tang, Philippe Laban, and Greg Durrett. *LLM-AggreFact Leaderboard*. 2024. URL: https://llm-aggrefact.github.io/ (visited on 2026-10-05) (cit. on pp. 36, 54, 55).

[90] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. "On Calibration of Modern Neural Networks". In: *International Conference on Machine Learning (ICML)*. 2017 (cit. on pp. 36, 47, 48).

[91] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. "Obtaining Well Calibrated Probabilities Using Bayesian Binning". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2015 (cit. on pp. 36, 48).

[92] Tony Z. Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. "Calibrate Before Use: Improving Few-Shot Performance of Language Models". In: *International Conference on Machine Learning (ICML)*. 2021 (cit. on pp. 36, 47).

[93] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, et al. *Language Models (Mostly) Know What They Know*. 2022. arXiv: 2207.05221 (cit. on p. 37).

[94] Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, et al. "Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback". In: *Proceedings of EMNLP 2023*. 2023 (cit. on pp. 37, 87).

[95] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, et al. "Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. on pp. 37, 87).

[96] Yonatan Geifman and Ran El-Yaniv. "Selective Classification for Deep Neural Networks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017 (cit. on pp. 37, 70).

[97] Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah Smith. *Fine-Tuning Pretrained Language Models: Weight Initializations, Data Orders, and Early Stopping*. 2020. arXiv: 2002.06305 (cit. on pp. 38, 66).

[98] Marius Mosbach, Maksym Andriushchenko, and Dietrich Klakow. "On the Stability of Fine-tuning BERT: Misconceptions, Explanations, and Strong Baselines". In: *International Conference on Learning Representations (ICLR)*. 2021 (cit. on pp. 38, 78, 122).

[99] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. "On the Difficulty of Training Recurrent Neural Networks". In: *International Conference on Machine Learning (ICML)*. 2013 (cit. on pp. 39, 78, 122).

[100] Pritam Deka. *From Retrieval to Typed Decisions: Calibrated System One Models from Biomedical Sentence Encoders*. 2026. arXiv: 2610.02486 (cit. on p. 48).

[101] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. *MS MARCO: A Human Generated MAchine Reading COmprehension Dataset*. CoCo Workshop at NIPS 2016. 2016. arXiv: 1611.09268 (cit. on p. 54).

[102] Karl Moritz Hermann, Tomáš Kočiský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. "Teaching Machines to Read and Comprehend". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2015 (cit. on p. 54).

[103] Abigail See, Peter J. Liu, and Christopher D. Manning. "Get To The Point: Summarization with Pointer-Generator Networks". In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2017 (cit. on p. 54).

[104] Yelp. *Yelp Open Dataset*. URL: https://business.yelp.com/data/resources/open-dataset/ (visited on 2026-10-05) (cit. on pp. 54, 73).

[105] Liyan Tang, Tanya Goyal, Alex Fabbri, Philippe Laban, Jiacheng Xu, Semih Yavuz, Wojciech Kryściński, Justin Rousseau, and Greg Durrett. "Understanding Factual Errors in Summarization: Errors, Summarizers, Datasets, Error Detectors". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023 (cit. on p. 55).

[106] Liyan Tang, Igor Shalyminov, Amy Wing-mei Wong, et al. "TofuEval: Evaluating Hallucinations of LLMs on Topic-Focused Dialogue Summarization". In: *Proceedings of NAACL 2024*. 2024 (cit. on p. 55).

[107] Nelson F. Liu, Tianyi Zhang, and Percy Liang. "Evaluating Verifiability in Generative Search Engines". In: *Findings of EMNLP 2023*. 2023 (cit. on p. 55).

[108] Chaitanya Malaviya, Subin Lee, Sihao Chen, Elizabeth Sieber, Mark Yatskar, and Dan Roth. "ExpertQA: Expert-Curated Questions and Attributed Answers". In: *Proceedings of NAACL 2024*. 2024 (cit. on p. 55).

[109] Hung-Ting Chen, Fangyuan Xu, Shane Arora, and Eunsol Choi. "Understanding Retrieval Augmentation for Long-Form Question Answering". In: *Conference on Language Modeling (COLM)*. 2024 (cit. on p. 55).

[110] Ryo Kamoi, Tanya Goyal, Juan Diego Rodriguez, and Greg Durrett. "WiCE: Real-World Entailment for Claims in Wikipedia". In: *Proceedings of EMNLP 2023*. 2023 (cit. on p. 55).

[111] Alon Jacovi, Yonatan Bitton, Bernd Bohnet, Jonathan Herzig, Or Honovich, Michael Tseng, Michael Collins, Roee Aharoni, and Mor Geva. "A Chain-of-Thought Is as Strong as Its Weakest Link: A Benchmark for Verifiers of Reasoning Chains". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. on p. 55).

[112] Yuxia Wang, Revanth Gangi Reddy, Zain Muhammad Mujahid, et al. "Factcheck-Bench: Fine-Grained Evaluation Benchmark for Automatic Fact-checkers". In: *Findings of EMNLP 2024*. 2024 (cit. on p. 55).

[113] Liyan Tang. *LLM-AggreFact dataset card (lytang/LLM-AggreFact)*. URL: https://huggingface.co/datasets/lytang/LLM-AggreFact (visited on 2026-10-05) (cit. on pp. 55, 73).

[114] Tal Schuster. *VitaminC dataset card (tals/vitaminc)*. URL: https://huggingface.co/datasets/tals/vitaminc (visited on 2026-10-06) (cit. on p. 55).

[115] Nipun Sadvilkar and Mark Neumann. "PySBD: Pragmatic Sentence Boundary Disambiguation". In: *Proceedings of the Second Workshop for NLP Open Source Software (NLP-OSS)*. 2020, pp. 110–114. DOI: 10.18653/v1/2020.nlposs-1.15. URL: https://aclanthology.org/2020.nlposs-1.15 (cit. on p. 56).

[116] Boxi Yu, Yuzhong Zhang, Liting Lin, Lionel Briand, and Emir Muñoz. *Retromorphic Testing with Hierarchical Verification for Hallucination Detection in RAG*. 2026. arXiv: 2603.27752. URL: https://arxiv.org/abs/2603.27752 (cit. on pp. 61, 123).

[117] Philipp Koehn. "Statistical Significance Tests for Machine Translation Evaluation". In: *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2004, pp. 388–395. URL: https://aclanthology.org/W04-3250 (cit. on p. 70).

[118] Taylor Berg-Kirkpatrick, David Burkett, and Dan Klein. "An Empirical Investigation of Statistical Significance in NLP". In: *Proceedings of EMNLP-CoNLL 2012*. 2012, pp. 995–1005. URL: https://aclanthology.org/D12-1091 (cit. on p. 70).

[119] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. "The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing". In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2018, pp. 1383–1392. URL: https://aclanthology.org/P18-1128 (cit. on p. 70).

[120] Alibaba Cloud. *Qwen Research License Agreement*. Sept. 19, 2024. URL: https://huggingface.co/Qwen/Qwen2.5-3B-Instruct/blob/main/LICENSE (visited on 2026-10-22) (cit. on p. 73).

[121] Liquid AI. *LFM Open License v1.0*. URL: https://huggingface.co/LiquidAI/d1-3B/blob/main/LICENSE (visited on 2026-10-22) (cit. on p. 73).

[122] Microsoft. *MS MARCO: terms and conditions*. URL: https://microsoft.github.io/msmarco/ (visited on 2026-10-22) (cit. on p. 73).

[123] Apple ML Explore. *MLX Swift*. URL: https://github.com/ml-explore/mlx-swift (visited on 2026-10-18) (cit. on p. 99).

[124] Apple ML Explore. *MLX Swift LM: LLMs and VLMs with MLX Swift*. Version 3.32.3. URL: https://github.com/ml-explore/mlx-swift-lm (visited on 2026-10-18) (cit. on p. 99).

[125] oxiglade. *mlx-rs: unofficial Rust bindings to Apple's MLX*. Version 0.32.0. URL: https://github.com/oxiglade/mlx-rs (visited on 2026-10-18) (cit. on p. 99).

[126] Hugging Face. *swift-transformers: Swift package for tokenizers and model loading*. Version 1.3.5. URL: https://github.com/huggingface/swift-transformers (visited on 2026-10-24) (cit. on p. 101).

[127] Apple. *NLTokenizer*. Apple Developer Documentation, Natural Language framework. URL: https://developer.apple.com/documentation/naturallanguage/nltokenizer (visited on 2026-10-18) (cit. on p. 101).

[128] SQLite. *SQLite FTS5 extension*. URL: https://www.sqlite.org/fts5.html (visited on 2026-10-18) (cit. on p. 103).

[129] Stephen Robertson and Hugo Zaragoza. "The Probabilistic Relevance Framework: BM25 and Beyond". In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389 (cit. on p. 103).

[130] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. "C-Pack: Packed Resources For General Chinese Embeddings". In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2024, pp. 641–649 (cit. on p. 103).

[131] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods". In: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2009, pp. 758–759 (cit. on p. 103).

[132] Luyu Gao et al. "RARR: Researching and Revising What Language Models Say, Using Language Models". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2023, pp. 16477–16508 (cit. on p. 106).

[133] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. "CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing". In: *International Conference on Learning Representations*. 2024 (cit. on p. 106).

[134] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. "Large Language Models Cannot Self-Correct Reasoning Yet". In: *International Conference on Learning Representations*. 2024 (cit. on p. 106).

[135] Eurostat. *Electricity price statistics*. Statistics Explained; household prices, second half of 2025. URL: https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Electricity_price_statistics (visited on 2026-10-19) (cit. on p. 120).

[136] Anthropic. *Claude API pricing*. URL: https://platform.claude.com/docs/en/about-claude/pricing (visited on 2026-10-19) (cit. on p. 120).

[137] OpenAI. *API pricing*. URL: https://developers.openai.com/api/docs/pricing (visited on 2026-10-19) (cit. on p. 120).

[138] Google. *Gemini API pricing*. URL: https://ai.google.dev/gemini-api/docs/pricing (visited on 2026-10-19) (cit. on p. 120).